

## Projektarbeit

---

# *Stoneforge*: Ein prozedural generiertes 2D-Spiel als Testumgebung für Reinforcement Learning

Spielentwicklung, Weltgenerator und die Generalisierung  
eines rekurrenten Navigationsagenten

---

**Vorgelegt von:** Florian Merlau  
Laurin Rößler

**Studiengang:** Masterstudiengang Informatik und Security (M.Sc.)

**Betreuer:** Prof. Dr. Carsten Leon

**Abgabedatum:** 14. August 2026

## Kurzfassung

Diese Projektarbeit verbindet zwei Themen, die auf den ersten Blick wenig miteinander zu tun haben: die prozedurale Erzeugung von Spielwelten und Reinforcement Learning unter partieller Beobachtbarkeit. Im ersten Teil bauen wir mit *Stoneforge* ein Spiel, dessen Generator reproduzierbare, nachweislich lösbare zweidimensionale Welten mit frei steuerbarer Zielentfernung erzeugt. Jede Karte ist lösbar. Jeder Seed reproduzierbar. Im zweiten Teil stellen wir eine unbequeme Frage: Bildet ein Agent, der nur einen kleinen lokalen Ausschnitt seiner Umgebung sieht, wirklich eine Navigationsstrategie, oder lernt er bloß die trainierten Karten auswendig? Wir testen ihn dort, wo es zählt: auf Welten, die er nie gesehen hat.

Der Generator erfüllt seine Anforderungen vollständig. Determinismus bis hin zu bitidentischen Trainingspfaden, 100 % Lösbarkeit über 3 150 geprüfte Seeds, eine Zielentfernung, die am tatsächlichen Laufweg gemessen exakt eingehalten wird. Der Agent, ein rekurrentes Verfahren mit internem Gedächtnis, erreicht über sieben unabhängige Läufe  $65,7\% \pm 12,4$  auf dem Testset und  $66,9\% \pm 12,8$  auf einem gesperrten Holdout. Die Holdoutschwelle ist erfüllt, die Testsetschwelle verfehlt.

Diese Zahlen tragen nicht. Eine ungelernete Kompassheuristik aus vier Zeilen Code erreicht auf derselben Aufgabe 92 % und ist bei vergleichbarer Erfolgsquote zugleich dreimal wegeffizienter. Die Erfolgsquote misst hier also nicht die Fähigkeit, um die es geht. Die Ursache liegt nicht im Agenten, sondern im Generator: Solange die Hindernisse aus unkorreliertem Rauschen stammen, entstehen kaum Sackgassen, und ohne Sackgassen erzwingt die Aufgabe kein Gedächtnis. Die zweite Forschungsfrage ist mit diesem Aufbau nicht entscheidbar.

Daraus folgt der übertragbare Beitrag. Die Schwierigkeit einer Lernaufgabe ist eine Eigenschaft der Weltgenerierung und gehört dort spezifiziert und geprüft, bevor ein Lernverfahren darauf angesetzt wird. Eine ungelernete Referenzpolitik leistet genau das, in Sekunden statt in Trainingsstunden.

**Schlagwörter:** Prozedurale Weltgenerierung, Spielentwicklung, Reinforcement Learning, POMDP, Generalisierung, RecurrentPPO, Curriculum Learning, Baseline-Vergleich

## Eidesstattliche Erklärung

Wir versichern hiermit, dass wir die vorliegende Projektarbeit mit dem Thema „*Stoneforge*: Ein prozedural generiertes 2D-Spiel als Testumgebung für Reinforcement Learning“ selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben. Alle Stellen, die wörtlich oder sinngemäß aus Veröffentlichungen oder anderen Quellen entnommen sind, sind als solche kenntlich gemacht. Die Arbeit wurde in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegt.

Die eingesetzten generativen KI-Werkzeuge sind auf der folgenden Seite in der „Erklärung zur Nutzung generativer KI“ vollständig ausgewiesen; die übrigen Hilfsmittel (verwendete Software und Bibliotheken, Literatur) sind an den jeweiligen Stellen dieser Arbeit angegeben. Der vollständige Quellcode sowie das lückenlos geführte Experiment-Changelog liegen dem Projekt-Repository bei.

---

Ort, Datum

---

Florian Merlau

---

Ort, Datum

---

Laurin Rößler

# Erklärung zur Nutzung generativer KI

Bei der Erstellung dieser Arbeit wurden generative KI-Werkzeuge als Hilfsmittel eingesetzt; Art und Umfang sind in der folgenden Tabelle offengelegt. Wir verantworten alle daraus übernommenen Inhalte vollumfänglich selbst, haben sie inhaltlich nachvollzogen und gegen die Primärdaten des Projekt-Repositories verifiziert. **Nicht** eingesetzt wurde generative KI zur Erzeugung von Messergebnissen: Sämtliche berichteten Zahlen stammen aus eigenen Trainings- und Evaluationsläufen und sind im Experiment-Changelog sowie über Seeds, Modell-Checkpoints und Konfigurationsdateien reproduzierbar.

## Übersicht der verwendeten KI-Werkzeuge

| Werkzeug                          | Einsatzform                                                                                                                                                     | Teile der Arbeit                                                                             | Bemerkungen                                                                                                                                                                                                                                                                                                                 |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Claude (Opus 5), Anthropic        | Vorschläge zur Prüfung des Manuskripts auf Rechen-, Zahlen- und Konsistenzfehler sowie zur Formulierung des Diagnoseskripts <code>reward_bilanz.py</code>       | Kapitel 3–5                                                                                  | Jeder Vorschlag wurde von uns gegen Quellcode, <code>eval_history.json</code> und <code>CHANGELOG.md</code> geprüft und erst nach eigener Verifikation übernommen; drei Beanstandungen erwiesen sich dabei als unzutreffend und wurden verworfen. Die Messwerte des Diagnoseskripts stammen aus eigenen Läufen.             |
| Claude Code (Opus 4.8), Anthropic | Textentwürfe und -überarbeitungen nach unseren Vorgaben (Gliederung, Kürzung, sprachlicher Stil); Vorschläge für Literaturbelege; Anpassungen am Dokumentlayout | Kurzfassung, Kapitel 1–2, Abschnitt 3.2, Abschnitt 4.2, Abschnitte in Kapitel 5–6, Deckblatt | Struktur, Aussagen und alle berichteten Ergebnisse legten wir fest; Claude lieferte Formulierungs- und Kürzungsvorschläge sowie Belegvorschläge, die wir gegen <code>references.bib</code> und die Originalquellen geprüft haben. Layoutänderungen (Logo, Farben, Abstände) sind rein optisch und ohne inhaltliche Wirkung. |

---

Ort, Datum

---

Florian Merlau

---

Ort, Datum

---

Laurin Rößler

# Inhaltsverzeichnis

|                                                                            |             |
|----------------------------------------------------------------------------|-------------|
| <b>Kurzfassung</b>                                                         | <b>ii</b>   |
| <b>Eidesstattliche Erklärung</b>                                           | <b>iii</b>  |
| <b>Erklärung zur Nutzung generativer KI</b>                                | <b>iv</b>   |
| <b>Abbildungsverzeichnis</b>                                               | <b>vii</b>  |
| <b>Tabellenverzeichnis</b>                                                 | <b>viii</b> |
| <b>Abkürzungs- und Symbolverzeichnis</b>                                   | <b>ix</b>   |
| <b>1 Einleitung (gemeinsam)</b>                                            | <b>1</b>    |
| 1.1 Motivation . . . . .                                                   | 1           |
| 1.2 Die Aufgabe in Kürze . . . . .                                         | 1           |
| 1.3 Problemstellung und Forschungsfrage . . . . .                          | 1           |
| 1.4 Zielsetzung und Erfolgskriterien . . . . .                             | 2           |
| 1.5 Einordnung der Arbeit . . . . .                                        | 3           |
| 1.6 Aufbau der Arbeit . . . . .                                            | 5           |
| <b>2 Grundlagen (gemeinsam)</b>                                            | <b>7</b>    |
| 2.1 Grundlagen der Weltgenerierung (Laurin) . . . . .                      | 7           |
| 2.1.1 Begriff, Seed und Determinismus . . . . .                            | 7           |
| 2.1.2 Algorithmen der prozeduralen Weltgenerierung . . . . .               | 7           |
| 2.2 Grundlagen des Reinforcement Learning (Florian) . . . . .              | 8           |
| 2.2.1 Reinforcement Learning und der Markov-Entscheidungsprozess . . . . . | 8           |
| 2.2.2 Partielle Beobachtbarkeit (POMDP) . . . . .                          | 9           |
| 2.2.3 Wertbasierte Verfahren: Q-Learning und DQN . . . . .                 | 10          |
| 2.2.4 Policy-Gradient- und Actor-Critic-Verfahren . . . . .                | 10          |
| 2.2.5 Proximal Policy Optimization (PPO) . . . . .                         | 11          |
| 2.2.6 Rekurrente Politiken (LSTM) . . . . .                                | 11          |
| 2.2.7 Prozedurale Welten im Reinforcement Learning . . . . .               | 12          |
| 2.2.8 Reward Shaping (PBRS) . . . . .                                      | 13          |
| 2.2.9 Curriculum Learning . . . . .                                        | 13          |
| 2.2.10 Messen und Bewerten . . . . .                                       | 13          |
| 2.2.11 Metriken einer Navigationsaufgabe . . . . .                         | 14          |
| 2.2.12 Streuung, Konfidenz und die Zahl der Läufe . . . . .                | 14          |
| 2.2.13 Referenzpolitiken als Bezugspunkt . . . . .                         | 15          |
| <b>3 Methodik (gemeinsam)</b>                                              | <b>16</b>   |
| 3.1 Methodik der Weltgenerierung (Laurin) . . . . .                        | 16          |
| 3.2 Methodik des RL-Trainings (Florian) . . . . .                          | 16          |
| 3.2.1 Wahl des Lernverfahrens . . . . .                                    | 16          |
| 3.2.2 Curriculum . . . . .                                                 | 16          |
| 3.2.3 Evaluationsprotokoll . . . . .                                       | 17          |
| 3.2.4 Versuchsaufbau . . . . .                                             | 18          |
| <b>4 Implementierung (gemeinsam)</b>                                       | <b>20</b>   |
| 4.1 Implementierung des Spiels Stoneforge (Laurin) . . . . .               | 20          |
| 4.1.1 Technologie-Stack . . . . .                                          | 20          |
| 4.1.2 Die Bausteine: Tile, Chunk und Welt . . . . .                        | 20          |

|          |                                                                                                  |           |
|----------|--------------------------------------------------------------------------------------------------|-----------|
| 4.1.3    | Der Weltgenerator . . . . .                                                                      | 21        |
| 4.1.4    | Exit-Platzierung und garantierte Lösbarkeit . . . . .                                            | 29        |
| 4.1.5    | Grafischer Client und Zuschaumodus . . . . .                                                     | 31        |
| 4.2      | Implementierung der Lernumgebung (Florian) . . . . .                                             | 32        |
| 4.2.1    | Versionsverlauf . . . . .                                                                        | 32        |
| 4.2.2    | Was der Agent davon sieht: POMDP-Charakter . . . . .                                             | 32        |
| 4.2.3    | Umgebung, Beobachtung und Aktionen . . . . .                                                     | 32        |
| 4.2.4    | Die Schnittstelle: was bei <code>reset()</code> und <code>step()</code> tatsächlich passiert . . | 33        |
| 4.2.5    | Reward-Design . . . . .                                                                          | 34        |
| 4.2.6    | Trainings-Pipeline . . . . .                                                                     | 36        |
| 4.2.7    | Monitoring und Werkzeuge . . . . .                                                               | 37        |
| <b>5</b> | <b>Evaluation (gemeinsam)</b>                                                                    | <b>38</b> |
| 5.1      | Bewertung der Plattform (Z1, Laurin) . . . . .                                                   | 38        |
| 5.2      | Bewertung des Agenten (Z2, Florian) . . . . .                                                    | 39        |
| 5.2.1    | Zentrale Gegenüberstellung: Gedächtnis statt lokalem Orakel . . . . .                            | 39        |
| 5.2.2    | Endergebnis (v12, $n = 7$ ) . . . . .                                                            | 39        |
| 5.2.3    | Ungelernte Referenzpolitiken und die Grenzen der Erfolgsquote . . . .                            | 40        |
| 5.2.4    | Der Det/Stoch-Gap: Weglängen-Analyse und POMDP-Einordnung . .                                    | 43        |
| 5.2.5    | Versuche zur Schließung des Det/Stoch-Gaps . . . . .                                             | 44        |
| <b>6</b> | <b>Fazit und Ausblick</b>                                                                        | <b>46</b> |
| 6.1      | Ergebnis (gemeinsam) . . . . .                                                                   | 46        |
| 6.2      | Beiträge (gemeinsam) . . . . .                                                                   | 47        |
| 6.3      | Einschränkungen (gemeinsam) . . . . .                                                            | 47        |
| 6.4      | Ausblick . . . . .                                                                               | 48        |
| 6.4.1    | Härtung der Weltgenerierung (Laurin) . . . . .                                                   | 48        |
| 6.4.2    | Weiterentwicklung des RL-Trainings (Florian) . . . . .                                           | 48        |
|          | <b>Literaturverzeichnis</b>                                                                      | <b>50</b> |

## Abbildungsverzeichnis

|    |                                                                            |    |
|----|----------------------------------------------------------------------------|----|
| 1  | Überblick über das Vorgehen . . . . .                                      | 3  |
| 2  | Agent-Umwelt-Interaktion im MDP . . . . .                                  | 9  |
| 3  | MDP gegenüber POMDP . . . . .                                              | 9  |
| 4  | Partielle Beobachtbarkeit am Beispiel Stoneforge . . . . .                 | 10 |
| 5  | Wirkung des PPO-Clippings . . . . .                                        | 11 |
| 6  | Rekurrente Politik über die Zeit . . . . .                                 | 12 |
| 7  | Ablauf des leistungsbasierten Curriculums . . . . .                        | 17 |
| 8  | Hash-Rauschen gegenüber interpoliertem Wertrauschen . . . . .              | 23 |
| 9  | Die Generator-Pipeline an einer echten Welt . . . . .                      | 26 |
| 10 | Pipeline der Weltgenerierung je Chunk . . . . .                            | 27 |
| 11 | Benutzeroberfläche des spielbaren C++-Clients . . . . .                    | 31 |
| 12 | Luftlinien- gegenüber BFS-Potential an einer Sackgasse . . . . .           | 35 |
| 13 | Architektur: gemeinsamer Simulationskern für Training und Client . . . . . | 37 |
| 14 | Lernkurve des historischen Referenzlaufs . . . . .                         | 40 |
| 15 | Endergebnis v12 mit Streuung über sieben Läufe . . . . .                   | 41 |
| 16 | Trainingsverläufe der sieben v12-Läufe . . . . .                           | 42 |
| 17 | Erfolgsquote nach Startdistanz zum Exit . . . . .                          | 44 |

## Tabellenverzeichnis

|    |                                                                          |    |
|----|--------------------------------------------------------------------------|----|
| 1  | Vergleich der Lösbarkeits- und Schwierigkeitssteuerung . . . . .         | 4  |
| 2  | Arbeitsteilung . . . . .                                                 | 6  |
| 3  | Curriculum in vier Phasen . . . . .                                      | 17 |
| 4  | Gemeinsame Basiskonfiguration aller Läufe . . . . .                      | 19 |
| 5  | Technologie-Stack . . . . .                                              | 20 |
| 6  | Belegung einer Chunk-Zeile Feld für Feld . . . . .                       | 25 |
| 7  | Biomabhängige Schwellwerte der Basisbelegung . . . . .                   | 27 |
| 8  | Tile-Zusammensetzung der Eval-Welten . . . . .                           | 27 |
| 9  | Aktive Mechanismen der Weltgenerierung . . . . .                         | 28 |
| 10 | Regelsweep der zellulären Glättung . . . . .                             | 29 |
| 11 | Versionsverlauf von Umgebung und Training . . . . .                      | 32 |
| 12 | Aufbau der Observation (Env v11) . . . . .                               | 33 |
| 13 | Reward-Komponenten (Env v11) . . . . .                                   | 36 |
| 14 | Bewertung der Plattform gegen die Anforderungen Z1 . . . . .             | 38 |
| 15 | Gegenüberstellung: Gedächtnis gegenüber lokalem BFS-Orakel . . . . .     | 39 |
| 16 | Endergebnis der v12-Läufe über sieben Seeds . . . . .                    | 40 |
| 17 | Gating-Bilanz der sieben v12-Läufe . . . . .                             | 41 |
| 18 | Ungelernte Referenzpolitiken gegenüber dem trainierten Agenten . . . . . | 42 |
| 19 | Erfolgsquote als Funktion des Episodenbudgets . . . . .                  | 43 |
| 20 | Bilanz der dichten Reward-Terme . . . . .                                | 43 |
| 21 | Varianten zur Schließung des Det/Stoch-Gaps . . . . .                    | 45 |
| 22 | Temperatur-Sweep der Evaluation . . . . .                                | 45 |



## Abkürzungs- und Symbolverzeichnis

| Kürzel          | Bedeutung                                                                        |
|-----------------|----------------------------------------------------------------------------------|
| A2C             | Advantage Actor-Critic, synchrone Variante von A3C                               |
| approx_kl       | Maß für die Stärke einer Politikänderung pro Update                              |
| BFS             | <i>Breadth-First Search</i> , Breitensuche; liefert den kürzesten begehbaren Weg |
| BPTT            | <i>Backpropagation Through Time</i> , Lernverfahren rekurrenter Netze            |
| CNN             | <i>Convolutional Neural Network</i> , faltendes neuronales Netz                  |
| DQN             | <i>Deep Q-Network</i> , wertbasiertes RL-Verfahren                               |
| DRQN            | <i>Deep Recurrent Q-Network</i> , DQN mit Gedächtnis                             |
| EV              | <i>Explained Variance</i> , Gütemaß der gelernten Wertfunktion                   |
| GAE             | <i>Generalized Advantage Estimation</i> , Schätzung des Vorteils                 |
| HARKing         | <i>Hypothesizing After the Results are Known</i>                                 |
| LSTM            | <i>Long Short-Term Memory</i> , rekurrente Netzarchitektur (Gedächtnis)          |
| MDP             | <i>Markov Decision Process</i> , Markow-Entscheidungsprozess                     |
| MLP             | <i>Multilayer Perceptron</i> , vorwärtsgerichtetes Netz ohne Gedächtnis          |
| MPS             | <i>Metal Performance Shaders</i> , GPU-Backend von Apple                         |
| PBRs            | <i>Potential-Based Reward Shaping</i> , politikerhaltende Belohnungsformung      |
| PLR             | <i>Prioritized Level Replay</i> , priorisierte Level-Wiederholung                |
| POMDP           | <i>Partially Observable MDP</i> , teilweise beobachtbarer MDP                    |
| PPO             | <i>Proximal Policy Optimization</i> , das hier verwendete RL-Hauptverfahren      |
| RL              | <i>Reinforcement Learning</i> , bestärkendes Lernen                              |
| SB3             | <i>Stable-Baselines3</i> , verwendete RL-Bibliothek                              |
| SR              | <i>Success Rate</i> , Erfolgsquote: Anteil gelöster Welten                       |
| $\gamma$        | Diskontfaktor: Gewicht künftiger Belohnungen (hier 0,999)                        |
| $\epsilon$      | Clipping-Grenze von PPO (Abschnitt 2.2.5)                                        |
| $\varepsilon$   | Zufallsanteil der Vergleichspolitik (Abschnitt 5.2.3)                            |
| $\eta$          | Pfادهffizienz: kürzester Weg geteilt durch tatsächliche Schritte                 |
| $\pi(a \mid s)$ | Politik: Wahrscheinlichkeit, im Zustand $s$ die Aktion $a$ zu wählen             |
| $\Phi(s)$       | Potentialfunktion des Reward Shapings                                            |
| $\sigma$        | Standardabweichung über Trainingsläufe                                           |

# 1 Einleitung (gemeinsam)

## 1.1 Motivation

Bei der Entwicklung eines Spiels müssen dessen Welten entworfen werden, entweder einzeln von Hand oder über ein Regelwerk, das sie berechnet. Diese Regelwerke werden als prozedurale Content-Generierung bezeichnet und seit den 1980er-Jahren in Computerspielen eingesetzt [1]. Sie nehmen den Entwicklenden die Erstellung einzelner Welten ab, verlagern aber Fragen zu Vielfalt, Lösbarkeit und Schwierigkeit jeder einzelnen Welt in den Generator selbst [2].

Auf der anderen Seite steht das Reinforcement Learning (RL), ein Lernverfahren, bei dem eine Politik über Belohnungssignale aus der wiederholten Interaktion mit einer Umgebung entsteht [3]. Sowohl in der Lehre als auch in weiten Teilen der Forschung wird dabei auf fertig vorliegenden, meist deterministischen Umgebungen gearbeitet [4], weil deren Aufbau vom eigentlichen Lernverfahren ablenkt. Das führt dazu, dass die Eigenschaften einer Umgebung als gegeben hingenommen werden und ihr Anteil am Ergebnis unsichtbar bleibt [5].

Vor dieser Projektarbeit bestand bereits etwas Erfahrung in beiden Richtungen: einzelne experimentelle Projekte im Bereich der Spieleentwicklung und eine besuchte Lehrveranstaltung zum Reinforcement Learning. Daraus entstand der Wunsch, beide Themengebiete im Rahmen dieser Projektarbeit praktisch zu vertiefen und miteinander zu verknüpfen.

Dadurch, dass beide Gebiete unabhängig voneinander weit entwickelt wurden, ergibt sich die Frage nach ihrer Verbindung: ob es möglich ist, einen Agenten in einem selbst entwickelten Spiel (dem hier konzipierten *Stoneforge*) mit prozedural erzeugten Welten zu trainieren und dabei eine Strategie zu erhalten, die auch auf nie gesehenen Welten funktioniert.

Auswendiglernen der Trainingswelten und echtes Navigieren wirken von außen gleich erfolgreich; erst der Test auf trainingsfremden Welten unterscheidet beide Fälle [6].

## 1.2 Die Aufgabe in Kürze

Im Folgenden wird die Aufgabe knapp umrissen, bevor sie in den Implementierungskapiteln ausgearbeitet wird.

Der Agent startet in der Mitte einer Welt, die er nie zuvor gesehen hat, und soll deren Ausgang finden. Das zugrunde liegende Spiel heißt *Stoneforge* und läuft rundenbasiert ab. Aus einer Zufallszahl, dem *Seed*, entsteht eine Landschaft aus quadratischen Feldern: Wiese, Wald, Fels und Wasser. Gleicher Seed, gleiche Welt; anderer Seed, andere Welt. Pro Zug bewegt sich der Agent ein Feld nach oben, unten, links oder rechts, mehr steht ihm nicht zur Verfügung (siehe Abschnitt 4.2.3).

Schwierig wird die Aufgabe dadurch, dass die Welt nicht überblickt werden kann. Sichtbar ist nur ein kleiner Ausschnitt um die eigene Position, der Rest bleibt schwarz. Die Richtung zum Ausgang ist bekannt, vergleichbar mit einem Kompass, der Weg dorthin jedoch nicht. Liegt eine lange Felswand dazwischen, muss der Agent selbst herausfinden, ob sie nördlich oder südlich zu umgehen ist, und sich merken, wo er bereits gelaufen ist. Abbildung 4 auf Seite 10 stellt beide Sichten nebeneinander und zeigt, wie wenig vom Weltzustand in die Beobachtung eingeht.

Gelernt wird über Belohnung: Das Erreichen des Ausgangs bringt einen großen Pluspunkt, jeder Schritt einen kleinen Abzug. Nach Millionen Versuchen stellt sich ein Verhalten ein, bei dem am Ende möglichst viel Belohnung übrig bleibt. Gezählt wird zum Schluss, wie viele von 50 unbekannten Welten gelöst werden (siehe Abschnitt 3.2.3).

## 1.3 Problemstellung und Forschungsfrage

Der beschriebene Aufbau enthält zwei Probleme, die zusammenhängen. Sie werden zunächst getrennt benannt und anschließend zu zwei Forschungsfragen verdichtet (siehe Abschnitt 1.4).

**Das erste Problem betrifft den Agenten.** Da nur ein Ausschnitt sichtbar ist, sehen zwei verschiedene Orte für ihn gleich aus, weshalb er seine Lage aus dem bisher Gesehenen erschließen muss. Diese Leistung ließe sich ihm abnehmen, indem ihm der kürzeste Weg vorberechnet wird, ein „Orakel“. Die Aufgabe wäre damit gelöst, ohne dass etwas gelernt wurde, das ohne dieses Orakel trägt. Richtig gestellt ist die Aufgabe daher erst, wenn dieser Ausweg ausgeschlossen ist (siehe Abschnitt 4.2.5).

**Das zweite Problem betrifft den Generator.** Er soll abwechslungsreiche Welten liefern, weil sonst keine Generalisierung geprüft wird. Er soll lösbare Welten liefern, weil sonst die Erfolgsquote nichts aussagt. Und er soll angeben können, wie schwer eine Welt ist, weil sich das Training sonst nicht steuern lässt. Diese Forderungen ziehen auseinander, und bereits die letzte ist offen, da kein etabliertes Maß dafür existiert, woran die Schwierigkeit einer erzeugten Welt zu bemessen wäre (siehe Abschnitt 4.1.3).

Wie schwer die Aufgabe für den Agenten ausfällt, bestimmt daher der Generator. Aus dieser Abhängigkeit ergeben sich zwei Fragen, von denen die zweite auf der ersten aufbaut.

***F1** Wie lässt sich ein prozeduraler Weltgenerator so konstruieren, dass er abwechslungsreiche, nachweislich lösbare Welten mit präzise steuerbarer Schwierigkeit erzeugt und dabei als reproduzierbare Versuchsumgebung für Reinforcement Learning taugt?*

***F2** Kann ein Reinforcement-Learning-Agent in solchen, nur teilweise einsehbaren 2D-Welten eine Navigationsstrategie lernen, die auch auf unbekannten Welten funktioniert, ohne dass ihm der Weg zum Ziel vorberechnet wird?*

F2 legt sich absichtlich nicht auf eine Architektur fest, weil die Frage nach einem internen Gedächtnis Teil der Untersuchung ist und nicht ihre Voraussetzung (siehe Abschnitt 3.2).

## 1.4 Zielsetzung und Erfolgskriterien

Aus den beiden Fragen folgen zwei Sorten von Kriterien. Die erste legt fest, wann die Umgebung brauchbar ist, die zweite, wann der Agent gut genug ist. Die zweite Sorte ist nur sinnvoll, wenn die erste erfüllt ist, da ohne tragfähige Umgebung keine Aussage über den Agenten möglich wäre (siehe Kapitel 5).

**Z1: Anforderungen an Spiel und Generator.** Diese Kriterien sind erfüllt oder nicht. Sie werden an der jeweils angegebenen Stelle nachgewiesen und in Abschnitt 5.1 zusammengeführt.

- **Determinismus:** Gleicher Seed, gleiche Welt und gleicher Episodenverlauf, auch wenn Weltausschnitte in anderer Reihenfolge oder parallel erzeugt werden (Abschnitt 4.1.3).
- **Lösbarkeit:** Jede erzeugte Welt hat einen begehbaren Weg vom Start zum Ziel, nachgewiesen über alle verwendeten Seeds (Abschnitt 4.1.4).
- **Steuerbare Schwierigkeit:** Die Zielentfernung lässt sich vorgeben und wird eingehalten, gemessen am tatsächlichen Laufweg statt an der Luftlinie (Tabelle 11).
- **RL-Tauglichkeit:** Anbindung an die Gymnasium-Schnittstelle, schnell genug für Läufe über Millionen Schritte auf einem normalen Rechner (Abschnitt 4.1.1).
- **Spielbarkeit:** Dieselbe Simulation ist für Menschen spielbar, und einem trainierten Agenten kann beim Spielen zugesehen werden (Abschnitt 4.1.5).

## Z2: Schwellen für den Agenten.

- **Testset A** (Seeds 7000–7049):  $\geq 70\%$  Erfolgsquote,
- **Holdout B** (Seeds 8000–8049):  $\geq 60\%$  Erfolgsquote,
- pro Konfiguration mindestens 3 Trainingsläufe (Mittelwert  $\pm$  Standardabweichung).

Die beiden Schwellen unter Z2 standen vor den Auswertungsläufen fest und wurden daher nicht nachträglich passend gemacht. Ihre Herleitung steht in Abschnitt 3.2.4. Z1 ist anders entstanden, weil es sich um Konstruktionsziele handelt, die sich während der Entwicklung geschärft haben. Die steuerbare Schwierigkeit etwa erhielt ihre heutige Form erst, nachdem sich das Luftlinienmaß als irreführend erwiesen hatte (Tabelle 11).

**Vorgehen.** Das Vorgehen folgt der Reihenfolge, in der die Bestandteile voneinander abhängen. Abbildung 1 zeigt die Kette vom Seed bis zur Auswertung und ordnet ihr die beiden Forschungsfragen zu. Daran wird sichtbar, dass F2 keinen eigenen Untersuchungsgegenstand besitzt, sondern vollständig auf der Umgebung aufsetzt, die F1 hervorbringt.

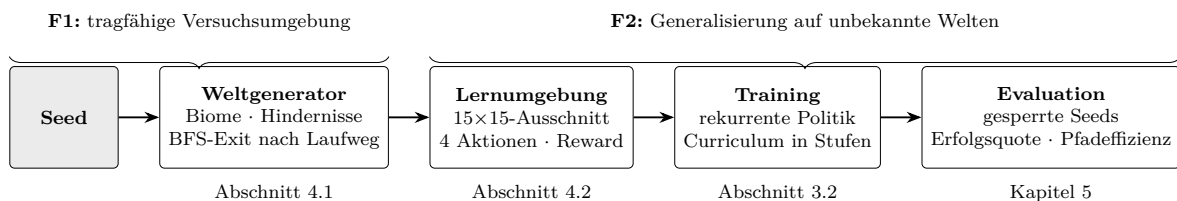


Abbildung 1: Einleitung - Ablauf vom Seed bis zur Auswertung mit der Zuordnung der beiden Forschungsfragen und der zugehörigen Abschnitte, eigene Darstellung.

Die Welt entsteht in Stufen. Ein grobes Landschaftsfeld legt fest, welches Biom wo liegt, woraus anschließend jedes einzelne Feld belegt wird, wobei der Anteil an Hindernissen vom Biom abhängt. Die Erreichbarkeit des Ziels wird nicht nachträglich geprüft, sondern durch die Konstruktion sichergestellt, da eine Breitensuche den Ausgang unter den begehbar erreichbaren Feldern mit der gewünschten Weglänge auswählt. Der Generator speichert nichts, weil jeder Ausschnitt allein von Seed und Koordinaten abhängt (Abschnitt 4.1.3).

Auf dieser Umgebung wird ein rekurrentes Verfahren mit internem Gedächtnis trainiert, das stufenweise erst kurze und dann zunehmend längere Wege zum Ziel bewältigen soll. Gemessen wird auf zwei Mengen von Welten, die im Training nie vorkommen und auch für die Modellauswahl gesperrt sind. Da einzelne Trainingsläufe im Reinforcement Learning stark schwanken können [7], beruht jede berichtete Zahl auf mehreren unabhängigen Läufen. Zusätzlich tritt der Agent gegen ungelernete Vergleichspolitiken an, die dieselbe Aufgabe unter demselben Protokoll lösen (Abschnitt 3.2).

Eine Lücke in dieser Aufstellung bestimmt am Ende das Ergebnis der Arbeit. Z1 verlangt lösbare Welten mit einstellbarer Entfernung, nicht jedoch, dass diese Welten schwer genug sind. Die Umgebung kann daher alle fünf Kriterien erfüllen und für F2 trotzdem untauglich sein. Genau dieser Fall ist eingetreten, denn eine ungelernete Kompassheuristik erreicht auf derselben Aufgabe 92 % (Abschnitt 5.2.3). Ein sechstes Kriterium, das die Härte der Welt selbst prüfen müsste, wird für Folgearbeiten in Abschnitt 6 vorgeschlagen.

## 1.5 Einordnung der Arbeit

Stoneforge liegt strukturell zwischen drei etablierten Benchmarks: MiniGrid (teilbeobachtbare Grid-Navigation, PPO+LSTM als Standard-Baseline) [8], Crafter (prozedurales 2D-Survival-Spiel mit Mining und Crafting) [9] und Procgen (Generalisierung über prozedurale Level)

[6]. Prozedurale Generierung als Mittel zur erzwungenen Generalisierung besitzt zudem eine eigene Forschungstradition [2, 5].

Die im eigenen Generator verwendeten Verfahren, rauschbasierte Basisbelegung, zelluläre Automaten und graphenbasierte Erreichbarkeitsprüfung, sind Standardbausteine der prozeduralen Inhaltserzeugung [1, 10]. Der Beitrag dieser Arbeit liegt daher nicht im Verfahren, sondern in dessen Kombination zu einer Umgebung, die zugleich spielbar und als Messinstrument kontrollierbar ist. Ein eigener Generator war nötig, weil sich Wandanteil, Hinderniskorrelation und Zielabstand bei MiniGrid, Crafter und Procgen nicht gezielt einstellen lassen; der Preis dafür ist die fehlende Einbettung in publizierte Vergleichswerte.

Tabelle 1 vergleicht, wie die drei Benchmarks Lösbarkeit und Schwierigkeit steuern. Die BFS-basierte Exit-Platzierung dieser Arbeit (Abschnitt 4.1.4) garantiert Lösbarkeit für jede einzelne Welt statt nur im Mittel über viele Level und macht die Zielentfernung dadurch erst als kontrollierte Variable nutzbar.

| Benchmark                    | Lösbarkeitsmechanismus                                                                    | Schwierigkeitssteuerung                    |
|------------------------------|-------------------------------------------------------------------------------------------|--------------------------------------------|
| MiniGrid                     | Platzierungsregeln erzeugen lösbare Layouts konstruktiv, keine nachgelagerte Prüfung [8]  | keine explizite Distanzvorgabe             |
| Crafter                      | kein Einzelziel, Bewertung über erreichte Achievements [9]                                | nicht anwendbar                            |
| Procgen                      | Eigenschaft der handentworfenen Levelgeneratoren, gilt im Mittel über viele Level [6, 11] | Verteilungsmodus easy/hard und Levelanzahl |
| Stoneforge<br>(diese Arbeit) | BFS-basierte Exit-Platzierung, für jede einzelne Welt geprüft                             | Zielentfernung direkt vorgebar             |

Tabelle 1: Einordnung in verwandte Arbeiten - Vergleich von Lösbarkeitsmechanismus und Schwierigkeitssteuerung etablierter Benchmarks, eigene Darstellung.

**Die Schwierigkeit steckt in der Struktur der Hindernisse.** Zelluläre Automaten formen aus verstreutem Rauschen zusammenhängende, konkave Strukturen wie Höhlen und Sackgassen, die eine reaktive, gedächtnislose Navigation zuverlässig scheitern lassen [10, 1]. Ohne diese Glättungsstufe verbleiben verstreute Einzelhindernisse (Tabelle 9), die ein Kompass-Folger auch ohne Gedächtnis umgeht; das erklärt die hohe Erfolgsquote der ungelerten Referenzpolitik unmittelbar (Abschnitt 5.2.3). Der Umkehrschluss trägt jedoch nicht: Die Glättungsstufe dieses Generators härtet die Welt nicht, sondern plant sie, weil die Ausgangsdichte unter der Geburtsschwelle des Automaten liegt (Wanddichte 0,037 statt 0,238, Umwegfaktor 1,001 statt 1,10, Tabelle 10). Publizierte PCG-Bausteine sind daher nicht parameterfrei übertragbar; die Schwierigkeit einer prozeduralen Umgebung sollte vor dem Training mit einer ungelerten Heuristik kalibriert werden (Abschnitt 6).

**Gedächtnis in teilbeobachtbaren Umgebungen.** Die Standardantwort auf partielle Beobachtbarkeit ist eine rekurrente Politik: DRQN erweiterte DQN um ein LSTM [12], R2D2 zeigte die Bedeutung korrekt geführter rekurrenter Zustände [13], und der POPGym-Benchmark findet einfache GRU-Architekturen insgesamt konkurrenzfähig zu aufwändigeren Modellen [14]. Der hier verwendete RecurrentPPO-Ansatz [15] folgt dieser Standard-Baseline; die eigene E3-Ablation bestätigt das Bild, da ein größeres LSTM (512 statt 256) keine Verbesserung bringt (Abschnitt 5.2.5).

**Curriculum und Exploration.** Curriculum Learning geht auf Bengio et al. zurück [16]; das leistungsorientierte Gating dieser Arbeit ist eine einfache Instanz davon. Prioritized Level Replay [17] wiederholt gezielt Level mit hohem Lernpotenzial; der hier verwendete Seed-Pool (Abschnitt 4.1.1) kehrt diese Auswahlrichtung um und ist deshalb kritisch zu sehen. Ein zählbasierter Explorations-Bonus für neu betretene Felder übernimmt die Rolle, die curiositätsgetriebene Verfahren wie ICM [18] und RND [19] in Umgebungen mit nicht direkt abzählbarer Neuheit spielen.

**Zur Vergleichbarkeit der absoluten Zahlen.** Ein direkter Vergleich der erreichten Erfolgsquote mit publizierten Werten ist nicht seriös möglich, da die Benchmarks unterschiedliche Metriken berichten: normalisierter Return bei Procgen [6], Achievements bei Crafter [9], aufgabenspezifische Erfolgsquoten bei MiniGrid. Belastbar ist der Vergleich nur innerhalb dieser Arbeit: gegen die eigene Gegenüberstellung (Tabelle 15), die Gap-Experimente (Abschnitt 5.2.5) und die vorab fixierten Kriterien (Abschnitt 3.2.4).

## 1.6 Aufbau der Arbeit

Nun folgt der Aufbau der Arbeit. Sie hält den üblichen Ablauf ein: zuerst die Begriffe, dann die Methodik, dann die technische Umsetzung, dann die Messungen und zuletzt deren Bewertung. Durchgehend steht dabei das Spiel vor dem Agenten, weil das eine die Voraussetzung des anderen ist (siehe Abbildung 1).

Kapitel 3 legt die Methodik beider Teile offen, bevor der Code dazu gezeigt wird: Abschnitt 3.1 begründet die Wahl der Generierungsverfahren und die Messmethodik der Welt-schwierigkeit, Abschnitt 3.2 die Wahl des Lernverfahrens, des Curriculums und des Eval-Protokolls. Kapitel 4 setzt diese Entscheidungen technisch um, wieder in zwei Abschnitten: Abschnitt 4.1 für das Spiel mit Simulationskern, Weltgenerator, Lösbarkeitsgarantie und Client, Abschnitt 4.2 für die darauf aufsetzende Lernumgebung mit partieller Beobachtbarkeit, Beobachtungsraum, Reward-Design, Trainings-Pipeline und Monitoring. Kapitel 5 bewertet in derselben Reihenfolge erst die Plattform (Abschnitt 5.1) und dann den Agenten (Abschnitt 5.2).

Der Versionsverlauf von Umgebung und Trainingskonfiguration ist als Tabelle in Abschnitt 4.2 zusammengefasst (Tabelle 11).

**Zur Versionsbezeichnung.** Die Kürzel *v11* und *v12* sind nötig, weil Ergebnisse verschiedener Entwicklungsstände nicht vergleichbar sind: Ändert sich die Weltgenerierung, erzeugt derselbe Seed eine andere Welt. *v11* bezeichnet die überarbeitete Umgebung, auf der alle Endergebnisse beruhen, *v12* die darauf aufbauende Trainingskonfiguration (Tabelle 4, Versionsüberblick in Tabelle 11).

**Zur Arbeitsteilung.** Die Arbeit entstand zu zweit, weshalb Tabelle 2 die Beiträge den Kapiteln zuordnet. „Schwerpunktmäßig“ bedeutet, dass Entwurf und Umsetzung überwiegend bei der genannten Person lagen. Beide Teile hängen enger zusammen, als die Aufteilung vermuten lässt, da mehrere der wichtigsten Befunde weder den Generator noch den Agenten allein betreffen, sondern ihr Zusammenspiel (siehe Abschnitt 5.2.3).

| Beitrag                                             | Schwerpunkt    | Kapitel           |
|-----------------------------------------------------|----------------|-------------------|
| Spielkonzept, C++-Simulationskern, Spiellogik       | Laurin Rößler  | 4.1, 4.1.1        |
| Prozedurale Weltgenerierung und Lösbarkeitsgarantie | Laurin Rößler  | 2.1.2, 3.1, 4.1.3 |
| Grafischer Spiel-Client mit KI-Zuschaumodus         | Laurin Rößler  | 4.1.1             |
| Gym-Anbindung, Beobachtungsraum, Reward-Design      | Florian Merlau | 2.2.8, 4.2        |
| Curriculum-Training, Hyperparameter, Fehleranalysen | Florian Merlau | 3.2, 4.2          |
| Evaluationsprotokoll, Referenzpolitiken, Auswertung | Florian Merlau | 3.2.3, 5          |
| Live-Monitoring, Werkzeuge, Experiment-Changelog    | gemeinsam      | 4.1.1             |

Tabelle 2: Einleitung - Zuordnung der Beiträge zu den Autoren und Kapiteln, eigene Darstellung.

## 2 Grundlagen (gemeinsam)

Dieses Kapitel legt die Begriffe fest, mit denen die folgenden Kapitel arbeiten, in zwei Abschnitten: Abschnitt 2.1 behandelt die prozedurale Erzeugung von Spielwelten, Abschnitt 2.2 das Lernen unter partieller Beobachtbarkeit samt der Steuerung und Bewertung dieses Lernprozesses. Für die vertiefte Einarbeitung liefern Sutton und Barto eine vollständige Einführung in das Reinforcement Learning [3], Shaker et al. eine in die prozedurale Inhaltserzeugung [1]; beide Werke tragen die Begriffsbildung dieses Kapitels.

### 2.1 Grundlagen der Weltgenerierung (Laurin)

#### 2.1.1 Begriff, Seed und Determinismus

Prozedurale Inhaltserzeugung (*Procedural Content Generation*, PCG) erzeugt Spielinhalte wie Karten, Level oder Gegenstände algorithmisch statt von Hand [1, 2]. Die Entwickelnden entwerfen dabei kein Level, sondern ein Regelwerk, das die Welt bei Bedarf berechnet. Das spart Speicher und Autorenaufwand, weil nur der Algorithmus abgelegt wird und nicht das fertige Level, und es liefert praktisch unbegrenzte Vielfalt [1].

Gesteuert wird die Erzeugung über einen *Seed*, den Startwert des Zufallszahlengenerators. Derselbe Seed erzeugt reproduzierbar dieselbe Welt, ein anderer Seed eine andere. Diese Reproduzierbarkeit ist keine technische Nebensache, sondern die Grundlage jedes kontrollierten Experiments mit prozeduralen Welten, da sich eine über ihre Seeds festgelegte Menge von Welten jederzeit exakt wiederherstellen lässt. Damit ist eine feste, benannte Stichprobe definiert. Wie diese Eigenschaft die Trennung von Trainings- und Testmenge trägt, zeigt Abschnitt 2.2.7.

#### 2.1.2 Algorithmen der prozeduralen Weltgenerierung

Der vorige Abschnitt klärt, *was* prozedurale Generierung ist. Dieser klärt, *wie* eine begehbare Welt entsteht. Die Darstellung folgt dem für gitterbasierte Generatoren üblichen dreistufigen Aufbau aus Basisbelegung, Formgebung und Validierung [1].

Gitterbasierte Generatoren bauen die Welt selten am Stück, sondern in quadratischen Kacheln fester Größe, sogenannten *Chunks*. Ein Chunk entsteht erst, wenn der Agent ihn betritt. Hängt seine Berechnung ausschließlich von Seed und Koordinaten ab, spielt die Reihenfolge der Erzeugung keine Rolle, weshalb die Welt ohne gespeicherten Zustand auskommt.

**Rauschbasierte Basisbelegung.** Die erste Stufe belegt jede Gitterzelle deterministisch aus einer Rauschfunktion und übersetzt deren Wert über Schwellwerte in Materialklassen, eines der Standardverfahren gitterbasierter PCG [1]. Zu unterscheiden sind zwei Varianten, deren Unterschied über die Spielbarkeit bestimmt. *Hash-Rauschen* berechnet den Wert jeder Zelle unabhängig aus ihren Koordinaten und dem Seed, wodurch unkorreliertes weißes Rauschen entsteht, aus dem ein Schwellwert verstreute *Einzelhindernisse* schneidet. *Wert- oder Gradientenrauschen* wertet dagegen nur ein gröberes Stützgitter aus und interpoliert dazwischen, klassisch mit Perlins Glättungsfunktion  $3t^2 - 2t^3$  [20], weshalb benachbarte Zellen korrelieren und *zusammenhängende* Formationen entstehen.

Werden die Eingabekoordinaten vor der Auswertung selbst durch ein Rauschfeld verschoben (*Domain Warping*), verlieren die Regionen ihre am Stützgitter ablesbare Rechteckigkeit. Beide Verfahren sind pro Seed reproduzierbar und kommen ohne gespeicherten Zustand aus, taugen also für theoretisch unbegrenzte Welten.

**Zelluläre Automaten als Formgebung.** Rein rauschbasierte Belegungen wirken häufig zerfranst, weshalb meist eine Glättungsstufe über einen zellulären Automaten folgt. Er



bewertet jede Zelle iterativ anhand der Zahl belegter Nachbarn, üblicherweise in der *Moore-Nachbarschaft* aus den acht direkt angrenzenden Zellen (horizontal, vertikal und diagonal). Geburts- und Überlebensregeln (*Birth/Survival Rules*) formen daraus fließende, natürlich wirkende Höhlen- und Klippenstrukturen: Eine freie Zelle wird belegt, sobald die Zahl belegter Nachbarn eine Geburtsschwelle erreicht, und eine belegte Zelle bleibt nur belegt, solange sie eine Überlebensschwelle erfüllt.

Wenige Iterationen genügen, um verstreutes Rauschen in zusammenhängende Strukturen zu überführen. Johnson et al. demonstrieren das Verfahren an echtzeitfähig erzeugten Höhlenleveln [10].

**Graphentheoretische Erreichbarkeitsanalyse.** Prozedurale Welten haben ein Kernproblem, die Spielbarkeit (*Playability*), also die Vermeidung unlösbarer Level-Zustände (*Deadlocks*). Für die Analyse der Konnektivität wird das Zellgitter als ungerichteter Graph gelesen, in dem begehbare Zellen die Knoten und Nachbarschaftsbeziehungen die Kanten bilden. Welche Nachbarschaft dabei richtig ist, folgt aus dem Bewegungsmodell: Bei ausschließlich achsenparalleler Bewegung gilt die Vierer-Nachbarschaft (*von Neumann*) und nicht die für die Glättung verwendete Moore-Nachbarschaft, denn eine über die Diagonale hergestellte Verbindung wäre für den Agenten nicht begehbar.

Die Breitensuche (*Breadth-First Search*, BFS) traversiert den Graphen ebenenweise vom Startknoten aus [21]. Interessiert dabei nur die erreichbare Menge und nicht die Distanz, wird von *Flood-Fill* gesprochen. Die Breitensuche beantwortet damit zwei Fragen zugleich, nämlich ob ein Zielknoten überhaupt erreichbar ist und wie der kürzeste Pfad auf einem ungewichteten Gitter verläuft [21]. Beide Eigenschaften kehren später wieder (siehe Abschnitte 2.2.8 und 2.2.11).

**Heuristische Sicherheitsnetze und Metriken.** Spielbarkeit sicherzustellen und unspielbare Level zu reparieren ist ein eigenständiger Baustein der PCG-Praxis [1]. Da die exakte graphenbasierte Validierung bei großen Weltausschnitten teuer wird, wird sie häufig mit billigeren Prüfungen kombiniert. Als Vorabschätzung dient die *Manhattan-Distanz*, die Distanz zweier Punkte auf einem orthogonalen Gitter unter achsenparalleler Bewegung:

$$D_{\text{Manhattan}} = |x_1 - x_2| + |y_1 - y_2|. \quad (1)$$

Sie ist in konstanter Zeit berechenbar und unterschätzt den tatsächlichen Laufweg, sobald Hindernisse dazwischenstehen. Als Ersatz für eine Erreichbarkeitsprüfung taugt sie deshalb nicht, wohl aber als untere Schranke und als Abbruchkriterium. Scheitert die Prüfung, greift ein zweites Sicherheitsnetz: Eine pragmatische *Fallback-Generierung* (*Carving*) räumt entlang direkter Geometrieachsen einen Korridor von Start zu Ziel frei, typischerweise erst in der einen und dann in der anderen Achse.

Beide Mittel wirken, aber sie sind nicht folgenlos. Ein gecarvter Korridor ist per Konstruktion eine gerade, hindernisfreie Linie und damit eine strukturelle Auffälligkeit, die das erzeugte Weltbild systematisch von einer gleichmäßig verteilten Hindernisstruktur wegzieht.

## 2.2 Grundlagen des Reinforcement Learning (Florian)

### 2.2.1 Reinforcement Learning und der Markov-Entscheidungsprozess

Reinforcement Learning beschreibt Lernen durch Interaktion: Der Agent wählt in einem Zustand eine Aktion, erhält eine Belohnung (Reward) und landet in einem Folgezustand [3]. Formal ist das ein Markov-Entscheidungsprozess (MDP), ein Tupel  $(S, A, P, R, \gamma)$  mit Zustandsmenge  $S$ , Aktionsmenge  $A$ , Übergangswahrscheinlichkeit  $P(s' | s, a)$ , Rewardfunktion  $R(s, a)$  und Diskontfaktor  $\gamma \in [0, 1)$ . Gesucht ist eine Politik  $\pi(a | s)$ , die den erwarteten diskontierten Return  $G_t = \sum_{k \geq 0} \gamma^k r_{t+k}$  maximiert. Die Zustandswertfunktion  $V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$

gibt den erwarteten Return ab einem Zustand an. Der Diskontfaktor  $\gamma$  gewichtet spätere Rewards geringer.

Abbildung 2 zeigt diesen Kreislauf, die Grundfigur des gesamten Verfahrens. Jedes hier trainierte Modell ist eine Variante dieser Schleife: Die Umgebung heißt *Stoneforge*, die Aktionen sind die vier Bewegungsrichtungen, und der Reward liefert das Lernsignal. An der Abbildung wird zugleich sichtbar, worauf es im weiteren Verlauf ankommt, weil der Agent ausschließlich über diesen Rückkanal etwas über die Welt erfährt.

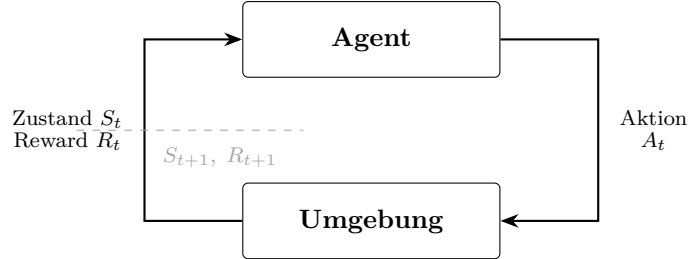


Abbildung 2: Grundlagen - Agent-Umwelt-Interaktion im Markov-Entscheidungsprozess mit dem Übergang zum nächsten Zeitschritt als gestrichelter Linie, eigene Darstellung in Anlehnung an Sutton und Barto [3, Abb. 3.1].

### 2.2.2 Partielle Beobachtbarkeit (POMDP)

Sieht der Agent nicht den vollständigen Zustand, sondern nur eine Beobachtung, liegt ein *Partially Observable* MDP (POMDP) vor [22]. Für die Beobachtung gilt die Markov-Eigenschaft dann nicht mehr, weil dieselbe lokale Beobachtung zu ganz verschiedenen globalen Zuständen gehören kann. Eine reaktive Politik, die nur die aktuelle Beobachtung nutzt, ist deshalb im Allgemeinen nicht optimal. Abhilfe schafft ein internes Gedächtnis, das vergangene Beobachtungen zu einem *Belief-State* verdichtet [12].

Abbildung 3 stellt beide Fälle gegenüber. Im MDP greift die Politik direkt auf den Zustand zu, im POMDP schiebt sich eine Beobachtungsfunktion dazwischen. Die Politik muss aus der Vorgeschichte erst rekonstruieren, wo sie vermutlich steht, und genau diese Zusatzleistung ist in der Abbildung grau hinterlegt.

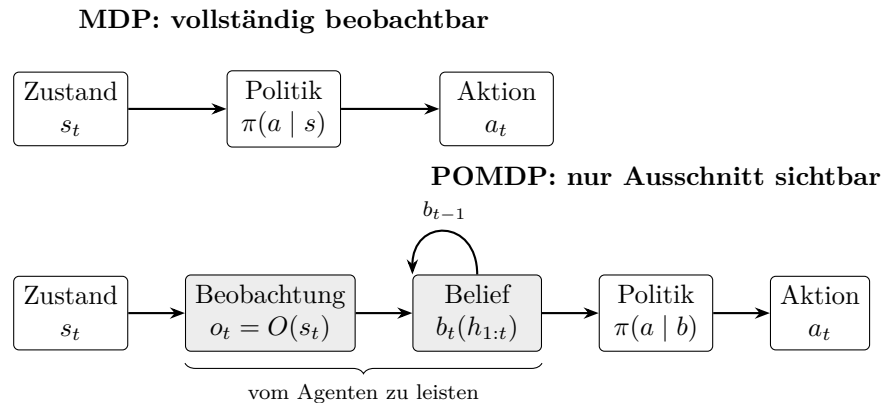


Abbildung 3: Grundlagen - Vollständige (oben) gegenüber partieller Beobachtbarkeit (unten) mit den beiden im POMDP zusätzlich anfallenden und grau hinterlegten Schritten, eigene Darstellung; Formalismus nach Kaelbling et al. [22].

Eine zweite Konsequenz trägt diese Arbeit weit. Unter partieller Beobachtbarkeit kann eine *stochastische* gedächtnislose Politik jede deterministische gedächtnislose deutlich über-

treffen [23]. Auf die Reichweite dieser Aussage kommt es an, denn sie gilt für *gedächtnislose* Politiken. Ist der Belief-State hinreichend genau rekonstruiert, existiert wieder eine optimale deterministische Politik (siehe Abschnitt 2.2.6).

Ein Leistungsabstand zwischen Sampling- und Argmax-Auswertung einer *rekurrenten* Politik ist damit gerade kein notwendiges Merkmal der Aufgabe, sondern ein Indikator für einen unvollständigen Belief-State. Genau in dieser Lesart wird er in der Evaluation geprüft.

Stoneforge ist ein POMDP, weil der Agent nur einen lokalen  $15 \times 15$ -Ausschnitt sieht. Abbildung 4 stellt den Unterschied am konkreten Weltausschnitt dar und macht sichtbar, dass der überwiegende Teil des Weltzustands außerhalb der Beobachtung liegt. Neben diesem unmittelbar sichtbaren lokalen Ausschnitt existiert eine zweite, weniger offensichtliche Quelle partieller Beobachtbarkeit. Sie entsteht erst aus der Forderung nach Generalisierung auf unbekannte Welten und wird in Abschnitt 2.2.7 behandelt, wo beide Themen zusammenlaufen.

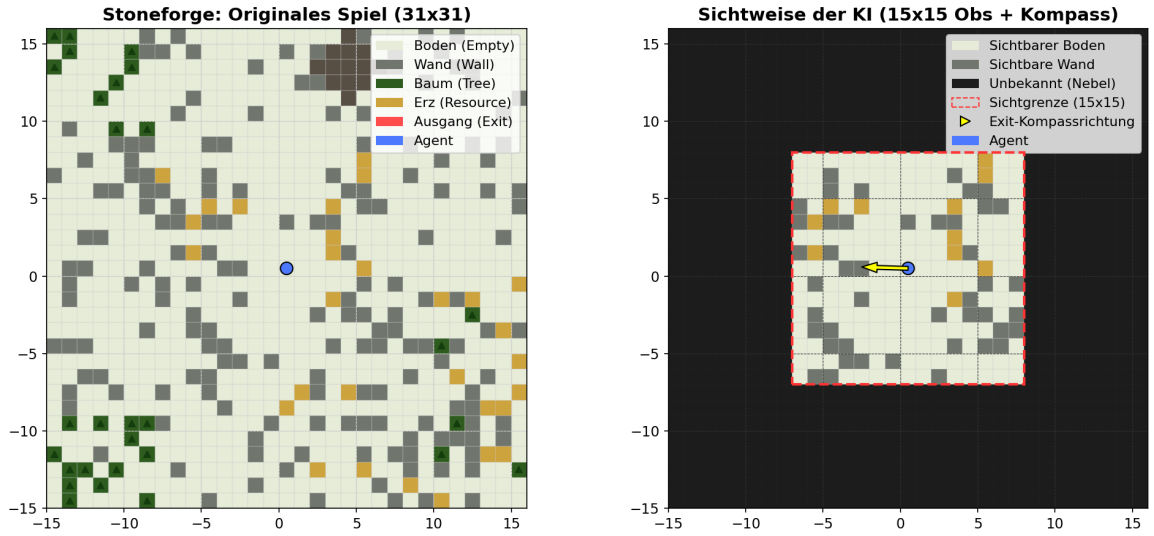


Abbildung 4: Grundlagen - Vollständiger Weltzustand als  $31 \times 31$ -Ausschnitt (links) gegenüber der Beobachtung des Agenten mit rot umrandetem  $15 \times 15$ -Sichtfeld und Kompasspfad zum Exit (rechts), eigene Darstellung.

### 2.2.3 Wertbasierte Verfahren: Q-Learning und DQN

Wertbasierte Verfahren lernen die Aktionswertfunktion  $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a]$  und leiten die Politik daraus ab, indem sie die Aktion mit dem höchsten  $Q$ -Wert wählen. Q-Learning aktualisiert  $Q$  über die Bellman-Gleichung. Deep Q-Networks (DQN) approximieren  $Q$  mit einem neuronalen Netz und stabilisieren das Training über einen Replay-Buffer und ein Target-Netz [24]. DQN verarbeitet nur diskrete Aktionsräume.

### 2.2.4 Policy-Gradient- und Actor-Critic-Verfahren

Policy-Gradient-Verfahren optimieren die Politik  $\pi_\theta$  direkt über den Gradienten des erwarteten Returns. Actor-Critic-Verfahren stellen ihnen eine gelernte Wertfunktion (Critic) zur Seite, die als Baseline die Varianz der Gradientenschätzung senkt. Der Vorteil (Advantage)  $A(s, a) = Q(s, a) - V(s)$  misst, wie viel besser eine Aktion gegenüber dem Durchschnitt ist; die Generalized Advantage Estimation (GAE) schätzt ihn über mehrere Zeitschritte mit einem Parameter  $\lambda$  [25]. Advantage Actor-Critic (A2C) ist die synchrone Variante von A3C [26].

Wie gut der Critic gelernt hat, verrät die *explained variance* (EV), der Anteil der Varianz der tatsächlichen Returns, den die Wertschätzung erklärt.  $EV \approx 1$  steht für eine verlässliche

Wertfunktion,  $EV \approx 0$  dafür, dass der Critic praktisch nichts gelernt hat. Diese Größe dient hier als wichtigste Diagnosemetrik des Trainings, weil sie eine instabile Wertschätzung früher anzeigt als die Erfolgsquote.

### 2.2.5 Proximal Policy Optimization (PPO)

PPO ist das Hauptverfahren dieser Arbeit [27]. Es ist ein Actor-Critic-Verfahren, das die Politik pro Update nur begrenzt verändert, um Instabilität durch zu große Schritte zu vermeiden. Dazu schneidet es das Wahrscheinlichkeitsverhältnis  $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{alt}}}(a_t | s_t)$  in der Zielfunktion ab (*clipping*):

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t[\min(r_t(\theta) A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)] .$$

Abbildung 5 zeigt die Wirkung des Clippings an einem einzelnen Zeitschritt. War die Aktion besser als erwartet ( $A_t > 0$ ), lohnt sich eine höhere Wahrscheinlichkeit, allerdings nur bis zum Faktor  $1 + \epsilon$ , weil die Zielfunktion darüber flach verläuft und der Anreiz für einen größeren Schritt wegfällt. War sie schlechter als erwartet ( $A_t < 0$ ), gilt dasselbe spiegelbildlich bei  $1 - \epsilon$ . Genau an solchen zu großen Aktualisierungen scheitern einfache Policy-Gradient-Verfahren, die ihre Schrittweite nicht begrenzen.

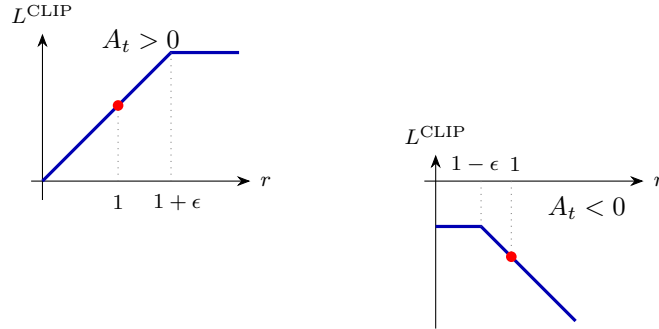


Abbildung 5: Grundlagen - Ein einzelner Term der abgeschnittenen Zielfunktion  $L^{\text{CLIP}}$  über dem Wahrscheinlichkeitsverhältnis  $r_t(\theta)$  für positiven (links) und negativen Advantage (rechts) mit dem Startpunkt der Optimierung als rotem Punkt, eigene Darstellung in Anlehnung an Schulman et al. [27, Abb. 1].

Hinzu kommt ein Entropie-Term, der Exploration fördert, weil sein Gewicht steuert, wie stochastisch die Politik bleibt. Trainiert wird in Zyklen: Zuerst sammeln die parallelen Umgebungen einen Rollout fester Länge, anschließend optimiert PPO diesen in Minibatches über mehrere Durchläufe. Verwendet wird die Implementierung aus Stable-Baselines3 [28].

Eine gelernte stochastische Politik lässt sich auf zwei Arten ausführen: *stochastisch*, indem die Aktion aus der Verteilung  $\pi(a | s)$  gezogen wird (Sampling), oder *deterministisch* über die stets wahrscheinlichste Aktion (Argmax). Dasselbe trainierte Modell kann in beiden Betriebsarten unterschiedlich gut abschneiden. Der Abstand zwischen beiden Messwerten wird durchgängig **Det/Stoch-Gap** genannt. Er ist einer der Hauptgegenstände dieser Arbeit, weil er sich als groß und hartnäckig erwiesen hat.

Anschaulich gesprochen löst der Agent die Aufgabe zuverlässiger, wenn er würfelt, als wenn er stets seine eigene beste Aktion wählt. Dass das unter partieller Beobachtbarkeit möglich ist, folgt aus Abschnitt 2.2.2. Dass es bei einer Politik *mit* Gedächtnis trotzdem erklärungsbedürftig bleibt, ebenfalls.

### 2.2.6 Rekurrente Politiken (LSTM)

Alle bisher genannten Verfahren laufen auch mit einem gewöhnlichen vorwärtsgerichteten neuronalen Netz, einem *Multilayer Perceptron* (MLP). Ein solches Netz bildet jede Beobachtung

unabhängig auf eine Aktion ab und besitzt kein Gedächtnis, weshalb es nicht unterscheiden kann, ob es eine Stelle zum ersten oder zum zehnten Mal sieht. Unter partieller Beobachtbarkeit ist das eine harte Einschränkung (siehe Abschnitt 2.2.2). Die Bezeichnung *MLP* steht im Folgenden durchgängig für diese gedächtnislose Variante.

Um in einem POMDP ein Gedächtnis bereitzustellen, wird die Politik um ein rekurrentes Netz erweitert. Long Short-Term Memory (LSTM) ist eine rekurrente Architektur, die Information über viele Zeitschritte hinweg hält [29], und ihr Zustand approximiert den Belief-State, indem er die bisher gesehenen Beobachtungen zusammenfasst. Rekurrentes Netz und wertbasiertes Reinforcement Learning wurden als DRQN kombiniert [12]; hier kommt die policy-gradient-Variante RecurrentPPO zum Einsatz.

Wie das Gedächtnis wirkt, zeigt Abbildung 6 in der über die Zeit „entrollten“ Darstellung. Der verborgene Zustand  $h_t$  wandert von Schritt zu Schritt weiter und sammelt Information, die in der einzelnen Beobachtung fehlt. Für Stoneforge ist das genau die Information „diese Stelle wurde bereits besucht“ beziehungsweise „diese Wand wurde bereits nach Norden abgelaufen“.

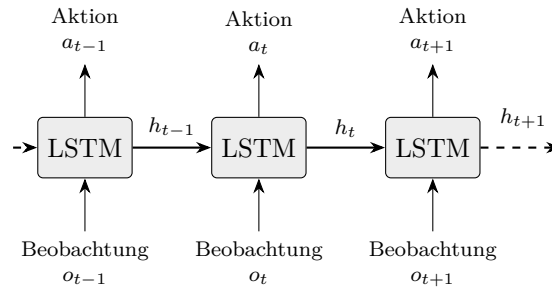


Abbildung 6: Grundlagen - Über die Zeit entrollte rekurrente Politik, bei der jeder Zeitschritt neben der aktuellen Beobachtung  $o_t$  den verborgenen Zustand  $h_{t-1}$  des Vorschritts erhält, eigene Darstellung in Anlehnung an Hausknecht und Stone [12].

Trainiert wird ein solches Netz per *Backpropagation Through Time* (BPTT), indem das Netz über eine Sequenz fester Länge entrollt wird und der Gradient über alle Zeitschritte zurückläuft. Die Sequenzlänge ist damit ein Hyperparameter mit zwei Seiten. Zu kurze Sequenzen begrenzen, wie weit zurück ein Zusammenhang überhaupt gelernt werden kann, zu lange Sequenzen lassen Gradienten verschwinden und bremsen das Training.

Dazu kommt die Frage, mit welchem verborgenen Zustand eine Sequenz startet. Kap-turowski et al. zeigen für rekurrente Verfahren, dass die Behandlung dieses Zustands die Leistung bei sonst unveränderter Architektur erheblich verschiebt, und schlagen unter anderem eine Einlaufphase (*Burn-in*) vor [13]. Bei der Auswertung stellt sich dieselbe Frage, denn ein zwischen Episoden nicht zurückgesetzter Zustand ergibt die Messung einer anderen als der trainierten Politik.

### 2.2.7 Prozedurale Welten im Reinforcement Learning

Die beiden vorangegangenen Abschnitte behandeln zwei eigenständige Themen, die prozedurale Erzeugung von Welten und das Lernen einer Politik unter partieller Beobachtbarkeit. Im Folgenden laufen sie zusammen, weil erst ihre Verbindung den Versuchsaufbau dieser Arbeit begründet.

Der Ausgangspunkt ist ein bekanntes Problem des maschinellen Lernens. Wird ein Modell stets auf denselben Daten trainiert und geprüft, lernt es womöglich die Antworten auswendig (*Overfitting*), statt eine übertragbare Regel zu bilden. Im Reinforcement Learning entspricht ein einzelnes festes Level solchen festen Daten, weshalb die Politik dann die konkrete Karte memorisiert, ohne allgemeine Navigation zu erlernen [6, 4].

Bekommt dagegen jede Episode eine neue, per Seed erzeugte Welt, entzieht sich die Aufgabe dem Auswendiglernen, da der Agent jede einzelne Karte zu selten sieht, um sie zu behalten. Er muss eine Strategie bilden, die über die Trainingswelten hinaus trägt. Prozedurale Generierung ist deshalb ein etabliertes Mittel, um Generalisierung im Reinforcement Learning zu prüfen und Overfitting auf einzelne Level zu verhindern [6, 5].

Die Seed-Reproduzierbarkeit aus Abschnitt 2.1.1 liefert dafür das Werkzeug, weil sie disjunkte Seed-Mengen und damit eine strenge Trennung von Trainings- und Testmenge erlaubt. Die Evaluation auf trainingsfremden Seeds folgt genau diesem Prinzip. Gemessen wird nicht, wie gut eine Politik gelernte Welten wiederfindet, sondern wie gut sie auf nie gesehene überträgt.

Die Verbindung reicht tiefer als bis zum Versuchsaufbau. Die Forderung, auf unbekannten Welten zu funktionieren, erzeugt selbst partielle Beobachtbarkeit, da der Politik zur Laufzeit unbekannt ist, in welcher der möglichen Welten sie steckt (siehe Abschnitt 2.2.2). Ghosh et al. zeigen, dass daraus auch dann ein POMDP wird, wenn jede einzelne Welt für sich vollständig beobachtbar wäre, und nennen das *epistemic POMDP* [30].

Die Unsicherheit liegt hier über der Identität der Welt und nicht über dem Zustand innerhalb einer bekannten Welt. Mehr Gedächtnis über die laufende Episode löst sie deshalb nicht auf. Auch die optimale Politik eines epistemic POMDP ist im Allgemeinen stochastisch, was für die spätere Untersuchung des Det/Stoch-Gaps zählt. Prozedurale Generierung und Lernen unter partieller Beobachtbarkeit sind für diese Arbeit also keine zufällig kombinierten Bausteine. Die Generalisierungsanforderung verknüpft sie ursächlich.

### 2.2.8 Reward Shaping (PBRs)

Zusätzliche Belohnungen können das Lernen beschleunigen, dürfen die optimale Politik aber nicht verändern. Potential-Based Reward Shaping (PBRs) leistet genau das, denn der Zusatzreward hat die Form  $F(s, s') = \gamma \Phi(s') - \Phi(s)$  mit einer Potentialfunktion  $\Phi$ . Ng et al. zeigen, dass diese Form policy-invariant ist und die optimale Politik daher unangetastet lässt [31]. In dieser Arbeit hängt  $\Phi$  an der BFS-Distanz zum Exit, also an der Länge des kürzesten begehbaren Wegs vom Agenten zum Exit in Schritten. Anders als die Luftlinie berücksichtigt dieses Maß Wände und Umwege, weshalb es dem Potential zugrunde liegt.

### 2.2.9 Curriculum Learning

Curriculum Learning trainiert mit ansteigender Aufgabenschwierigkeit statt in zufälliger Reihenfolge [16]. Hier wird die Exit-Distanz stufenweise erhöht, wobei der Übergang zwischen den Stufen leistungsbasiert läuft und eine Stufe ab einer Ziel-Erfolgsquote als bestanden gilt.

Davon zu unterscheiden ist die Steuerung auf Ebene einzelner Welten. *Prioritized Level Replay* (PLR) wiederholt gezielt jene Level, an denen eine Politik noch am meisten lernen kann, statt sie gleichverteilt zu ziehen, und verbessert dadurch Sample-Effizienz und Generalisierung [17]. Auf die Auswahlrichtung kommt es an, weil Level mit hohem Lernpotenzial wiederholt werden und gerade nicht die bereits gemeisterten.

### 2.2.10 Messen und Bewerten

Die bisherigen Abschnitte behandeln, wie Welten entstehen und wie Politiken lernen. Nun folgt die Frage, woran sich beides beurteilen lässt. Das ist kein Nebenschauplatz, weil zwei der drei Befunde dieser Arbeit nicht das Verhalten des Agenten betreffen, sondern die Aussagekraft einer Messung.

### 2.2.11 Metriken einer Navigationsaufgabe

**Erfolgsquote.** Die naheliegende Metrik ist der Anteil gelöster Welten. Formal ist sie der Schätzwert  $\hat{p} = k/n$  eines Bernoulli-Parameters, denn jede Welt ist ein Versuch, der gelingt oder nicht. Daraus folgen zwei Eigenschaften unmittelbar. Erstens ist die Erfolgsquote nur zusammen mit dem Abbruchkriterium definiert, da von der erlaubten Suchdauer abhängt, ob eine Welt als gelöst zählt; das Episodenlimit gehört damit zur Metrik und nicht zu ihrem Umfeld. Zweitens ist  $\hat{p}$  bei kleinem  $n$  ungenau, wie Abschnitt 2.2.12 beziffert.

**Pfادهffizienz.** Die Erfolgsquote sagt nichts darüber, *wie* eine Welt gelöst wurde. Dafür dient die Pfادهffizienz

$$\eta = \frac{d^*(s_0)}{T}, \quad (2)$$

das Verhältnis aus kürzestem Weg  $d^*$  vom Startzustand und tatsächlich benötigten Schritten  $T$ . Es gilt  $0 < \eta \leq 1$ , wobei  $\eta = 1$  optimale Navigation und  $\eta = 0,05$  einen zwanzigfachen Umweg bedeutet. Zwei Feinheiten sind zu beachten, weil die Größe sonst als Rechenfehler gelesen wird.

Zum einen macht es einen Unterschied, ob über die *Quotienten* der Einzelepisoden gemittelt oder der Quotient der Mittelwerte gebildet wird. Beide Größen fallen auseinander, sobald die Schrittzahl schief verteilt ist. Da  $1/T$  konvex in  $T$  ist, liefert die Jensen-Ungleichung

$$\mathbb{E}\left[\frac{d^*}{T}\right] \geq \frac{d^*}{\mathbb{E}[T]}, \quad (3)$$

das Mittel der Quotienten liegt also systematisch höher. Welche Variante berichtet wird, gehört daher in den Text.

Zum anderen ist  $\eta$  nur auf erfolgreichen Episoden definiert. Werden zwei Politiken mit unterschiedlicher Erfolgsquote verglichen, wird die schwächere auf der leichteren Teilmenge bewertet. Die Verzerrung arbeitet also zugunsten der schwächeren Politik. Bleibt sie beim Interpretieren unberücksichtigt, wird eine Stärke gelesen, wo keine ist.

**Betriebsarten.** Eine gelernte stochastische Politik lässt sich auf zwei Arten ausführen, durch Ziehen aus  $\pi(a | s)$  oder durch Wahl der wahrscheinlichsten Aktion (Argmax). Dazwischen liegt die temperaturskalierte Auswertung, die die Logits vor dem Softmax durch einen Parameter  $T$  teilt, wobei  $T \rightarrow 0$  Argmax und  $T = 1$  das ungeänderte Sampling ergibt. Dieselbe Politik kann in beiden Betriebsarten unterschiedlich abschneiden, weshalb jede Angabe die gemessene Betriebsart nennen muss. Dass ein solcher Unterschied unter partieller Beobachtbarkeit theoretisch zu erwarten ist, folgt aus Abschnitt 2.2.2.

### 2.2.12 Streuung, Konfidenz und die Zahl der Läufe

Die Messunsicherheit dieser Experimente hat zwei voneinander unabhängige Quellen. Werden beide vermischt, entsteht eine Aussage ohne Gehalt.

**Unsicherheit innerhalb eines Laufs.** Wird ein festes Modell auf  $n$  Welten geprüft, ist die Erfolgsquote ein Bernoulli-Anteil mit Standardfehler  $\sqrt{p(1-p)/n}$ . Für  $n = 50$  und den ungünstigsten Fall  $p = 0,5$  ergibt das ein 95 %-Konfidenzintervall von rund  $\pm 14$  Prozentpunkten. Unterschiede unterhalb dieser Größe lassen sich aus einer einzelnen Messung nicht interpretieren, unabhängig davon, wie sorgfältig sie erhoben wurde.

**Unsicherheit zwischen Läufen.** Zwei Trainingsläufe derselben Konfiguration unterscheiden sich allein durch Zufallsinitialisierung und Reihenfolge der Erfahrungen. Henderson et al. zeigen, dass diese Streuung im Deep Reinforcement Learning so groß ausfällt, dass Aussagen aus wenigen Läufen systematisch in die Irre führen, und fordern deshalb Mittelwert und Standardabweichung über mehrere unabhängige Läufe statt des besten Laufs [7]. Für die Aggregation über  $m$  Läufe gilt die *Stichproben*-Standardabweichung,

$$s = \sqrt{\frac{1}{m-1} \sum_{i=1}^m (x_i - \bar{x})^2}, \quad (4)$$

mit dem Nenner  $m-1$  statt  $m$  (Bessel-Korrektur), da die Läufe eine Stichprobe aus allen möglichen Läufen sind und nicht die Grundgesamtheit. Das zugehörige Konfidenzintervall folgt bei kleinem  $m$  der  $t$ -Verteilung mit  $m-1$  Freiheitsgraden und nicht der Normalverteilung.

**Vergleich zweier Gruppen.** Sollen zwei Chargen von Läufen verglichen werden, etwa um zu prüfen, ob ein Unterschied eine systematische Ursache hat oder bloß Streuung ist, eignet sich der Welch-Test. Er ist die Variante des  $t$ -Tests, die ungleiche Varianzen und ungleiche Gruppengrößen zulässt, und damit die konservative Wahl, solange über die Varianzen nichts bekannt ist. Ein Testergebnis ohne Signifikanz auf dem gewählten Niveau belegt keine Gleichheit, sondern besagt nur, dass die Daten einen Unterschied nicht belegen.

**Vorabfestlegung von Schwellen.** Werden Erfolgskriterien erst nach Kenntnis der Ergebnisse formuliert oder nachträglich angepasst, verlieren sie ihre Aussagekraft. Kerr nennt dieses Vorgehen *HARKing* (*Hypothesizing After the Results are Known*) [32]. Das Gegenmittel ist billig, nämlich die dokumentierte Festlegung vor der Messung. Allgemeiner fassen Pineau et al. die Anforderungen an reproduzierbare Berichte in einer Checkliste zusammen, die neben der Zahl der Läufe auch den durchsuchten Hyperparameterbereich und die Trennung der Datenmengen umfasst [33].

### 2.2.13 Referenzpolitiken als Bezugspunkt

Ein Messwert ohne Bezugspunkt ist nicht interpretierbar. Ob eine Erfolgsquote von 65 % eine gelernte Fähigkeit ausdrückt, lässt sich nur beantworten, wenn bekannt ist, was ohne Lernen erreichbar wäre. Dafür dienen *ungelernte Referenzpolitiken*, die dieselbe Beobachtung erhalten und unter demselben Protokoll laufen. Zu unterscheiden sind zwei Typen.

Eine **Zufallspolitik** wählt gleichverteilt aus dem Aktionsraum und markiert die absolute Untergrenze. Sie beantwortet, ob überhaupt etwas gelernt wurde, nicht aber, ob das Gelernte über triviale Strategien hinausgeht.

Eine **Heuristik** nutzt einen Teil der Beobachtung nach einer fest programmierten Regel, ohne Parameter und ohne Training. Sie ist der schärfere Bezugspunkt, weil sie ausdrückt, welcher Anteil des Ergebnisses schon ohne Lernen zu haben ist. Übertrifft eine solche Heuristik den trainierten Agenten, liegt das Problem nicht beim Agenten, sondern bei der Aufgabenstellung, da diese die zu messende Fähigkeit gar nicht verlangt. Genau dieser Fall tritt in dieser Arbeit ein.

Damit bekommt die Referenzpolitik eine zweite Rolle, die über den Vergleich hinausgeht. Sie taugt als *Kalibrierinstrument*, das die Schwierigkeit einer Umgebung vor jedem Training bestimmt, in Sekunden statt in Trainingsstunden. Kirk et al. argumentieren in dieselbe Richtung, wenn sie einen rein prozeduralen Ansatz beim Benchmark-Entwurf als hinderlich für den Fortschritt der Generalisierungsforschung bezeichnen [4]. Ohne ein Instrument, das trennt, warum ein Verfahren gelingt oder scheitert, sagt der Messwert allein wenig.



## 3 Methodik (gemeinsam)

### 3.1 Methodik der Weltgenerierung (Laurin)

Für die Erzeugung der Welten wird eine Kombination etablierter Verfahren der prozeduralen Inhaltserzeugung gewählt: rauschbasierte Basisbelegung, ein zellulärer Automat zur Formgebung und eine graphenbasierte Erreichbarkeitsprüfung (Grundlagen in Abschnitt 2.1.2). Maßgeblich für die Auswahl sind drei Designziele: Die Welt muss spielbar sein, ihre Schwierigkeit muss steuerbar sein, und ihre Lösbarkeit muss für jede einzelne generierte Welt feststehen, nicht nur im Mittel über viele Welten (Abschnitt 1.5). Letzteres schließt eine nachträgliche Erreichbarkeitsprüfung aus; die Lösbarkeit wird stattdessen durch die Konstruktion selbst erzwungen, indem der Exit gezielt aus dem Baum der von der Startposition erreichbaren Felder gewählt wird (Abschnitt 4.1.4).

Die Schwierigkeit einer generierten Welt wird nicht geschätzt, sondern gemessen: über die Waddichte und den Umwegfaktor (BFS-Distanz geteilt durch Manhattan-Distanz) sowie über eine ungelernte BFS-Orakel-Politik als Referenz. Diese Methodik erlaubt es, Eingriffe in den Generator vor jedem Training zu bewerten, statt ihre Wirkung erst am Agenten abzulesen; die gemessenen Werte stehen in Tabelle 10 und werden in Abschnitt 5.1 gegen die Anforderungen der Arbeit gestellt.

Die konkrete technische Umsetzung, also Chunk-Struktur, Rauschfunktion, Automatenregeln und Exit-Konstruktion im Code, folgt in Abschnitt 4.1.

### 3.2 Methodik des RL-Trainings (Florian)

Dieser Abschnitt legt fest, mit welchem Lernverfahren, welchem Trainingsplan und welchem Messverfahren die Forschungsfrage F2 beantwortet werden soll, bevor in Kapitel 4 die technische Umsetzung und in Kapitel 5 die Ergebnisse folgen. Keine dieser Festlegungen entstand in einem einzigen Entwurf: Mehrere Annahmen mussten im Projektverlauf revidiert werden, wenn sich ein Vorgehen beim Ausprobieren als ungeeignet erwies (Überblick in Tabelle 11).

#### 3.2.1 Wahl des Lernverfahrens

Trainiert wird ein rekurrentes Actor-Critic-Verfahren: RecurrentPPO, eine LSTM-Erweiterung von PPO [27, 29], umgesetzt über die Bibliothek Stable-Baselines3 [28]. Drei Eigenschaften der Aufgabe führen zu dieser Wahl.

Erstens ist der Aktionsraum diskret, was sowohl wertbasierte als auch policy-basierte Verfahren erlaubt und die Wahl allein noch nicht entscheidet. Zweitens ist die Aufgabe partiell beobachtbar (Abschnitt 2.2.2): Der Agent sieht nur einen Ausschnitt der Welt, wofür eine Politik mit Gedächtnis im Allgemeinen besser geeignet ist als eine rein reaktive. Drittens war die Umgebung während der Entwicklung selbst noch fehlerbehaftet; ein off-policy-Verfahren trägt die Folgen eines zwischenzeitlich fehlerhaften Reward-Signals über dessen Korrektur hinaus im Replay-Buffer mit, ein on-policy-Verfahren wie PPO nicht. Diese Robustheit gegenüber eigenen Fehlern gab den Ausschlag.

Die Trainings-Infrastruktur unterstützt zusätzlich DQN und A2C als mögliche Vergleichsverfahren; sie wurden auf der finalen Umgebung nicht vermessen, was als Grenze der Arbeit ausgewiesen wird (Abschnitt 6). Die vollständige Parameterbelegung steht in Tabelle 4 am Ende dieses Abschnitts.

#### 3.2.2 Curriculum

Ein Agent, der von Anfang an auf der vollen Zielentfernung von 35 bis 45 Feldern trainiert wird, erreicht den Exit fast nie und erhält dadurch kaum verwertbares Lernsignal. Das Training wird deshalb in vier Phasen mit wachsender Exit-Distanz gegliedert (Curriculum

Learning, Abschnitt 2.2.9). Eine Phase endet, sobald die Politik eine feste Ziel-Erfolgsquote erreicht, spätestens aber nach einem festen Schrittbudget; Abbildung 7 zeigt den Ablauf, Tabelle 3 die genauen Distanzbereiche, Budgets und Ziel-Erfolgsquoten je Phase.

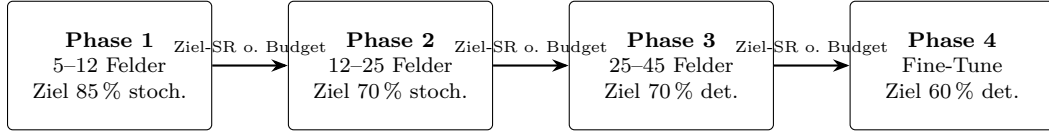


Abbildung 7: Methodik - Ablauf des leistungsorientierten Curriculums über vier Phasen, eigene Darstellung.

Der wachsende Abstand verändert nebenbei, wie sichtbar das Ziel für den Agenten überhaupt ist. Eine Messung über 200 frisch erzeugte Welten je Phase zeigt: Der Exit liegt beim Episodenstart in Phase 1 noch in 72 % der Welten im Sichtfeld, in Phase 2 nur noch in 6 %, in Phase 3 in keiner. Das Curriculum führt die Politik damit implizit von einfacher Zielsuche im Sichtfeld zu langreichweitiger Kompass-Navigation mit Gedächtnis, ohne dass das Sichtfeld selbst verändert werden muss. Ein zusätzliches Curriculum über die Sichtfeldgröße wäre daher redundant und zudem riskant, weil ein großes Sichtfeld anfangs eine rein reaktive Strategie belohnen würde, die beim späteren Verkleinern wertlos wird.

| Phase | Exit-Distanz |       | Budget | Ziel-SR | Gating | ent_coef     |
|-------|--------------|-------|--------|---------|--------|--------------|
|       | Training     | Eval  |        |         |        |              |
| 1     | 5–12         | 5–12  | 500k   | 85 %    | stoch. | 0.05         |
| 2     | 12–25        | 12–25 | 500k   | 70 %    | stoch. | 0.05         |
| 3     | 25–45        | 35–45 | 1,0M   | 70 %    | det.   | 0.05 → 0.001 |
| 4     | 25–45        | 35–45 | 200k   | 60 %    | det.   | 0.0001       |

Tabelle 3: Methodik - Distanzbereiche, Schrittbudget, Ziel-Erfolgsquote und Gating-Metrik der vier Curriculum-Phasen (Stand v11), eigene Darstellung.

Das Distanz-Curriculum staffelt dabei implizit auch die *Beobachtbarkeit* des Ziels, und damit die Fähigkeit, die je Phase gelernt werden muss. Eine Messung über je 200 frisch erzeugte Welten pro Phasenkonfiguration ergibt (eine Eigenschaft des Generators, unabhängig von den Evaluations-Seedmengen): In Phase 1 liegt der Exit beim Episodenstart in **72 %** der Welten bereits im  $15 \times 15$ -Sichtfeld (zu lernen: „geh zum sichtbaren Ziel, weiche lokalen Hindernissen aus“); in Phase 2 nur noch in **6 %** (zu lernen: dem Kompass folgen, Umwege um Hindernisse); in Phase 3 in **0 %** (reine Kompass-Navigation über lange Distanz, inklusive Sackgassen, die nur mit Gedächtnis überwindbar sind). Ein zusätzliches Curriculum über die Sichtfeldgröße (in der Literatur als Übergang von voller zu partieller Beobachtbarkeit beschrieben) wäre daher redundant; zudem wäre es hier riskant, weil ein großes Sichtfeld anfangs eine *reaktive* Strategie ohne Gedächtnisnutzung belohnt, die beim Verkleinern des Fensters wertlos wird (dieselbe Nicht-Übertragbarkeit, die die Gegenüberstellung in Abschnitt 5.2.1 für das BFS-Orakel zeigt).

### 3.2.3 Evaluationsprotokoll

Alle in dieser Arbeit berichteten Messungen folgen dem hier festgelegten Protokoll. Mehrere seiner Festlegungen, insbesondere das Episodenlimit und die Seed-Trennung, haben sich als ergebnisbestimmend erwiesen.

**Metrik.** Zentrale Metrik ist die *Erfolgsquote* (*Success Rate*, SR): der Anteil von 50 festen, trainingsfremden Welten, in denen der Agent den Exit innerhalb des Episodenlimits

erreicht. Ergänzend wird die *Pfادهffizienz*  $\eta$  berichtet, das Verhältnis aus kürzestem Weg und tatsächlich benötigten Schritten (Abschnitt 5.2.3). Als Diagnosemetrik des Trainings dienen `explained_variance` des Critics und `approx_kl`. Eine hohe Erfolgsquote allein ist dabei kaum aussagekräftig: Bei einem Episodenlimit von 4000 Schritten und einer mittleren Optimallänge von rund 40 Feldern erreicht schon eine ungelernete Kompass-Heuristik 92 % (Abschnitt 5.2.3). Die Erfolgsquote wird deshalb stets zusammen mit der Pfادهffizienz und einer ungelernen Referenzpolitik berichtet.

**Episodenlimit.** Gemessen wird stets mit dem vollen Limit von 4000 Schritten, dem Maximum der Umgebung: Bei einem Limit von 600 Schritten misst sich dasselbe Modell nur als 48-%-, bei 4000 Schritten als 86-%-Agent, weil erfolgreiche Episoden ohne Orakel lange Suchwege enthalten und ein zu knappes Limit gerade die kompetenten, aber langsamen Läufe abschneidet. Ein Episodenlimit gehört deshalb zu jedem berichteten SR-Wert dazu. Zusätzlich bleibt die Early-Stop-Truncation der Umgebung aktiv (256 Schritte ohne positiven Reward, Abschnitt 4.2) und gilt für Agent und Referenzpolitiken gleichermaßen (Abschnitt 5.2.3); die berichteten Erfolgsquoten sind dadurch eine untere Schranke gegenüber einem Protokoll ohne Truncation.

**Seed-Trennung.** Die Seed-Mengen sind disjunkt und fest: Ein Validierungsbereich (6000–6199, davon 6000–6049 für Phasenwechsel und Modellauswahl) steuert Training und Diagnose. Testset A (7000–7049) und Holdout B (8000–8049) sind ausschließlich für die finale Messung reserviert und fließen in keine Auswahl- oder Diagnoseentscheidung ein.

**Betriebsarten.** Jede Evaluation misst beide Betriebsarten, stochastisch (Sampling aus der Politik) und deterministisch (Argmax). Bei der rekurrenten Politik werden LSTM-Zustand und Episodenstart-Signal bei jedem Vorhersageschritt korrekt geführt und zu Beginn jeder Episode zurückgesetzt; wird das versäumt, misst die Auswertung ein Gedächtnismodell ohne Gedächtnis und damit systematisch zu niedrige Werte, ohne dass der Fehler auffällt [13].

**Aggregation und Kurvenvergleich.** Für das Endergebnis werden sieben unabhängige Trainingsläufe gemittelt und mit Standardabweichung berichtet, nie der beste Einzellauf [7]. Vergleiche zwischen Konfigurationen werden zudem nur auf ganzen Eval-Kurven gezogen, nie auf einzelnen Endständen, weil die Lerndynamik dieser Umgebung stark schwankt: Eine Änderung der Batch-Größe (8 auf 64) etwa galt nach einem günstigen Messpunkt zunächst als Verbesserung, erwies sich bei Betrachtung der gesamten Kurve aber als destabilisierend.

### 3.2.4 Versuchsaufbau

**Herkunft der Erfolgsschwellen.** Die in Abschnitt 1.4 genannten Schwellen (70 % auf Testset A, 60 % auf Holdout B) ersetzen die im Exposé angesetzten  $\geq 90\%$  über 100 unbekannte Seeds und stehen bereits seit dem 16.05.2026 im Experiment-Changelog, also vor den finalen Läufen und ohne Kenntnis der späteren Ergebnisse (kein Fall von *HARKing* [32]). Drei zu Projektbeginn unbekannte Eigenschaften der Aufgabe machen einen Wert nahe 90 % unerreichbar: die partielle Beobachtbarkeit der Umgebung [23, 30], die vor der Umgebungsrevision v11 als Luftlinie unterschätzte Weglänge (Tabelle 11) und eine Streuung von rund  $\pm 12$  Prozentpunkten über Trainingsläufe (Tabelle 16). Dass am Ende nur das Holdout-, nicht aber das Testset-Kriterium erreicht wird, zeigt: Auch die revidierte Schwelle war keine bequeme. Gegen die ursprüngliche Exposé-Metrik gemessen verfehlen Testset A und Holdout B mit  $66,3\% \pm 12,1$  (stochastisch) die 90 %-Schwelle deutlich, die zweite Zielgröße (minimale Schritte pro Episode) mit einer Pfادهffizienz von  $\eta \approx 0,05$  (Abschnitt 5.2.3) noch deutlicher.

Alle Läufe der Arbeit teilen die in Tabelle 4 gelistete Basiskonfiguration. Je Lauf existiert eine `config.json` mit genau diesen Werten; die vollständige Historie inklusive Zwischenständen liegt im Experiment-Changelog.

Auf dieser Basis wurden drei Varianten mit je einem veränderten inhaltlichen Faktor gerechnet (E1 Critic-Gewicht, E2 Curriculum-Zuschnitt, E3 Gedächtnisgröße). Da sie einer Fragestellung dienen, die sich erst aus den Messergebnissen stellt, sind Setup und Befund gemeinsam in Abschnitt 5.2.5 dargestellt.

| Komponente                                                         | Wert                                                                   |
|--------------------------------------------------------------------|------------------------------------------------------------------------|
| Algorithmus                                                        | RecurrentPPO (SB3-contrib), <code>MlpLstmPolicy</code>                 |
| $n_{\text{steps}}$ / <code>batch_size</code> / $n_{\text{epochs}}$ | 256 / 8 / 10                                                           |
| Lernrate / $\gamma$ / $\lambda_{\text{GAE}}$ / clip                | $3 \cdot 10^{-4}$ / 0.999 / 0.95 / 0.2                                 |
| <code>ent_coef</code>                                              | 0.05 (Phase 3: Annealing auf 0.001; Phase 4: 0.0001)                   |
| <code>vf_coef</code>                                               | 0.5                                                                    |
| LSTM                                                               | 1 Schicht, 256 Hidden Units, separater Critic-LSTM                     |
| Beobachtung                                                        | 229 Features (Grid $15 \times 15$ + HP + Kompass + Step-Frac), Env v11 |
| Parallele Umgebungen                                               | 16 ( <code>DummyVecEnv</code> ), Device CPU                            |
| Seed-Pool („Swarm“)                                                | aktiv, Sampling-Wahrscheinlichkeit 0.3                                 |
| Curriculum                                                         | 4 Phasen (Tab. 3), leistungsbasiert mit Schrittbudget                  |
| Eval                                                               | Testset A 7000–7049, Holdout B 8000–8049, Cap 4000, det + stoch        |

Tabelle 4: Methodik - Gemeinsame Basiskonfiguration aller v12-Läufe, eigene Darstellung.

## 4 Implementierung (gemeinsam)

Kapitel 3 hat begründet, welche Verfahren gewählt wurden und warum. Dieses Kapitel zeigt die technische Umsetzung dieser Entscheidungen, in zwei aufeinander aufbauenden Abschnitten: Abschnitt 4.1 behandelt das Spiel, Abschnitt 4.2 die darauf aufsetzende Lernumgebung. Beide Teile sind technische Ingenieursleistungen gleichen Gewichts und Bestandteil derselben Implementierung; die Trennung beschreibt die Zuständigkeit (Tabelle 2), nicht eine Rangfolge.

### 4.1 Implementierung des Spiels Stoneforge (Laurin)

Stoneforge ist ein selbst entwickeltes, rundenbasiertes 2D-Spiel: ein C++-Simulationskern, ein grafischer Client, in dem ein Mensch das Spiel tatsächlich spielen kann, und eine Python-Anbindung über pybind11, über die ein Lernverfahren *dieselbe* Simulation ansteuert. Was der Agent erlebt, ist damit bis auf die Wahrnehmung identisch mit dem, was ein Spieler erlebt.

Dieser Abschnitt behandelt das Spiel: den Simulationskern, den Weltgenerator und den grafischen Client. Er endet mit einer Welt, die reproduzierbar, lösbar und in ihrer Zielentfernung steuerbar ist, aber von Reinforcement Learning noch nichts weiß. Abschnitt 4.2 setzt darauf auf und macht daraus eine Lernumgebung. Der Schwerpunkt dieses Abschnitts liegt auf dem Weltgenerator (Abschnitt 4.1.3), weil er festlegt, wie die Welten aussehen, in denen später alles gemessen wird.

#### 4.1.1 Technologie-Stack

Das System besteht aus drei Schichten: einem C++-Kern für Simulation und Weltgenerierung, einem Python-Binding und dem Python-Trainingsstack. Tabelle 5 listet die verwendeten Komponenten.

| Komponente      | Technologie                         | Zweck                                                     |
|-----------------|-------------------------------------|-----------------------------------------------------------|
| Simulationskern | C++20 (ca. 4300 LOC)                | Weltgenerierung, Spiellogik, Reward, BFS                  |
| Binding         | pybind11 2.13                       | C++-Kern als Python-Modul ( <code>stoneforge_sim</code> ) |
| Build           | CMake                               | Kern, Binding, Clients                                    |
| RL-Umgebung     | Python 3.12, Gymnasium 1.2          | <code>StoneforgeWorldEnv</code> (Gym-API)                 |
| RL-Algorithmen  | Stable-Baselines3 / sb3-contrib 2.8 | PPO, DQN, A2C, RecurrentPPO                               |
| Deep Learning   | PyTorch 2.12 (CPU)                  | Netz- und LSTM-Training                                   |
| Spiel-Client    | raylib 5.5 (C++, ca. 4500 LOC)      | spielbarer Client, KI-Zuschaumodus                        |
| Monitoring      | TensorBoard, WebSockets             | Trainingsmetriken, Live-Visualisierung                    |
| Auswertung      | NumPy, Matplotlib                   | Evaluation, Abbildungen                                   |

Tabelle 5: Implementierung - Technologie-Stack der Simulation und des Trainings, eigene Darstellung.

Die Simulation ist deterministisch pro Seed. Training und Evaluation laufen vollständig auf der CPU; ein Vergleich mit der GPU ergab für dieses kleine Netz keine Beschleunigung. Der Durchsatz liegt je nach Konfiguration bei ca. 88–190 Umgebungsschritten pro Sekunde; die für die Endergebnisse verwendete Konfiguration (`batch_size=8`) liegt am unteren Ende dieser Spanne. Die verwendete Hardware und der zugehörige Benchmark sind in einem kontrollierten Vergleich beider Geräteklassen ermittelt worden.

#### 4.1.2 Die Bausteine: Tile, Chunk und Welt

Bevor von Generierung die Rede sein kann, braucht es die drei Dinge, die erzeugt werden. Sie bauen streng aufeinander auf.

**Das Tile.** Die kleinste Einheit der Welt ist ein Feld auf einem ganzzahligen Gitter. Ein Tile trägt genau einen Typ aus einer Aufzählung von 15 Werten (`TileType` in `types.hpp`): leer, Wand, Erz, Exit, Baum, Werkbank, Holzwand, Holzstamm und sieben biomspezifische Struktur-Tiles. Es gibt keine Höhe, keine Ebenen und keine Teilbelegung, ein Feld ist genau eine Sache.

Jedem Typ ist ein `WorldObject` zugeordnet, das seine Eigenschaften kennt, darunter die für alles Weitere entscheidende Frage, ob man darüber laufen kann. Die Voreinstellung lautet *nein*: Nur leere Felder und der Exit sind begehbar, alles andere blockiert. Das klingt nach einem Detail, hat aber Folgen bis in die Ergebnisse, denn Bäume und Erzadern sind damit genauso Hindernis wie Wände (Tabelle 8).

**Der Chunk.** Tiles werden nicht einzeln verwaltet, sondern in Blöcken von  $8 \times 8$  Feldern. Ein Chunk ist ein flaches Array aus 64 Tile-Typen plus einem Flag, ob er bereits erzeugt wurde. Mehr nicht.

**Die Welt.** `World` hält eine Hashtabelle von Chunk-Koordinate auf Chunk. Diese Tabelle ist zu Beginn leer und füllt sich erst, während gespielt wird: Jeder Zugriff auf ein Tile rechnet die Weltkoordinate in Chunk- und lokale Koordinate um, und findet sich der Chunk nicht in der Tabelle, wird er in diesem Moment erzeugt und abgelegt. Materialisiert wird also nur, was jemand tatsächlich ansieht.

Daraus folgt die zentrale Eigenschaft des Generators, und sie ist bewusst so gebaut: Die Erzeugung eines Chunks hängt *ausschließlich* vom Seed und seinen eigenen Koordinaten ab, von keinem anderen Chunk und von keinem Zustand. Der Generator ist eine reine Funktion

$$(seed, c_x, c_y) \mapsto 64 \text{ Tile-Typen.} \quad (5)$$

Die Welt ist damit praktisch unbegrenzt, ohne je gespeichert zu werden, und zwei Durchläufe liefern denselben Chunk unabhängig davon, in welcher Reihenfolge der Spieler die Welt betreten hat.

**Warum das nicht nur elegant, sondern notwendig ist.** Genau diese Reproduzierbarkeit trägt später das Evaluationsprotokoll (Abschnitt 3.2.3). Testset A besteht nicht aus gespeicherten Karten, sondern aus 50 Zahlen (Seeds 7000 bis 7049). Der Seed *ist* die Welt. Nur weil derselbe Seed bitgenau dieselben Tiles liefert, lassen sich sieben Trainingsläufe auf demselben Testset vergleichen, lässt sich ein Trainingsseed sauber von einem Testseed trennen, und lässt sich eine Episode im grafischen Client exakt nachspielen. Ein Generator mit einem fortlaufenden Zufallsgenerator statt einer reinen Funktion würde all das zerstören, weil dann die Reihenfolge der Aufrufe das Ergebnis mitbestimmt.

#### 4.1.3 Der Weltgenerator

Damit ist die Frage gestellt, die dieser Abschnitt beantwortet: Wie werden aus einem Seed und zwei Chunk-Koordinaten 64 Tile-Typen? Die begrifflichen Grundlagen dazu (Hash- gegenüber Wertrauschen, Domain Warping, zelluläre Automaten, Breitensuche) stehen in Abschnitt 2.1.2; hier geht es darum, wie sie zusammengeschaltet sind.

Der Einstieg ist `World::generateChunk()`, und die Funktion ist kurz genug, um sie ganz zu zeigen (Listing 1). Sie ist die Landkarte für alles Weitere: Erst wird jedes der 64 Felder einzeln ausgewürfelt, dann optional geglättet, dann eventuell eine Landmarke eingestempelt. Mehr passiert nicht. Die Biomwahl steckt dabei in `sampleBaseTile` und wird gleich als eigene Stufe behandelt.

```
1 void World::generateChunk(int cx, int cy, Chunk& chunk) const {
```

```

2     const auto& cfg = gameConfig().world;
3
4     // Stufe 0+1: Basisbelegung, Feld fuer Feld
5     for(int ly = 0; ly < kChunkSize; ++ly) {
6         for(int lx = 0; lx < kChunkSize; ++lx) {
7             const int wx = cx * kChunkSize + lx;    // lokal ->
8                 Weltkoordinate
9             const int wy = cy * kChunkSize + ly;
10            chunk.tiles[ly * kChunkSize + lx] = sampleBaseTile(wx, wy, cfg)
11            ;
12        }
13    }
14    // Stufe 2: optionale Glaettung (in dieser Arbeit abgeschaltet)
15    runCellularSmoothingStage(cx, cy, chunk, cfg);
16    // Stufe 3: seltene Landmarke
17    placeBiomeStructure(cx, cy, chunk);
18 }

```

Listing 1: Die vollständige Erzeugung eines Chunks (`World::generateChunk()`, `src/core/world.cpp`). Jedes Feld wird unabhängig aus seinen Weltkoordinaten bestimmt; nichts in dieser Funktion greift auf einen Nachbar-Chunk oder auf gespeicherten Zustand zu.

**Woher der Zufall kommt.** Alle Stufen ziehen ihre Zufallszahlen aus einer einzigen Funktion, `noise01(x, y, salt)` (Listing 2). Sie tut nichts weiter, als ihre drei Argumente zu einer 64-Bit-Zahl zu verrühren und daraus einen Wert in  $[0, 1)$  zu machen: Die Koordinaten und der Seed werden mit großen ungeraden Konstanten multipliziert und per XOR verknüpft, dann läuft das Ergebnis durch einen Bit-Mixer, dessen Multiplikatoren aus `SplitMix64` stammen [34]. Der Mixer sorgt dafür, dass sich benachbarte Eingaben auf völlig verschiedene Ausgaben verteilen. Die unteren 48 Bit ergeben, skaliert, die Zufallszahl.

```

1 double World::noise01(int worldX, int worldY, std::uint64_t salt) const {
2     std::uint64_t value = static_cast<std::uint64_t>(worldX) * 0
3         x9e3779b185ebca87ULL;
4     value ^= static_cast<std::uint64_t>(worldY) * 0xc2b2ae3d27d4eb4fULL;
5     value ^= seed_ * 0x165667b19e3779f9ULL;
6     value ^= salt;
7     value = mix(value);    // SplitMix64-Finalizer
8
9     constexpr double kScale = 1.0 / static_cast<double>(0xFFFFFFFFFULL);
10    return static_cast<double>(value & 0xFFFFFFFFFULL) * kScale;
11 }

```

Listing 2: Die zustandsfreie Rauschquelle des Generators (`src/core/world.cpp`). Der Rückgabewert hängt ausschließlich von den Argumenten ab, nicht davon, wann oder wie oft die Funktion aufgerufen wird.

Der Unterschied zu einem gewöhnlichen Zufallszahlengenerator ist der Kern der Sache. Ein solcher Generator besitzt einen Zustand und liefert seine Werte in einer festen Reihenfolge; wer den tausendsten Wert braucht, muss die 999 davor ziehen. Damit hinge die Welt davon ab, in welcher Reihenfolge man ihre Chunks erzeugt, und die in Abschnitt 4.1.2 begründete Reproduzierbarkeit wäre dahin. `noise01` hat keinen Zustand. Der Wert eines Feldes hängt nur von dessen Koordinaten ab, ist also jederzeit, beliebig oft und in beliebiger Reihenfolge neu berechenbar, immer mit demselben Ergebnis.

Das dritte Argument, der *Salt*, ist eine feste Konstante je Merkmal. Er fließt in den Hash ein und erzeugt so aus derselben Koordinate mehrere voneinander unabhängige Rauschfelder. Der Generator hält getrennte Salts für Wanddichte, Erz und Bäume bereit. Ohne sie lägen alle drei Merkmale an exakt denselben Stellen, weil sie dieselbe Zahl auswerten würden.

Damit ist auch schon gesagt, welche Art von Rauschen hier vorliegt: Es ist *Hash-Rauschen* im Sinne von Abschnitt 2.1.2, also unkorreliertes weißes Rauschen. Nachbarfelder wissen nichts voneinander. Abbildung 8 zeigt, was das für die Form der Welt bedeutet, und der Unterschied ist drastisch: Derselbe Schwellwert schneidet aus Hash-Rauschen verstreute Einzelfelder, aus interpoliertem Wertrauschen dagegen zusammenhängende Barrieren. Diese Eigenschaft wird uns am Ende des Abschnitts wieder einholen.

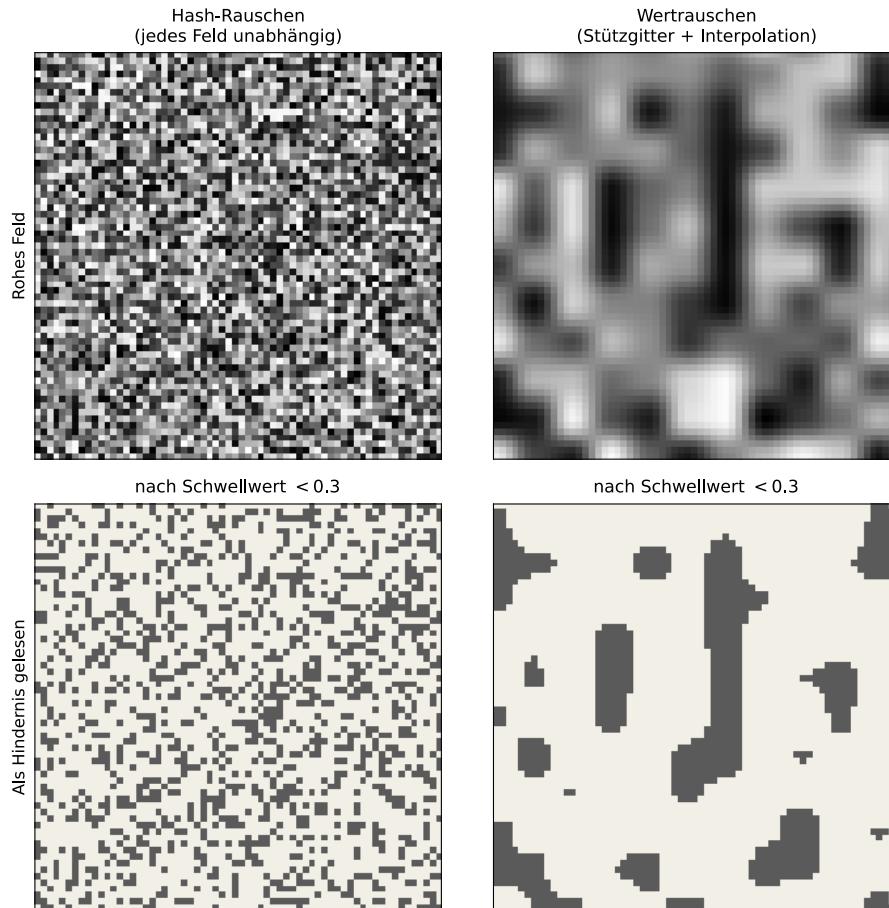


Abbildung 8: Implementierung - Hash-Rauschen gegenüber interpoliertem Wertrauschen, oben die Rohfelder und unten die daraus gelesenen Hindernisse, eigene Darstellung.

### Stufe 0: Welches Biom hat dieser Chunk?

Zuerst bekommt der *ganze Chunk* ein Biom, nicht jedes Feld einzeln. Das ist eine bewusste Vergröberung: Sie hält die Landschaft zusammenhängend, statt Wald und Wüste feldweise abwechseln zu lassen.

Für das Hash-Rauschen aus Listing 2 allein wäre das nicht zu haben, denn es liefert für benachbarte Chunks völlig unabhängige Werte. Die Stufe baut deshalb daraus *Wertrauschen* (Abschnitt 2.1.2): Sie wertet `noise01` nur an den Ecken eines groben Stützgitters aus und interpoliert dazwischen bilinear. Damit die Übergänge nicht als Gitterkanten sichtbar werden, läuft der Interpolationsparameter zuvor durch Perlins Glättungsfunktion  $3t^2 - 2t^3$  [20]. Benachbarte Chunks bekommen so ähnliche Werte, und es entstehen Regionen statt Flimmern.

Darüber liegen zwei weitere Kunstgriffe (Listing 3). Erstens werden drei solcher Felder mit steigender Frequenz gewichtet addiert (0,62 : 0,28 : 0,10); das grobe Feld gibt die Grundform vor, die feineren rauhen die Ränder auf. Zweitens *Domain Warping*: Bevor das Feld ausgewertet



wird, verschieben zwei weitere Rauschfelder die Abtastkoordinate selbst. Ohne diesen Schritt lägen die Biomgrenzen erkennbar am Stützgitter und sähen rechteckig aus.

```

1 double x = static_cast<double>(cx) * 0.22;    // Chunk -> Feldkoordinate
2 double y = static_cast<double>(cy) * 0.22;
3
4 // Domain Warping: die Abtastkoordinate selbst verrauschen
5 const double warpX = sampleValue(x * 0.57 + 19.3, y * 0.57 - 7.1, ...);
6 const double warpY = sampleValue(x * 0.57 - 11.8, y * 0.57 + 13.4, ...);
7 x += (warpX - 0.5) * 2.8;
8 y += (warpY - 0.5) * 2.8;
9
10 // Drei Oktaven: grobe Form + zwei feinere Detailebenen
11 const double n1 = sampleValue(x, y, ...);
12 const double n2 = sampleValue(x * 2.1 - 3.7, y * 2.1 + 1.9, ...);
13 const double n3 = sampleValue(x * 4.2 + 8.4, y * 4.2 - 5.6, ...);
14
15 return std::clamp(n1 * 0.62 + n2 * 0.28 + n3 * 0.10, 0.0, 1.0);

```

Listing 3: Kern der Stufe 0 (`World::biomeFieldForChunk()`, `src/core/world.cpp`, gekürzt). `sampleValue` ist das interpolierte Wertrauschen; `warpX` und `warpY` verschieben die Abtastkoordinate, bevor die drei Oktaven überlagert werden.

Der Rückgabewert liegt in  $[0, 1)$  und wird in sieben gleich breite Intervalle geschnitten. Diese ergeben die Biome Grasland, Wald, Wüste, Bergland, Steppe, Tundra und Hölle. Weil die Chunk-Koordinaten mit 0,22 skaliert eingehen, verändert sich das Feld nur langsam: Ein Biom erstreckt sich typischerweise über vier bis fünf Chunks, also rund 36 Felder, und ist damit deutlich größer als das  $15 \times 15$ -Sichtfenster eines Agenten. Ein Agent sieht also fast nie eine Biomgrenze, sondern immer nur eine Landschaft.

### Stufe 1: Was liegt auf diesem einzelnen Feld?

Jetzt erst wird Feld für Feld entschieden, und hier fällt die Entscheidung, die für die Navigation zählt: Hindernis oder frei. Der Generator zieht für jedes Feld drei unabhängige Rauschwerte, einen für Wanddichte, einen für Erz, einen für Bäume, und vergleicht jeden mit einem Schwellwert, der vom Biom des Chunks abhängt. Die Schwellwerte stehen vollständig in Tabelle 7, die Entscheidungslogik in Listing 4.

```

1 const double density = noise01(worldX, worldY, cfg.densitySalt);
2 const double ore      = noise01(worldX, worldY, cfg.oreSalt);
3 const double trees    = noise01(worldX, worldY, cfg.treeSalt);
4 // wallThreshold / oreThreshold / treeThreshold: je nach Biom gesetzt
5
6 TileType tile = TileType::Empty;
7
8 // 1. Seen sind hindernisfrei und schlagen alles Weitere
9 if(lakeMaskAt(worldX, worldY)) { return TileType::Empty; }
10 // 2. Wand
11 if(density < wallThreshold) { tile = TileType::Wall; }
12 // 3. Erz, nur im Bergland; ueberschreibt auch eine Wand
13 if(biomeTag == 3 && ore < oreThreshold) { tile = TileType::Resource; }
14 // 4. Baum, nur auf noch leerem Feld
15 if(treeThreshold > 0.0 && tile == TileType::Empty && trees < treeThreshold)
16 {
17     tile = TileType::Tree;
18 }
19 return tile;

```

Listing 4: Die Entscheidung über ein einzelnes Feld (`World::sampleBaseTile()`,

`src/core/world.cpp`, gekürzt). Die Reihenfolge der vier Prüfungen ist die Regel: See schlägt alles, Erz überschreibt bedingungslos, ein Baum wächst nur auf einem bis dahin leeren Feld.

Zwei Dinge daran sind leicht zu übersehen. Erstens ist die Reihenfolge der Prüfungen selbst eine Regel und kein Implementierungsdetail: Erz wird bedingungslos gesetzt und kann eine bereits bestimmte Wand ersetzen, ein Baum dagegen nur dort wachsen, wo bis dahin nichts liegt. Zweitens ist `oreThreshold` für sechs der sieben Biome wirkungslos, weil die Prüfung zusätzlich `biomeTag == 3` verlangt. Erz gibt es ausschließlich im Bergland; die übrigen Zeilen der Erzspalte in Tabelle 7 laufen ins Leere.

Die vorgelagerte Seenmaske ergibt sich aus einer gewichteten Linearkombination zweier Rauschfelder,

$$Score = 0,75 a + 0,25 b, \quad (6)$$

und weist eine Zelle oberhalb eines harten Cutoffs von 0,86 als Wasser aus. Beide Felder werden dabei auf ganzzahlig geteilten Koordinaten ausgewertet ( $x/7$  bzw.  $x/3$ ), sind also blockweise konstant; die Seen sind deshalb grobkörnig und nicht rund. Sie sind hindernisfrei und überschreiben die Basisbelegung vollständig.

**Ein Chunk, Feld für Feld.** Wie das konkret aussieht, zeigt Tabelle 6 an einer echten Zeile aus einer Eval-Welt. Alle acht Felder liegen im selben Chunk und damit im selben Biom, sehen also dieselben Schwellwerte; allein die drei Rauschwerte unterscheiden sie. Feld  $(-32, -31)$  zeigt dabei die Vorrangregel: Die Dichte 0,330 hätte für sich genommen keine Wand erzeugt, der Erzwert 0,002 setzt trotzdem Erz, weil Erz eine Wand bedingungslos überschreibt. Die gezeigten Werte stammen aus einem Nachbau des Generators in Python, der auf 3663 geprüften Feldern dieser Welt ausnahmslos dieselben Tile-Typen liefert wie der C++-Kern.

| Feld         | Dichte       | Erz          | Baum         | Ergebnis |
|--------------|--------------|--------------|--------------|----------|
|              | < 0,25?      | < 0,10?      | < 0,02?      |          |
| $(-32, -31)$ | 0,330        | <b>0,002</b> | 0,622        | Erz      |
| $(-31, -31)$ | 0,401        | 0,249        | <b>0,013</b> | Baum     |
| $(-30, -31)$ | 0,712        | 0,164        | 0,537        | leer     |
| $(-29, -31)$ | <b>0,078</b> | 0,436        | 0,378        | Wand     |
| $(-28, -31)$ | 0,357        | 0,994        | 0,217        | leer     |
| $(-27, -31)$ | 0,370        | 0,376        | 0,154        | leer     |
| $(-26, -31)$ | <b>0,186</b> | 0,149        | 0,064        | Wand     |
| $(-25, -31)$ | 0,559        | 0,481        | 0,579        | leer     |

Tabelle 6: Implementierung - Belegung einer vollständigen Chunk-Zeile aus der Welt zu Seed 7000 mit den drei Rauschwerten je Feld, eigene Darstellung.

## Stufe 2: Glättung, in dieser Arbeit abgeschaltet

Auf die Basisbelegung könnte ein zellulärer Automat folgen, der verstreute Einzelhindernisse zu zusammenhängenden Formationen verschmilzt (Moore-Nachbarschaft, Geburts- und Überlebensschwelle, Abschnitt 2.1.2). Er ist implementiert, in allen berichteten Läufen aber abgeschaltet; Abbildung 10 zeigt ihn deshalb gestrichelt. Warum das Einschalten die Welten nicht härter, sondern leerer macht, steht weiter unten.

Seine Konstruktion verdient trotzdem eine Bemerkung, weil sie ein echtes Problem löst. Ein Automat braucht die acht Nachbarn jeder Zelle. Am Chunk-Rand liegen die im Nachbar-Chunk, und den zu erzeugen würde eine Abhängigkeit zwischen Chunks einführen und damit genau die Reinheit zerstören, auf der die Reproduzierbarkeit beruht. Die Stufe berechnet deshalb einen *Halo* in Breite der Iterationszahl aus der Basisbelegung mit, lässt den Automaten

darüber laufen und verwirft ihn danach. Das Ergebnis hängt so weiterhin nur von Seed und Chunk-Koordinaten ab.

### Stufe 3: Landmarken

Zuletzt setzt der Generator mit einer Wahrscheinlichkeit von  $P = 0,10$  je Chunk eine biomspezifische Landmarke. Ihre Form stammt aus handgeschriebenen ASCII-Matrizen der Größe  $5 \times 5$ , je eine pro Biom, die an einer aus dem Rauschen bestimmten Position im Chunk eingestempelt wird. Bei einer Chunk-Kantenlänge von 8 bleiben dafür drei Felder Spielraum in jeder Richtung. Chunks, die Spawn oder Exit enthalten, überspringt die Stufe, damit die Strukturen weder den Start noch das Ziel verbauen.

**Die Pipeline an einer echten Welt.** Abbildung 9 zeigt alle Stufen nebeneinander an einer der 50 Welten des Testsets. Links das Biomfeld, dessen Blöcke sichtbar an den Chunk-Grenzen ausgerichtet sind und dessen Regionen sich über mehrere Chunks erstrecken. In der Mitte die Basisbelegung: Der Biomwechsel ist als Dichtewechsel zu erkennen, im Bergland liegen sichtbar mehr Wände und Erz als in der Tundra. Rechts die fertige Welt mit Landmarken, Spawn, Exit und dem kürzesten Weg. Die Tile-Darstellungen stammen direkt aus dem C++-Kern; das im Python-Binding nicht zugängliche Biomfeld wurde aus einem Nachbau der Generatorfunktionen berechnet, der die Tile-Typen des Kerns auf allen 8115 geprüften Feldern dieses Ausschnitts exakt reproduziert.

Dieser Weg ist die eigentliche Aussage der Abbildung. Er ist eine grobe Treppe zwischen zwei Punkten und weicht kaum einmal seitlich aus. Ein knappes Viertel der Felder ist blockiert, und trotzdem gibt es nichts zu umrunden, weil die Hindernisse einzeln stehen statt Wände zu bilden. Genau das quantifiziert der Umwegfaktor weiter unten.

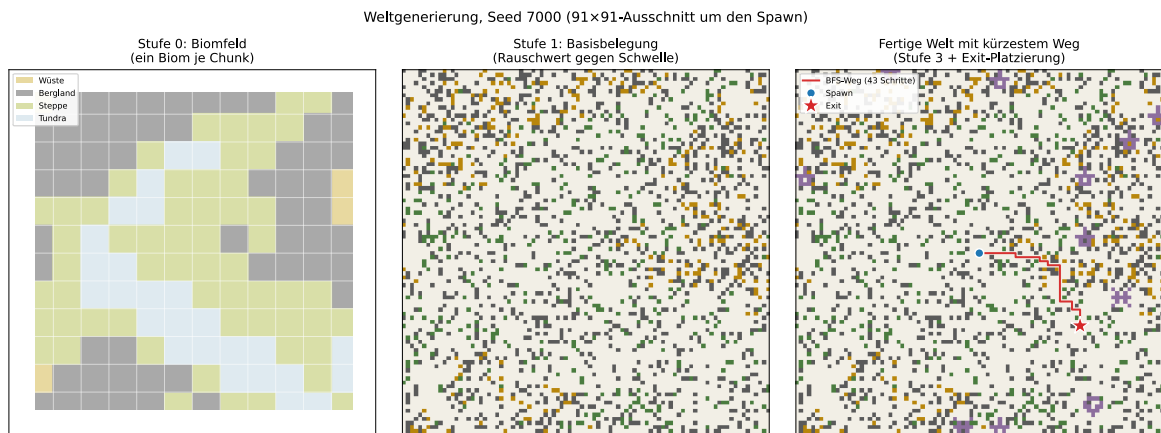


Abbildung 9: Implementierung - Biomfeld, Basisbelegung und fertige Welt zu Seed 7000 nebeneinander, eigene Darstellung.

Tabelle 7 listet die Schwellwerte der Stufe 1 je Biom: Ein Feld wird zur Wand, wenn sein Dichte-Rauschwert unter der Wandschwelle liegt, entsprechend für Erz und Bäume mit eigenen Rauschfeldern; Erz entsteht ausschließlich im Bergland, die Erzspalte der übrigen Biome bleibt wirkungslos. Die Werte sind im Quellcode der Weltgenerierung hartkodiert und über die Konfigurationsdatei nicht erreichbar.

**Was am Ende tatsächlich im Weg steht.** Die Schwellwerte in Tabelle 7 sind Parameter, keine Messwerte, und sie beschreiben allein die Wände. Wie viel eine erzeugte Welt dem Spieler wirklich in den Weg legt, ist eine andere Zahl: Weil Bäume, Erz und Landmarken ebenfalls blockieren, liegt der Anteil unpassierbarer Felder deutlich über jeder einzelnen

| Biom     | Wand | Erz   | Baum | Charakter                                |
|----------|------|-------|------|------------------------------------------|
| Wald     | 0,07 | 0,015 | 0,23 | dicht bewachsen, kaum Stein              |
| Grasland | 0,10 | 0,03  | 0,05 | offen                                    |
| Steppe   | 0,11 | 0,03  | 0,09 | offen, etwas mehr Bewuchs                |
| Tundra   | 0,14 | 0,05  | 0,07 | ausgeglichen                             |
| Wüste    | 0,19 | 0,08  | 0,00 | steinig, baumlos                         |
| Hölle    | 0,23 | 0,09  | 0,06 | steinig                                  |
| Bergland | 0,25 | 0,10  | 0,02 | dichtester Fels, <i>einziges Erzbiom</i> |

Tabelle 7: Implementierung - Wand-, Erz- und Baumschwellwerte der Basisbelegung je Biom, eigene Darstellung.

Wandschwelle. Tabelle 8 schlüsselt ihn auf, gezählt über die 100 Seeds beider Eval-Sets in einem  $81 \times 81$ -Fenster um den Spawn (656 100 Tiles): Nur ein knappes Sechstel der Welt besteht aus Wänden, Bäume und Erz steuern zusammen weitere sieben Prozentpunkte zur Undurchdringlichkeit bei. Die Zusammensetzung erklärt zugleich, warum die Wandschwellen im Bereich 0,07 bis 0,25 liegen, die weiter unten diskutierte Hindernisdichte aber bei rund 0,24: Es sind zwei verschiedene Größen, und nur die zweite entscheidet über die Geometrie der Wege.

| Tile                       | Anteil        | begehrbar |
|----------------------------|---------------|-----------|
| Leer                       | 0,7511        | ja        |
| Wand                       | 0,1562        | nein      |
| Baum                       | 0,0421        | nein      |
| Erz                        | 0,0322        | nein      |
| Landmarken                 | 0,0183        | nein      |
| Exit                       | 0,0002        | ja        |
| <b>unpassierbar gesamt</b> | <b>0,2487</b> |           |

Tabelle 8: Implementierung - Tile-Zusammensetzung der erzeugten Eval-Welten über die 100 Seeds beider Testsets, eigene Darstellung.

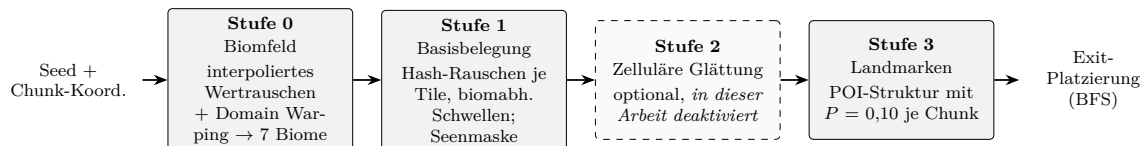


Abbildung 10: Implementierung - Vierstufige Pipeline der Weltgenerierung je Chunk mit nachgelagerter Exit-Platzierung, eigene Darstellung.

**Nachbearbeitung und eine Grenze der Konfigurierbarkeit.** Nach den vier Stufen räumt der Generator noch feste Radien um Spawn und Exit frei ( $5 \times 5$  bzw.  $3 \times 3$  Felder), damit weder Spieler noch Ziel in einem Hindernis stecken. Die Biomzuordnung dagegen ist starr: Die Sieben-Intervall-Teilung des Biomfeldes steht im Code und lässt sich über `game_config.json` nicht verändern. Ein Audit der Konfiguration entfernte hier mehrere Schlüssel (`coldBiomeMax`, `warmBiomeMax` und die `moss`-Schwellwerte), die zwar eingelesen, durch die feste Binning-Logik aber nie ausgewertet wurden. Sie versprochen ein Tuning, das faktisch wirkungslos war (Tabelle 11). Dasselbe gilt für die Schwellwerte in Tabelle 7: Wer die Welten härter machen will, muss den Generator anfassen, nicht die Konfiguration.

Die Exit-Platzierung und die Lösbarkeitssicherung setzen auf der so erzeugten Welt auf und sind in Abschnitt 4.1.4 beschrieben. Der Generator implementiert darüber hinaus die in

Abschnitt 2.1.2 genannten Sicherheitsnetze. Für die Bewertung der Ergebnisse ist entscheidend, welche davon in den berichteten Läufen tatsächlich wirksam waren (Tabelle 9).

| Mechanismus                                              | Aktiv | Begründung                                                                                                          |
|----------------------------------------------------------|-------|---------------------------------------------------------------------------------------------------------------------|
| BFS-Exit-Platzierung nach Laufweg                        | ja    | Garantiert Erreichbarkeit per Konstruktion und steuert zugleich die Curriculum-Distanz (Tabelle 11).                |
| Zelluläre Glättung (Stufe 2)                             | nein  | Abgeschaltet; die Welten entstehen allein aus Stufe 1.                                                              |
| Flood-Fill-Validierung des Start-Ziel-Fensters           | nein  | Durch die BFS-Platzierung redundant: Der Exit wird ausschließlich aus begehbar erreichbaren Zellen gewählt.         |
| Carving ( <code>forceGuaranteedPath</code> und Fallback) | nein  | Aus demselben Grund nicht erforderlich; damit entfällt zugleich der gerade Korridor als strukturelle Auffälligkeit. |

Tabelle 9: Implementierung - Im Generator implementierte Mechanismen und ihr Status in der berichteten Konfiguration, eigene Darstellung.

**Eine Konsequenz für die Ergebnisse.** Aus Tabelle 9 folgt eine Eigenschaft der Welten, die für die Bewertung in Abschnitt 5.2.3 unmittelbar relevant ist: Da die Glättung abgeschaltet ist, stammt die Hindernisverteilung ausschließlich aus *unkorreliertem* Hash-Rauschen. Jedes Feld entscheidet für sich, ohne seine Nachbarn zu kennen, denn `noise01` ist genau dafür gebaut. Die Hindernisse sind damit statistisch unabhängig voneinander verteilt und bilden überwiegend verstreute Einzelfelder statt zusammenhängender Barrieren (Abbildung 4). Was den Generator reproduzierbar macht, kostet ihn also zugleich jede Struktur: Genau die Eigenschaft, die Abschnitt 4.1.2 als notwendig begründet, verhindert hier Sackgassen. Genau diese Struktur macht die Welten „offen“ im Sinne von Abschnitt 5.2.3: Sackgassen, die Gedächtnis erfordern würden, entstehen selten, und eine ungerichtete Zufallsbewegung verlässt sie zuverlässig.

**Wie offen die Welten sind, misst der Umwegfaktor.** Quantifizieren lässt sich diese Offenheit über das Verhältnis aus BFS-Laufweg und Manhattan-Distanz zwischen Spawn und Exit. Bei 1,0 ist der kürzeste Weg die Luftlinie, es gibt also nichts zu umrunden. Über beide Eval-Sets gemessen (100 Seeds, `scripts/probe_world_geometry.py`) liegt der Faktor bei **1,10**, bei einer Hindernisdichte von 0,238 (Tabelle 8): Der kürzeste Weg ist zehn Prozent länger als die Luftlinie. Ein knappes Viertel der Welt ist blockiert, und trotzdem gibt es fast nichts zu umrunden. Das ist die quantitative Ursache dafür, dass ein vierzeiliger Kompass-Folger 92 % erreicht (Abschnitt 5.2.3). Nicht die Politik ist stark, die Geometrie ist schwach.

**Die naheliegende Reparatur funktioniert nicht.** Es liegt nahe, dagegen die vorhandene Glättungsstufe einzuschalten; in der PCG-Literatur ist gerade sie es, die aus verstreutem Rauschen konkave Strukturen formt [10, 1]. Für diesen Generator haben wir das nachgemessen, und es trifft nicht zu. Mit den hinterlegten Regeln (Geburt ab 5, Überleben ab 4, zwei Iterationen) fällt die Waddichte von 0,238 auf 0,037 und der Umwegfaktor auf 1,001. Die Welt wird zur leeren Ebene statt zum Labyrinth. Schuld ist die niedrige Ausgangsdichte: Bei 24 % Wandanteil hat kaum ein Wandtile die geforderten fünf soliden Nachbarn, also stirbt fast jede Wand und keine wird geboren. Wer die Stufe aktiviert, treibt die Erfolgsquote der ungelerten Heuristik nach oben statt sie einbrechen zu lassen.

Der Regelsweep in Tabelle 10 ( $b$  die Geburts-,  $s$  die Überlebensschwelle des Automaten) zeigt darüber hinaus einen harten Zielkonflikt. Jede Kombination, die den Umwegfaktor spürbar hebt, zerstört die Lösbarkeit; alles, was bei 50 von 50 lösbaren Welten bleibt, liegt beim Umweg im Rauschen der Referenz. Die Ursache liegt außerhalb des Automaten. `enableFloodFillValidation` und `enableMacroGraphPrecheck` stehen beide auf `false` (`assets/base/game_config.json`), und die Basisdichte ist in `World::sampleBaseTile()` hartkodiert. Der Automat zählt Nachbarn, von Erreichbarkeit weiß er nichts. Eine Härtung der Umgebung ist deshalb kein Konfigurationsschalter, sondern ein Eingriff in den Generator (Abschnitt 6).

| Regel der Glättungsstufe             | lösbar      | Wanddichte | BFS  | Umweg        | p90   |
|--------------------------------------|-------------|------------|------|--------------|-------|
| aus (berichtete Konfiguration)       | 50/50       | 0,240      | 40,1 | <b>1,123</b> | 1,312 |
| an, 2 Iter., $b=5$ / $s=4$ (Default) | 50/50       | 0,036      | 40,1 | 1,001        | 1,000 |
| an, 2 Iter., $b=4$ / $s=3$           | 50/50       | 0,169      | 40,1 | 1,101        | 1,327 |
| an, 1 Iter., $b=3$ / $s=3$           | 50/50       | 0,271      | 39,8 | 1,145        | 1,637 |
| an, 2 Iter., $b=3$ / $s=2$           | 41/50       | 0,447      | 39,4 | 1,158        | 1,357 |
| an, 4 Iter., $b=3$ / $s=2$           | 19/50       | 0,699      | 39,7 | 1,173        | 1,607 |
| an, 2 Iter., $b=2$ / $s=1$           | <b>3/50</b> | 0,875      | 35,3 | <b>1,636</b> | 2,299 |

Tabelle 10: Implementierung - Regelsweep der zellulären Glättungsstufe auf Testset A, eigene Darstellung.

#### 4.1.4 Exit-Platzierung und garantierte Lösbarkeit

**Warum das überhaupt eine Rolle spielt.** Die zentrale Messgröße dieser Arbeit ist die Erfolgsquote: der Anteil der Welten, in denen der Agent den Exit erreicht. Diese Zahl ist nur dann interpretierbar, wenn jede gescheiterte Episode auch tatsächlich am Agenten lag. Erzeugt der Generator Welten, in denen der Exit hinter einer geschlossenen Wandbarriere liegt, mischt sich in jede Erfolgsquote ein unbekannter Anteil unlösbarer Karten. Ein Modell mit 65 % wäre dann von einem Modell mit 80 % nicht mehr zu unterscheiden, wenn 15 % der Welten von vornherein niemand lösen kann. Der Generator muss also *jede* Welt lösbar erzeugen, und zwar nicht „meistens“, sondern immer.

**Der übliche Weg und warum wir ihn nicht gehen.** Naheliegend wäre Rejection Sampling: Welt erzeugen, Exit zufällig setzen, Erreichbarkeit per Flood-Fill prüfen, bei Misserfolg neu würfeln. Das ist teuer, weil der Verwerfungsanteil mit der Wanddichte steigt, und es hat keine obere Laufzeitschranke. Die verbreitete Alternative, das nachträgliche Freiräumen eines Korridors zwischen Spawn und Exit (`forceGuaranteedPath`), ist noch schlechter: Sie hinterlässt einen geraden, ein Tile breiten Gang durch die Welt. Ein Agent kann lernen, genau diese Struktur zu erkennen und ihr zu folgen, statt zu navigieren. Beide Verfahren sind im Generator implementiert, beide sind in den berichteten Läufen abgeschaltet (Tabelle 9).

**Stattdessen: den Exit aus dem Erreichbarkeitsbaum ziehen.** Statt einen Exit zu setzen und danach zu prüfen, kehrt der Generator die Reihenfolge um. Er lässt eine Breitensuche vom Spawn aus über *ausschließlich begehbare* Tiles laufen und wählt den Exit erst danach, aus genau der Menge, die diese Suche erreicht hat. Damit ist Erreichbarkeit keine Eigenschaft, die man prüfen müsste, sondern eine Eigenschaft der Konstruktion: Ein Tile, das die Breitensuche nie besucht hat, kann gar nicht erst Kandidat werden. Der Ablauf in `World::chooseExitPoint()` umfasst vier Schritte.

1. **Spawn freiräumen.** Um den festen Spawn wird ein  $5 \times 5$ -Feld (Radius 2) auf *begehrbar* gesetzt, damit der Agent nicht in einem Hindernis startet.

2. **Breitensuche vom Spawn.** Eine Vier-Nachbarschafts-BFS expandiert über begehbare Tiles, begrenzt auf ein Fenster um den Spawn. Ihre Tiefe ist der *echte Laufweg* in Schritten, nicht der geometrische Abstand. Die Suche bricht bei Tiefe `exitMaxDistance` ab, weil die BFS-Tiefe monoton wächst.
3. **Kandidatenmenge bilden.** Jedes besuchte Tile mit einer Tiefe zwischen `exitMinDistance` und `exitMaxDistance` kommt in die Kandidatenliste. Diese Liste steuert zugleich das Curriculum: Ein Distanzfenster von 5 bis 12 erzeugt kurze Welten, eines von 35 bis 45 die Welten des Evaluationsprotokolls.
4. **Kandidat wählen.** Ein aus dem Seed gemischter Index bestimmt den Startpunkt in der Liste. Von dort prüft der Generator bis zu 24 Kandidaten gegen die Distanzbedingung des nächsten Absatzes und nimmt den ersten, der besteht.

**Die Falle: die Freiräumung erzeugt eine Abkürzung.** Nach der Wahl räumt `World::reset()` auch um den Exit ein Feld frei, hier  $3 \times 3$  (Radius 1), damit das Ziel nicht in einer Wand verschwindet. Genau das kann die eben gemessene Distanz wieder kaputt machen. Öffnet die Freiräumung eine Wand, die vorher einen Umweg erzwungen hat, entsteht eine Abkürzung, und der reale Laufweg fällt unter `exitMinDistance`. Bei einem Fenster von 35 bis 45 wäre der Agent dann in einer Welt, die nominell zur Phase 3 gehört, tatsächlich aber deutlich leichter ist. Für ein distanzgesteuertes Curriculum ist das kein Schönheitsfehler, sondern ein verfälschter Schwierigkeitsgrad.

Die Lösung ist eine zweite Breitensuche, die die Freiräumung vorwegnimmt. Listing 5 zeigt den Kern: Tiles im Freiräumungsradius des Kandidaten gelten in dieser Suche bereits als begehbar, obwohl sie es noch nicht sind. Gemessen wird also der Laufweg der Welt, die *nach* der Freiräumung existiert. Nur wenn dieser Weg noch mindestens `exitMinDistance` beträgt, wird der Kandidat angenommen.

```

1 // BFS vom Spawn zum Kandidaten. Tiles im Freiraemungsradius zaehlen
2 // als begehbar, weil World::reset() sie ohnehin oeffnen wird:
3 const bool virtuallyCleared =
4     std::abs(nx - target.x) <= cfg.exitClearRadius &&
5     std::abs(ny - target.y) <= cfg.exitClearRadius;
6 if(!virtuallyCleared && !isPassable(nx, ny)) { continue; }
7
8 // ... Kandidat nur annehmen, wenn der Weg auch danach lang genug bleibt:
9 for(std::size_t k = 0; k < maxTries; ++k) {
10     const Vec2i cand = candidates[(start + k) % n];
11     if(clearedDistance(cand) >= minDist) { return cand; }
12 }
13 return candidates[start]; // Fallback: erreichbar, Distanz ggf. kuerzer

```

Listing 5: Kandidatenprüfung bei der Exit-Platzierung (`src/core/world.cpp`, gekürzt). Die zweite Breitensuche behandelt Tiles im Freiräumungsradius des Kandidaten als begehbar und misst damit den Laufweg der Welt *nach* der Freiräumung. Ohne diese Vorwegnahme kann die Freiräumung eine Abkürzung öffnen und den Laufweg nachträglich unter `exitMinDistance` drücken.

**Was garantiert ist und was nicht.** Die beiden Zusagen sind unterschiedlich stark, und die Trennung ist wichtig:

**Erreichbarkeit: hart garantiert.** Jeder Kandidat stammt aus dem Erreichbarkeitsbaum der ersten Breitensuche. Ein unlösbarer Exit ist damit nicht unwahrscheinlich, sondern konstruktiv ausgeschlossen. Das gilt auch für den Fallback in der letzten Zeile von Listing 5.

**Distanz: garantiert bis auf einen Fallback.** Besteht keiner der 24 geprüften Kandidaten die Distanzbedingung, nimmt der Generator den zuerst gezogenen. Die Welt bleibt lösbar, ihr Laufweg kann aber unter `exitMinDistance` liegen. Dass dieser Zweig in der berichteten Konfiguration nicht greift, ist gemessen und nicht angenommen: Über 150 geprüfte Seeds lag der reale Laufweg ausnahmslos im Fenster  $[35, 45]$  (Minimum 35, Mittel 39,7 bis 40,1, Maximum 45), und in Phase 1 ebenso im Fenster  $[5, 12]$ . Bei deutlich höherer Waddichte oder einem engeren Distanzfenster wäre die Annahme neu zu prüfen.

**Belege.** Die Konstruktion wurde nicht nur behauptet, sondern nachgemessen. Über 3 150 geprüfte Seeds (alle Eval-Sets und die Trainingsbereiche) sowie eine erneute Stichprobe am 06.07.2026 (150 von 150) ergaben durchgängig **100 % Lösbarkeit**. Ein zweiter, schärferer Test prüft nicht nur die Weltgenerierung, sondern die gesamte Kette: Ein BFS-Orakel-Agent, der den kürzesten Weg direkt aus dem Distanzfeld des Kerns abliest, erreicht den Exit auf allen Test-Seeds in *exakt* der BFS-Distanz. Dass die tatsächlich gelaufene Schrittzahl mit der berechneten Distanz übereinstimmt, schließt neben unlösbaren Welten auch stille Fehler in Bewegungsmechanik, Kollisionsprüfung und Distanzfeld aus. Der gemessene Umwegfaktor von 1,10 (Abschnitt 4.1.3) stammt aus derselben Rechnung.

Damit ist eine methodische Frage entschieden, die im Evaluationsteil sonst offen bliebe: Wenn ein Agent in einer Welt scheitert, liegt es nie an der Welt.

#### 4.1.5 Grafischer Client und Zuschaumodus

Der raylib-Client (ca. 4500 LOC) macht Stoneforge zu einem tatsächlich spielbaren 2D-Spiel: Ein Mensch steuert dieselbe Simulation, die auch das Training ansteuert, in Echtzeit über Tastatur und Maus. Jede Episode lässt sich damit nachspielen. Abbildung 11 zeigt die Oberfläche aus Sicht eines menschlichen Spielers.



Abbildung 11: Benutzeroberfläche des spielbaren C++-Clients (Stoneforge 2D, echter Screenshot): Gezeigt ist die Spielumgebung aus Sichtweise eines menschlichen Spielers mit vollem Sichtbereich, dem Status-Header (Lebenspunkte, Energie, Schritte) und der Spielfigur im Zentrum.

Neben dem Spieler-Modus besitzt der Client einen KI-Zuschaumodus: `demo_agent.py` lädt einen trainierten Checkpoint und reicht dessen Aktionen über die Standardeingabe an den Client weiter, der dabei denselben Seed wie die Trainings-Umgebung verwendet und so dieselbe Welt erzeugt. Ein Mensch kann dem Agenten auf diese Weise beim Navigieren



zusehen. Eine Launcher-GUI (Tkinter) bündelt Training, Evaluation und Abspielen zu einer gemeinsamen Oberfläche.

## 4.2 Implementierung der Lernumgebung (Florian)

Der vorige Abschnitt endet mit einem lauffähigen Spiel. Dieser Abschnitt macht daraus eine Lernumgebung, und zwar ohne die Simulation dafür zu verändern: Der C++-Kern aus Abschnitt 4.1.1 bleibt derselbe, ein Mensch spielt weiterhin dieselbe Welt wie der Agent. Hinzu kommen eine Übersetzungsschicht, die das Spiel als Markov-Entscheidungsproblem darstellt, ein Reward-Design und die Trainings- und Monitoring-Infrastruktur.

### 4.2.1 Versionsverlauf

Umgebung und Trainingskonfiguration durchliefen mehrere Überarbeitungen. Tabelle 11 fasst die Versionen zusammen, auf die im weiteren Verlauf dieses Kapitels sowie in Methodik und Evaluation Bezug genommen wird; die vollständige Änderungshistorie liegt im Experiment-Changelog des Repositories.

| Version                  | Wesentliche Änderungen                                                                                                                                                                                                                                | Ergebnis (Testset A)                  |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|
| vor v11 (bis 06.07.2026) | BFS-Orakel-Feature in der Observation, Exit-Platzierung nach Luftlinie, Straf-Stacking aus Wand- und Loop-Penalty                                                                                                                                     | 98 % stoch. / 2 % det. unter einer    |
| v11 (06.07.2026)         | Tote Features entfernt (231 → 229 Dimensionen), Straf-Stacking entschärft, Exit-Platzierung nach BFS-Laufweg statt Luftlinie, prozess-globales Konfigurationsleck behoben, Mining/Bauen/Kampf aus dem RL-Pfad entfernt, Entropie-Annealing korrigiert | neue Baseline für alle folgenden M    |
| v12 (ab 07.07.2026)      | <code>batch_size</code> von 64 auf 8 korrigiert (stabilisiert den LSTM-Critic), Eval-Episoden-Cap durchgängig auf 4 000 vereinheitlicht, finale Konfiguration über sieben unabhängige Läufe                                                           | <b>65,7 % / 66,9 %</b> (A/B, stochast |

Tabelle 11: Implementierung der Lernumgebung - Versionsverlauf von Umgebung und Training, eigene Darstellung.

### 4.2.2 Was der Agent davon sieht: POMDP-Charakter

Die Welt ist damit vollständig beschrieben: Sie ist reproduzierbar, lösbar und in ihrer Zielentfernung gesteuert. Für den Agenten ist von alledem nichts unmittelbar zugänglich. Er sieht nur einen lokalen  $15 \times 15$ -Ausschnitt der Welt sowie einen *Luftlinien*-Kompass zum Exit, nicht den Weg. Wände, Sackgassen und Umwege außerhalb des Sichtfensters sind unbekannt: Die Aufgabe ist ein *Partially Observable Markov Decision Process* (POMDP, Abschnitt 2.2.2). Beide Perspektiven stellt Abbildung 4 auf Seite 10 gegenüber: links die Welt, wie sie tatsächlich aussieht, rechts das, was der Agent davon sieht. Für eine gedächtnislose Politik sind dabei verschiedene Weltpositionen ununterscheidbar (Abschnitt 5.2.1).

### 4.2.3 Umgebung, Beobachtung und Aktionen

Damit ein Lernverfahren die Simulation ansteuern kann, muss sie sich als *Markov-Entscheidungsproblem* präsentieren: ein Zustand, den der Agent beobachtet, eine Menge von Aktionen, aus denen er wählt, und ein Reward pro Schritt. Stoneforge ist von Haus

aus kein solches Problem, sondern ein Spiel mit Kamera, Inventar und Echtzeitschleife. `StoneforgeWorldEnv` (Python) übersetzt zwischen beiden Welten und kapselt den C++-Kern hinter der Gymnasium-API. Was der Agent dabei zu sehen bekommt, listet Tabelle 12.

| Bereich                 | Inhalt                                          | Dim.       |
|-------------------------|-------------------------------------------------|------------|
| Lokales Grid            | 15×15 Tile-Typen um den Agenten (Sichtradius 7) | 225        |
| HP                      | Lebenspunkte (normiert)                         | 1          |
| Kompass                 | exitDx, exitDy (Luftlinie, normiert)            | 2          |
| Episodenfortschritt     | Schritt / 4000                                  | 1          |
| <b>Gesamt (Env v11)</b> |                                                 | <b>229</b> |

Tabelle 12: Implementierung der Lernumgebung - Aufbau der 229-dimensionalen Observation in Umgebungsversion v11, eigene Darstellung.

Der Aktionsraum ist `Discrete(4)` (hoch, runter, links, rechts). Die im Spiel vorhandenen Aktionen Mining, Bauen und Kampf sind im RL-Binding deaktiviert und werfen bei Aufruf einen Fehler; sie existieren nur im spielbaren Client (Abschnitt 4.1.5). Eine Episode endet bei Erreichen des Exits (Sieg), nach 4000 Schritten (Timeout) oder als Early-Stop-Truncation nach 256 Schritten ohne positiven Reward.

Tabelle 13 führt daneben „Tod“ als terminales Ereignis und einen Schadens-Penalty auf. Beide können im RL-Pfad nicht eintreten: Die einzigen Schadensquellen des Simulationskerns sind Mob-Nähe und Verhungern, und die Umgebung schaltet beides ab (`disable_mobs`, `disable_energy`). Die Lebenspunkte bleiben damit über die gesamte Episode konstant. **Das HP-Merkmal in Tabelle 12 ist folglich ein totes Feature**, so wie es Energie und Inventar vor der v11-Revision waren (Tabelle 11); es wurde dort übersehen und ist eine der 229 Dimensionen ohne Informationsgehalt. Der Schadens-Penalty ist entsprechend inert. Beides ist folgenlos für die berichteten Ergebnisse, aber bei einer Neufassung der Beobachtung zu streichen.

#### 4.2.4 Die Schnittstelle: was bei `reset()` und `step()` tatsächlich passiert

Die Aufgabenteilung zwischen den beiden Sprachen ist eng geschnitten. Der C++-Kern besitzt den Weltzustand und rechnet alles, was pro Schritt anfällt: Weltgenerierung, Bewegung samt Kollision, das BFS-Distanzfeld und die Reward-Terme aus Tabelle 13. Die Python-Seite besitzt keinen Weltzustand. Sie führt Buch über die Episode, formt die Beobachtung und implementiert genau jene Regeln, die sich ohne Neuübersetzung ändern lassen sollten. Das `pybind11`-Binding dazwischen ist bewusst dünn: Es exportiert im Wesentlichen `reset(seed)`, `step(action)` und eine Handvoll lesender Abfragen wie `current_bfs_distance_to_exit()` oder `player_pos()`. Es enthält keine Spiellogik. Wer den Reward ändern will, ändert C++ und übersetzt neu; wer das Curriculum ändert, ändert Python.

**Ein `reset()` in vier Schritten.** Der Aufruf legt eine neue Episode an, und die Reihenfolge ist nicht beliebig:

1. **Seed bestimmen.** Übergibt der Aufrufer einen Seed, wird dieser verwendet. Das ist der Fall im Evaluationsprotokoll, wo die Testsets aus festen Seeds bestehen. Im Training zieht die Umgebung stattdessen selbst einen Zufallsseed, sodass jede Episode eine neue Welt sieht.
2. **Episodenbuchführung zurücksetzen.** Schrittzähler, Besuchszähler je Tile, Zähler für Schritte ohne positiven Reward und der Aktionspuffer werden geleert. Diese Zähler tragen den Revisit-Penalty und den Early Stop, existieren also nur auf der Python-Seite.

3. **Weltgenerator-Konfiguration neu setzen.** Erst danach folgt der eigentliche Reset des Kerns. Der Grund steht im nächsten Absatz.
4. **Kern zurücksetzen und Beobachtung bauen.** Der Kern erzeugt die Welt zum Seed, platziert den Exit (Abschnitt 4.1.4) und liefert einen rohen Zustandsvektor. Die Python-Seite normiert ihn und schneidet die toten Legacy-Felder heraus.

**Ein Detail, das lange Zeit die Messungen verfälscht hat.** Die Weltgenerator-Konfiguration liegt im C++-Kern als *prozessglobale* Struktur. Das ist für ein Spiel unproblematisch, in dem genau eine Welt existiert. Im Training existieren aber 16 parallele Umgebungen, und der Evaluations-Callback erzeugt zusätzlich eine eigene. Setzt diese Evaluations-Umgebung ihr Distanzfenster auf 35 bis 45, überschreibt sie damit das Fenster *aller* Trainingsumgebungen, obwohl sie fachlich nichts miteinander zu tun haben. In Phase 3 verschob der Evaluations-Callback auf diese Weise die Trainingsverteilung nach der ersten Evaluation still von 25 bis 45 auf 35 bis 45. Das Curriculum trainierte also nicht mehr auf dem Bereich, der in der Konfiguration stand.

Die Lösung ist die dritte Zeile der Aufzählung oben: Jede Umgebung stempelt ihre Konfiguration vor jedem einzelnen `reset()` neu, statt sie einmalig im Konstruktor zu setzen. Nachweisen lässt sich der Fehler mit einem BFS-Orakel: Vor dem Fix brauchte es auf den Test-Seeds durchschnittlich 8,3 Schritte, nachdem eine zweite Umgebung mit dem Fenster 5 bis 12 erzeugt worden war; danach wieder die korrekten 40,2 (Tabelle 11). Der Fehler war im Python-Code nicht zu sehen, denn dort stand die richtige Konfiguration. Sichtbar wurde er erst an Zahlen, die sich nicht erklären ließen.

**Ein `step()` in zwei Hälften.** Der Aufruf reicht die Aktion an den Kern durch, der Bewegung, Kollision und Reward rechnet und den neuen Zustand zurückgibt. Danach ergänzt die Python-Seite drei Regeln, die absichtlich nicht in C++ stehen, weil sie zur Trainingsmethodik gehören und nicht zum Spiel:

- **Revisit-Penalty.** Ein Besuchszähler je Tile bestraft ab dem 26. Besuch desselben Feldes, gedeckelt ab dem 50. Er greift damit nur bei ausgeprägtem Kreiseln (Tabelle 13).
- **Early Stop.** Bleiben 256 Schritte ohne positiven Reward, endet die Episode als Truncation. Ohne diese Regel verbraucht ein feststeckender Agent bis zu 4 000 Schritte pro Episode und der Durchsatz bricht ein.
- **Seed-Pool („Swarm“).** Erfolgreiche Seeds werden gesammelt und in 30 % der Fälle wiederholt.

Zwei Dinge sind an dieser Aufteilung erwähnenswert. Erstens ist der Early Stop eine *Truncation* und kein Terminal: Der Wert des Endzustands wird weiter gebootstrappt, die Episode gilt nicht als Misserfolg im Sinne des Zielkriteriums. Zweitens ist die Trennung nicht kosmetisch. Jede Änderung an den drei Python-Regeln wirkt sofort, während jede Änderung am Reward einen Rebuild des Kerns erfordert. Über die Laufzeit des Projekts hat das die Iterationsgeschwindigkeit spürbar bestimmt.

#### 4.2.5 Reward-Design

Das Reward-Design folgt dem Prinzip: *sparse Primärsignal + theoretisch sauberes dichtes Shaping*. Kernstück ist Potential-Based Reward Shaping (PBRS) auf der BFS-Pfaddistanz zum Exit:

$$F(s, s') = \gamma \Phi(s') - \Phi(s), \quad \Phi(s) = -\frac{d_{\text{BFS}}(s)}{128}, \quad (7)$$

mit  $\gamma = 0.999$  (identisch zum RL-Discount) und Gewicht  $\beta = 2.5$ . Ein Tile echter Fortschritt liefert damit rund  $+0.02$ ; abzüglich des Schritt-Malus von  $-0.01$  (Tabelle 13) verbleiben netto etwa  $+0.01$ . Listing 6 zeigt die Umsetzung im Kern.

```

1 constexpr float PBRs_MAX_DIST = 128.0F; // Normierung der BFS-Distanz
2 constexpr float PBRs_GAMMA    = 0.999F; // identisch zum RL-Discount
3 constexpr float PBRs_BETA     = 2.5F;   // +0,01 netto pro Tile bei d=40
4
5 const float phi_prev = -static_cast<float>(previousDistance) /
    PBRs_MAX_DIST;
6 const float phi_cur  = -static_cast<float>(currentDistance) /
    PBRs_MAX_DIST;
7 const float shaping  = PBRs_GAMMA * phi_cur - phi_prev;
8 reward += PBRs_BETA * shaping;

```

Listing 6: Potential-Based Reward Shaping auf der BFS-Distanz (`Simulation::computeReward()`, `src/core/simulation.cpp`). Der Shaping-Discount ist bewusst identisch zum Discount des Lernverfahrens; nur unter dieser Bedingung ist  $F$  policy-invariant [31].

Beide Konstanten sind aus dieser Netto-Vorgabe rückwärts bestimmt. Der Nenner 128 normiert die BFS-Distanz auf die Größenordnung der übrigen Reward-Terme und entspricht zugleich der Skalierung des BFS-Puffers im C++-Kern, sodass  $\Phi$  auch im ungünstigsten Fall betragsmäßig klein gegenüber dem Terminal-Reward bleibt. Das Gewicht  $\beta = 2.5$  ist so gewählt, dass sich bei der mittleren Eval-Distanz  $d \approx 40$  genau die genannten  $+0.01$  netto pro Fortschritts-Tile ergeben. Ein zuvor verwendetes  $\beta = 5.0$  wurde halbiert, nachdem ein zu klein dimensionierter BFS-Puffer in Kombination mit dem Manhattan-Fallback zeitweise Beträge um  $+2.5$  pro Schritt erzeugt und damit ein falsches Lernziel gesetzt hatte. PBRs ist beweisbar *policy-invariant*; es beschleunigt die Konvergenz, ohne die optimale Politik zu verändern [31]. Entscheidend ist, dass das Potential die *echte* Pfaddistanz (BFS durch Wände gerechnet) verwendet, nicht die Luftlinie: Ein Luftlinien-Potential würde den Agenten systematisch in Sackgassen führen. Abbildung 12 zeigt den Unterschied an einem minimalen Beispiel.

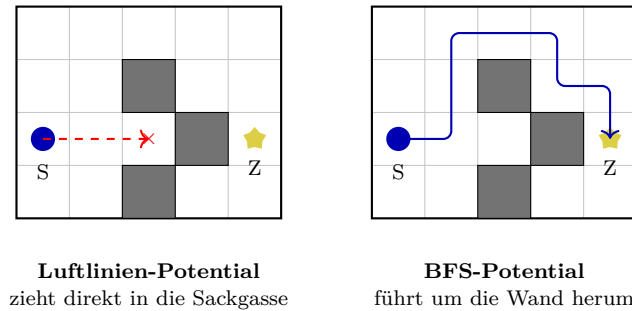


Abbildung 12: Implementierung der Lernumgebung - Schematische Sackgasse, in der ein Luftlinien-Potential den Agenten hineinführt (links) und das verwendete BFS-Pfaddistanz-Potential korrekt um die Wand herumführt (rechts), eigene Darstellung.

Diese Garantie gilt allerdings nur für den PBRs-Term selbst. Die übrigen Komponenten in Tabelle 13 (Explorations-Bonus, Schritt-, Loop-, Idle-, Schadens- und Revisit-Penalty) sind gewöhnliche Reward-Terme und verändern die optimale Politik prinzipiell mit. Der frühere eskalierende Wand-Penalty ( $-0.05/-0.25$ ) wurde aus genau diesem Grund entfernt: Die Summe vieler kleiner Strafen machte Erkunden in engen Korridoren unwirtschaftlicher als Stillstand. Der Revisit-Penalty ist der einzige verbliebene Term dieser Art; er greift erst ab dem 26. Besuch *desselben* Tiles und damit ausschließlich bei ausgeprägtem Kreiseln, wirkt in der Sache aber in dieselbe Richtung und ist bei einer Neufassung des Reward-Designs

(Abschnitt 6) mit zu prüfen.

| Komponente                                  | Wert               |
|---------------------------------------------|--------------------|
| Exit erreicht (terminal)                    | +100               |
| Timeout / Tod (terminal)                    | -10                |
| PBRs (BFS-Distanz, pro Schritt)             | $\approx \pm 0.02$ |
| Explorations-Bonus (neue Tile)              | +0.02              |
| Schritt-Penalty                             | -0.01              |
| 2-Schritt-Loop-Penalty                      | -0.05              |
| Idle-Penalty (Aktion ohne Positionswechsel) | -0.04              |
| Schadens-Penalty (pro verlorenem HP)        | -0.5               |
| Revisit-Penalty (ab 26. Besuch eines Tiles) | -0.03 bis -0.06    |

Tabelle 13: Implementierung der Lernumgebung - Vollständige Reward-Komponenten der Umgebungsversion v11, eigene Darstellung.

#### 4.2.6 Trainings-Pipeline

`train_curriculum.py` orchestriert die vier Phasen aus Tabelle 3. Ein Lauf ist damit kein einzelnes Training, sondern eine Kette aus vier Trainings, die einander das Modell weiterreichen. Vier Bausteine tragen diese Kette.

**Parallele Umgebungen.** Je Lauf arbeiten 16 Umgebungsinstanzen in einem `DummyVecEnv`. Sie laufen im selben Prozess, nacheinander pro Schritt, und bündeln ihre Beobachtungen zu einem Batch. Der Gewinn liegt nicht in echter Parallelität, sondern darin, dass ein Rollout Erfahrung aus 16 verschiedenen Welten gleichzeitig enthält, statt aus einer.

**Evaluation und Gating.** Alle 25 000 Schritte misst ein Callback das aktuelle Modell auf 50 Validierungs-Seeds, in beiden Betriebsarten. Diese Messung erfüllt zwei Aufgaben zugleich. Sie entscheidet erstens, wann eine Phase endet: Erreicht die maßgebliche Metrik die Ziel-Erfolgsquote in zwei aufeinanderfolgenden Evaluationen, schaltet das Curriculum weiter. Die Doppelbedingung filtert Ausreißer, denn ein einzelner guter Messpunkt ist bei dieser Lern-dynamik kein Beleg (Abschnitt 3.2.3). Zweitens wählt sie das beste Modell der Phase aus, und genau dieses wird beim Phasenwechsel geladen. Welche Metrik dabei zählt, wechselt: Die Phasen 1 und 2 gaten auf der stochastischen Erfolgsquote, die Phasen 3 und 4 auf der deterministischen.

**Absenkung der Exploration.** Der Entropie-Koeffizient hält die Politik früh breit gestreut und muss später aus dem Weg. In Phase 3 senkt ein Callback ihn linear von 0,05 auf 0,001 über 500 000 Schritte, Phase 4 trainiert mit 0,0001. Diese letzte Phase existiert nur, um die deterministische Politik nachzuziehen (Abschnitt 5.2.4).

**Seed-Pool.** Ein optionaler Pool („Swarm“) sammelt bereits gelöste Seeds und startet 30 % der Episoden auf einem davon. Bei jedem Phasenwechsel wird er geleert, damit gelöste Welten der leichten Phase nicht die schwere Phase verwässern.

Jeder Lauf legt pro Phase Modell-Checkpoints sowie `eval_history.json` und `config.json` ab; alle Experimente sind zusätzlich im `CHANGELOG.md` des Repositories dokumentiert.

#### 4.2.7 Monitoring und Werkzeuge

Trainingsmetriken werden nach TensorBoard geschrieben. Neben der bereits eingeführten `explained_variance` (Güte der Wertfunktion, Abschnitt 2) und der Entropie der Politik ist dabei `approx_kl` von Bedeutung: ein Schätzwert dafür, wie stark sich die Politik durch ein Update verändert hat. Werte nahe null zeigen an, dass das Training faktisch stillsteht; sehr große Werte deuten auf zu grobe Schritte hin. Die Größe dient in dieser Arbeit als schneller Test, ob ein Lauf überhaupt lernt. Zusätzlich streamt ein WebSocket-Server den Trainingszustand in eine Browser-Ansicht: Erfolgsquoten-Kurven (deterministisch und stochastisch), Minimaps der 16 parallelen Agenten und eine Explorations-Heatmap.

Abbildung 13 fasst die Implementierung dieses Kapitels zusammen: Ein einziger C++-Simulationskern wird auf zwei unabhängigen Wegen angesteuert, einmal über das Python-Binding zum Training, einmal direkt vom grafischen Client. Beide Pfade erzeugen bei gleichem Seed dieselbe Welt, was den KI-Zuschaumodus (Abschnitt 4.1.5) überhaupt erst sinnvoll macht.

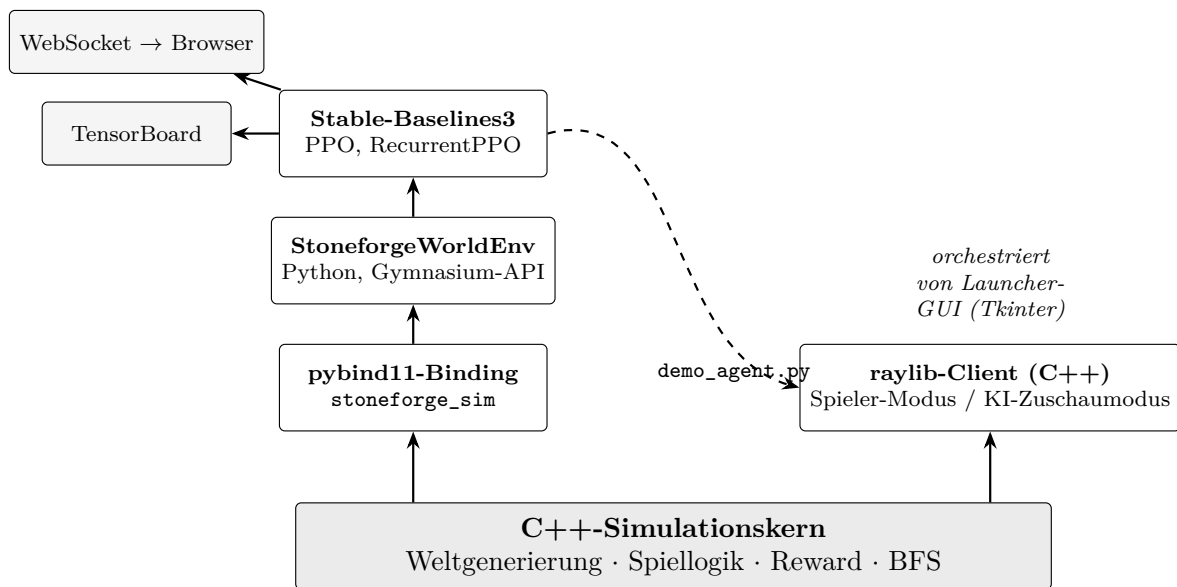


Abbildung 13: Implementierung - Gemeinsamer C++-Simulationskern mit den beiden Ansteuerungswegen Training und Client, eigene Darstellung.

## 5 Evaluation (gemeinsam)

Das Evaluationsprotokoll (Seed-Trennung, Metrik, Betriebsarten) ist in Abschnitt 3.2 beschrieben. Bewertet werden beide Teile des Projekts, und zwar in der Reihenfolge, in der sie aufeinander aufbauen: zuerst die Plattform gegen die Anforderungen Z1 (Abschnitt 5.1), dann der Agent gegen die Schwellen Z2 (Abschnitt 5.2).

### 5.1 Bewertung der Plattform (Z1, Laurin)

Die Anforderungen aus Abschnitt 1.4 sind funktionaler Art; sie werden hier zusammengeführt und gegen die jeweils erhobene Evidenz gestellt.

| Anforderung              | Nachweis                                                                                                                                                                                                                                                                                                                                                                       | Status  |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| Determinismus            | Weltgenerierung und Simulation hängen ausschließlich von Seed und Koordinaten ab. Die chunkweise Erzeugung ist durch die Halo-Berechnung der Glättungsstufe reihenfolgeunabhängig; ein prozessglobales Konfigurationsleck, das dies zeitweise verletzte, wurde gefunden und behoben (Tabelle 11). Zwei Trainingspfade stimmen über 8192 Schritte <i>bit-identisch</i> überein. | erfüllt |
| Garantierte Lösbarkeit   | Exit-Platzierung ausschließlich über BFS-erreichbare Zellen. 3150 geprüfte Seeds sowie eine Nachmessung über 150 Seeds ergaben 100 % Lösbarkeit; ein BFS-Oracle-Agent erreicht das Ziel auf allen Test-Seeds in exakt der BFS-Distanz (Abschnitt 4.1.4).                                                                                                                       | erfüllt |
| Steuerbare Schwierigkeit | Nach der Umstellung von Luftlinie auf BFS-Laufweg liegen alle 150 Eval-Seeds im vorgegebenen Distanzband; zuvor streute eine Vorgabe von „35–45“ real auf 42–75 Tiles (Tabelle 11).                                                                                                                                                                                            | erfüllt |
| RL-Tauglichkeit          | Gymnasium-Schnittstelle, 16 parallele Umgebungen, 88–190 Schritte/s auf einem Notebook; ein vollständiger Curriculum-Lauf über 2,2 Mio. Schritte dauert rund 8 h (Abschnitt 4.1.1).                                                                                                                                                                                            | erfüllt |
| Spielbarkeit             | Grafischer raylib-Client mit KI-Zuschaumodus; Client und Trainingsumgebung erzeugen aus demselben Seed dieselbe Welt (Abschnitt 4.1.1).                                                                                                                                                                                                                                        | erfüllt |

Tabelle 14: Evaluation - Bewertung der Plattform gegen die Anforderungen Z1, eigene Darstellung.

**Was damit belegt ist: und was nicht.** Alle fünf Anforderungen sind erfüllt; die Plattform leistet, wofür sie gebaut wurde. Drei der fünf Nachweise entstanden allerdings erst als Korrektur eines zuvor unbemerkten Fehlers: Der Determinismus war durch das prozessglobale Konfigurationsleck verletzt, die Schwierigkeitssteuerung durch das Luftlinienmaß, und die Lösbarkeitsgarantie stand vor der BFS-basierten Exit-Platzierung auf dem schwächeren Fundament eines gecarvten Korridors. Keiner dieser Fehler war am Verhalten des Agenten unmittelbar ablesbar, sie wurden sichtbar, weil das RL-Experiment Zahlen lieferte, die sich

nicht erklären ließen. Das ist der erste von zwei Punkten, an denen die beiden Projektteile ineinandergreifen.

Der zweite folgt in Abschnitt 5.2.3 und wiegt schwerer: Z1 verlangt lösbare und in ihrer Entfernung steuerbare Welten, nicht aber *hinreichend schwierige*. Genau diese fehlende Anforderung führt dazu, dass die erzeugten Welten die Fähigkeit, um die es in F2 geht, gar nicht einfordern. Die Plattform erfüllt damit ihre Spezifikation vollständig, und die Spezifikation erweist sich als unvollständig.

## 5.2 Bewertung des Agenten (Z2, Florian)

Der zweite Teil der Evaluation bewertet den Agenten gegen die Schwellen Z2. Berichtet werden die Ergebnisse in der Reihenfolge, in der sie sich gegenseitig einschränken: zuerst die Gegenüberstellung mit und ohne lokales BFS-Orakel, dann das Endergebnis über sieben Läufe, dann der Vergleich gegen ungelernete Referenzpolitiken, der die Aussagekraft der vorherigen Zahlen relativiert, zuletzt der Det/Stoch-Gap und die Versuche zu seiner Schließung.

### 5.2.1 Zentrale Gegenüberstellung: Gedächtnis statt lokalem Orakel

| Bedingung               | Arch.       | BFS in Obs | Curric. | Obs-Dim. | SR stoch.   | SR det.     |
|-------------------------|-------------|------------|---------|----------|-------------|-------------|
| A (ppo_phase4)          | MLP         | ja         | ja      | 236      | 32 %        | 0 %         |
| B (ppo_no_bfs)          | MLP         | nein       | nein    | 230      | –           | ≈ 0 %       |
| C (ppo_lstm_curriculum) | <b>LSTM</b> | nein       | ja      | 231      | <b>86 %</b> | <b>36 %</b> |

Tabelle 15: Evaluation - Gegenüberstellung dreier Modelle auf Testset A, je ein Einzellauf, Stand Juni 2026, eigene Darstellung.

Die drei Bedingungen sind keine kontrollierte Ablation: A und C unterscheiden sich gleichzeitig in Architektur und Beobachtungsraum, B ist ein bei 720 k von 2 Mio. Schritten abgebrochener Lauf, und alle drei stammen aus der Zeit vor der Umgebungsrevision v11 unter einem inzwischen verworfenen Kurzdistanz-Protokoll (Tabelle 11). Belastbar ist daher nur die schwache Aussage, dass eine rekurrente Politik ohne lokale BFS-Features eine Leistung erreicht, die eine gedächtnislose Politik mit diesen Features unter demselben Protokoll nicht erreicht hat.

Bedingung C erzielte ihre 86 % zudem auf kürzeren Wegen als das finale Modell: Die damalige Exit-Vorgabe „35–45“ war eine Luftlinien-Distanz und entsprach real 42–75 Feldern, gegenüber der heute exakt eingehaltenen BFS-Laufweg-Distanz (Tabelle 11). Die 65,7 % des finalen Modells (Abschnitt 5.2.2) sind damit nicht als Rückschritt zu lesen. Abbildung 14 zeigt den Trainingsverlauf dieses historischen Referenzlaufs.

### 5.2.2 Endergebnis (v12, $n = 7$ )

Die sieben finalen Curriculum-Läufe (alle vier Phasen durchlaufen) ergeben auf dem je Lauf nach Validierungs-SR besten Modell:

Die sieben Einzelläufe streuen auf Testset A zwischen 50 % und 84 %; die Erfolgsschwelle aus Abschnitt 1.4 liegt mitten in dieser Streuung. Das Holdout-Kriterium ist damit erfüllt ( $66,9\% \geq 60\%$ ), das Testset-Kriterium verfehlt ( $65,7\%$  gegenüber  $\geq 70\%$ , Abstand 4,3 Prozentpunkte); deterministisch werden beide Kriterien verfehlt ( $29,1\% / 32,6\%$ , Einordnung in Abschnitt 5.2.4). Ausgewertet wird je Lauf nicht der letzte Checkpoint, sondern das nach Validierungs-Erfolgsquote beste Modell, ausschließlich auf den disjunkten Seeds 6000–6049 bestimmt.

Nur die Phasen 1 und 2 erreichten ihr Ziel in einzelnen Läufen; die Phasen 3 und 4, aus denen das berichtete Modell stammt, erreichten ihr Ziel in *keinem* der sieben Läufe und



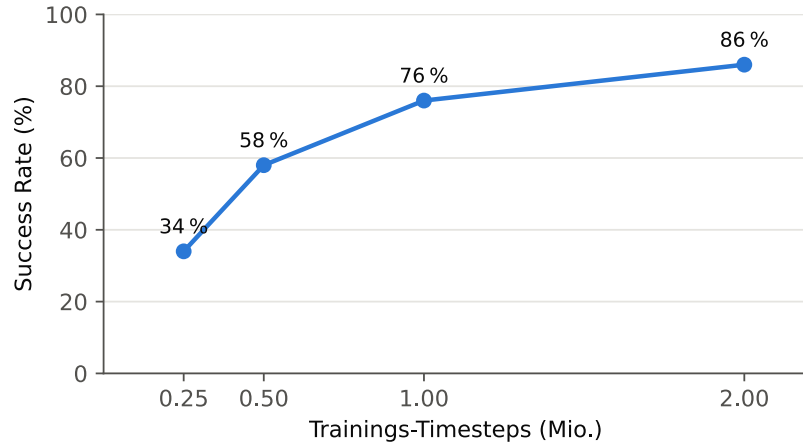


Abbildung 14: Evaluation - Lernkurve des historischen Referenzlaufs (Bedingung C, Umgebung vor der v11-Revision), eigene Darstellung.

| Lauf                               | A stoch.                          | A det.         | B stoch.                          | B det.          |
|------------------------------------|-----------------------------------|----------------|-----------------------------------|-----------------|
| Seed 1                             | 62 %                              | 38 %           | 76 %                              | 46 %            |
| Seed 2                             | 84 %                              | 26 %           | 76 %                              | 44 %            |
| Seed 3                             | 76 %                              | 32 %           | 80 %                              | 36 %            |
| Seed 4                             | 74 %                              | 40 %           | 74 %                              | 38 %            |
| Seed 5                             | 58 %                              | 20 %           | 62 %                              | 34 %            |
| Seed 6                             | 50 %                              | 20 %           | 50 %                              | 14 %            |
| Seed 7                             | 56 %                              | 28 %           | 50 %                              | 16 %            |
| <b>Mittel <math>\pm</math> Std</b> | <b>65,7 <math>\pm</math> 12,4</b> | 29,1 $\pm$ 8,0 | <b>66,9 <math>\pm</math> 12,8</b> | 32,6 $\pm$ 12,7 |
| 95 %-CI                            | [54,2; 77,2]                      | [21,8; 36,5]   | [55,0; 78,7]                      | [20,8; 44,4]    |

Tabelle 16: Evaluation - Endergebnis der sieben v12-Läufe auf Testset A und Holdout B mit Standardabweichung und Konfidenzintervall, eigene Darstellung.

endeten stattdessen ausnahmslos am Schrittbudget (Tabelle 17). Das Curriculum war für das berichtete Modell damit faktisch budget- statt leistungsgesteuert. Abbildung 16 zeigt die zugrunde liegenden Trainingsverläufe: Die hohe Volatilität der Lerndynamik innerhalb der Phasen ist dieselbe Streuungsquelle, die sich in den  $\pm 12$  Punkten des Endergebnisses niederschlägt.

Eine Zwischenauswertung über die ersten drei Läufe hatte mit 73,3 % / 80,0 % (A/B) beide Kriterien noch erfüllt; die gezielte Aufstockung auf sieben Läufe senkte beide Mittelwerte deutlich. Messfehler, Unterschiede im Code-Stand zwischen den beiden Lauf-Chargen und die Laufzeitumgebung wurden als Ursachen geprüft und ausgeschlossen. Es handelt sich um Seed-Varianz zwischen unabhängigen Trainingsläufen, wie sie Henderson et al. [7] für Deep RL beschreiben und weshalb sie Mittelwert und Standardabweichung über mehrere Läufe statt eines Einzellaufs fordern.

### 5.2.3 Ungelernte Referenzpolitiken und die Grenzen der Erfolgsquote

Eine Erfolgsquote ist ohne Untergrenze nicht interpretierbar: Ohne Bezugspunkt bleibt offen, ob 65 % eine gelernte Fähigkeit ausdrücken oder bereits durch Zufall erreichbar sind. Verglichen werden zwei ungelernte Politiken, die *dieselbe* Beobachtung erhalten wie der Agent:

- **Random:** gleichverteilte Bewegung; absolute Untergrenze.

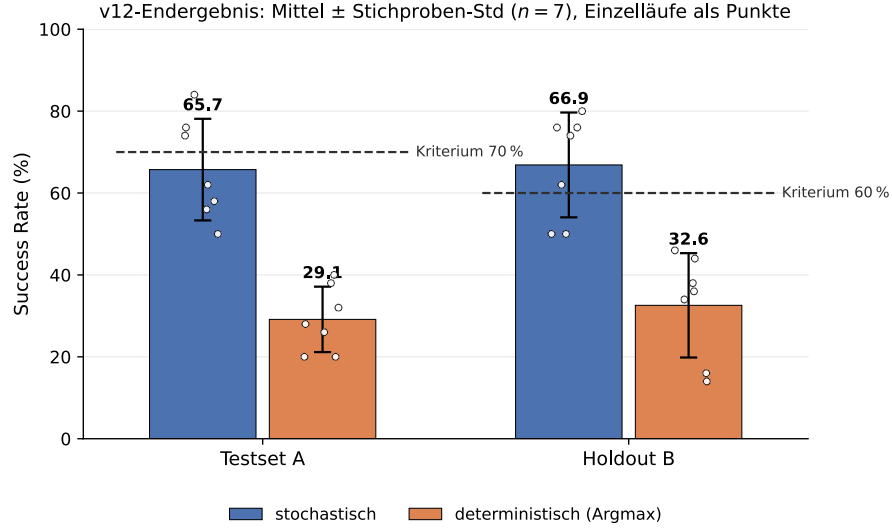


Abbildung 15: Evaluation - Endergebnis aus Tabelle 16 als Balken mit Fehlerbalken, den sieben Einzelläufen und den vorab festgelegten Erfolgsschwellen, eigene Darstellung.

| Phase | Ziel-SR       | Ziel berührt | Phase bestanden |
|-------|---------------|--------------|-----------------|
| 1     | 85 % (stoch.) | 5 / 7        | 1 / 7           |
| 2     | 70 % (stoch.) | 7 / 7        | 3 / 7           |
| 3     | 70 % (det.)   | 0 / 7        | 0 / 7           |
| 4     | 60 % (det.)   | 0 / 7        | 0 / 7           |

Tabelle 17: Evaluation - Gating-Bilanz der sieben v12-Läufe je Curriculum-Phase, eigene Darstellung.

- **Kompass-Zufallslauf:** mit Wahrscheinlichkeit  $\varepsilon$  gleichverteilt zufällig, sonst ein Schritt in die dominante Richtung des Luftlinien-Kompasses ( $exitDx/exitDy$ ). Diese Politik nutzt *zwei* der 229 Merkmale, hat kein Gedächtnis, keine Parameter und wurde nicht trainiert. Berichtet werden zwei Einstellungen, ausschließlich auf den Validierungs-Seeds gewählt: die erfolgsoptimale ( $\varepsilon = 0,9$ ) und die effizienzoptimale ( $\varepsilon = 0,4$ ).

Als zweite Metrik dient die **Pfادهffizienz**  $\eta = d_{\text{BFS}}(s_0) / T_{\text{Erfolg}}$ , gemittelt über die erfolgreichen Episoden: das Verhältnis aus kürzestem Weg und tatsächlich benötigten Schritten.  $\eta = 1$  bedeutet optimale Navigation,  $\eta = 0,05$  einen zwanzigfachen Umweg. Die Metrik ist erforderlich, weil die Erfolgsquote bei einem Episodenlimit von 4000 Schritten und einer mittleren Optimallänge von 40 Schritten kaum diskriminiert. Tabelle 18 stellt beide Metriken für alle Politiken gegenüber.

**Befund.** Der Kompass-Zufallslauf übertrifft den trainierten Agenten auf *beiden* Achsen. Bei vergleichbarer Pfادهffizienz ( $\eta \approx 0,05$ ) erreicht er 92 % statt 65,7%; bei vergleichbarer Erfolgsquote ( $\approx 60\%$ ) ist er mit  $\eta = 0,15$  rund dreimal wegeffizienter. Der Agent liegt zwar deutlich über der Zufallsuntergrenze von 5–10%, erreicht aber an keinem Betriebspunkt die Leistung einer Heuristik aus vier Zeilen Code, die 227 seiner 229 Eingangsmerkmale ignoriert. Der Befund ist robust gegenüber der Wahl des Politik-Zufallsstroms: Ein Wechsel des Zufallsstroms verschiebt die Random-Zeile deutlich (Verdopplung), die Kompass-Zeile dagegen kaum ( $92,0 \rightarrow 89,6$  bei  $\varepsilon = 0,9$ ), und der Abstand zwischen Kompass und Agent bleibt in beiden Fällen groß.

v12: Success Rate auf den Validierungs-Seeds, je Curriculum-Phase

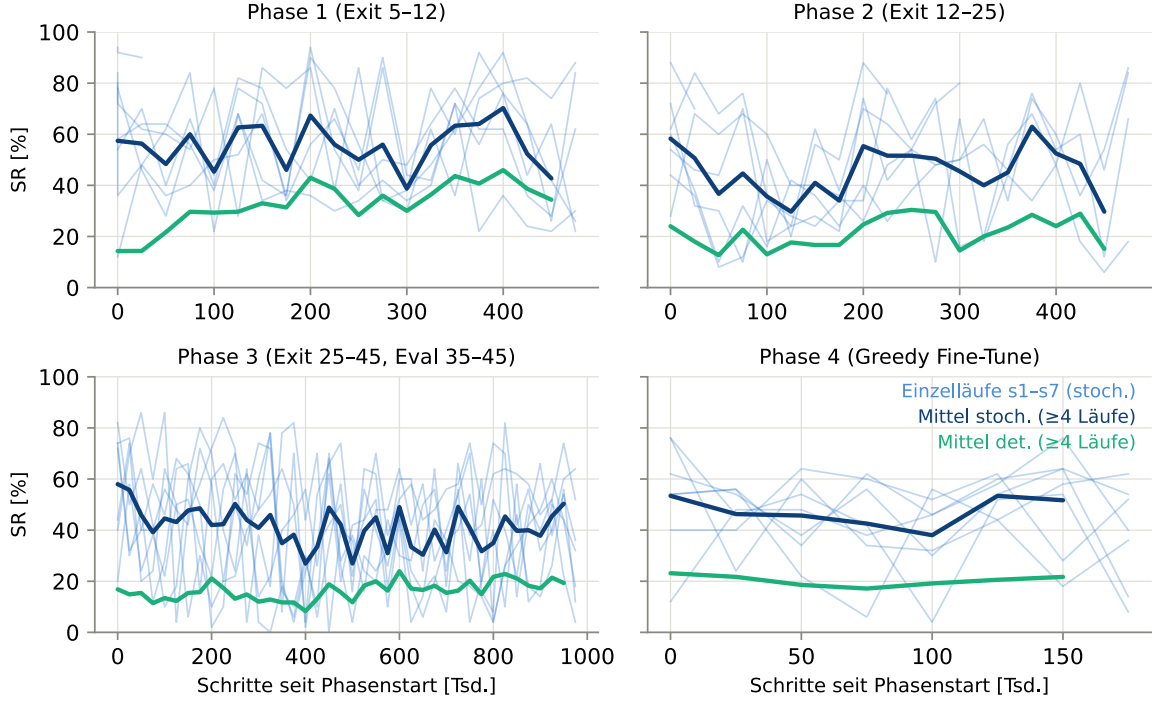


Abbildung 16: Evaluation - Trainingsverläufe der sieben v12-Läufe auf den Validierungs-Seeds, getrennt nach Curriculum-Phase, eigene Darstellung.

| Politik                                  | Testset A                        |              | Holdout B                        |              |
|------------------------------------------|----------------------------------|--------------|----------------------------------|--------------|
|                                          | SR                               | $\eta$       | SR                               | $\eta$       |
| Random (gleichverteilt)                  | $5,2 \pm 2,7$                    | 0,021        | $10,4 \pm 1,7$                   | 0,025        |
| Kompass-Zufallslauf, $\varepsilon = 0,4$ | $60,4 \pm 4,8$                   | <b>0,153</b> | $61,6 \pm 5,0$                   | <b>0,161</b> |
| Kompass-Zufallslauf, $\varepsilon = 0,9$ | <b><math>92,0 \pm 5,1</math></b> | 0,047        | <b><math>91,6 \pm 4,3</math></b> | 0,048        |
| RecurrentPPO v12 ( $n = 7$ , stoch.)     | $65,7 \pm 12,4$                  | 0,049        | $66,9 \pm 12,8$                  | 0,057        |

Tabelle 18: Evaluation - Erfolgsquote und Pfadeffizienz ungelerner Referenzpolitiken gegenüber dem trainierten Agenten unter identischem Protokoll, eigene Darstellung.

**Kein Episodenbudget kehrt das Bild um.** Naheliegender wäre der Einwand, das Problem liege allein am großzügigen Limit von 4000 Schritten. Da wer eine Welt bei Schritt  $T$  löst, sie auch unter jedem Budget  $\geq T$  löst, folgt aus den protokollierten Erfolgsschritten die vollständige Kurve *Erfolgsquote über Budget*:

Bei *jedem* Budget erreicht oder übertrifft mindestens eine der beiden ungelerten Politiken alle drei trainierten Läufe. Ein engeres Episodenlimit ist als Korrektur damit nicht ausreichend: Drei Eigenschaften der Aufgabe wirken zusammen, das großzügige Episodenlimit, der dauerhaft verfügbare Luftlinien-Kompass und die hinreichend offenen Welten, unter denen Gedächtnis für die Erfolgsquote schlicht nicht erforderlich ist.

**Das Reward-Design verstärkt diesen Effekt.** Tabelle 20 bilanziert die dichten Reward-Terme über eine typische erfolgreiche Episode (Läufe 1–3, Testset A).

Entscheidend ist die Asymmetrie in der letzten Spalte. Das einzige dichte Signal, das *Richtungen unterscheidet*, ist der PBRs-Term, und er trägt nur rund 5 % der dichten Reward-

|                           | Budget | Kompass-Zufallslauf |                     | RecurrentPPO v12 |        |        |
|---------------------------|--------|---------------------|---------------------|------------------|--------|--------|
|                           |        | $\varepsilon = 0,4$ | $\varepsilon = 0,9$ | Lauf 1           | Lauf 2 | Lauf 3 |
| 120 ( $3 \times$ Optimum) |        | <b>4 %</b>          | 0 %                 | 0 %              | 2 %    | 0 %    |
| 200                       |        | <b>22 %</b>         | 0 %                 | 0 %              | 4 %    | 0 %    |
| 300                       |        | <b>28 %</b>         | 0 %                 | 2 %              | 10 %   | 6 %    |
| 500                       |        | <b>36 %</b>         | 10 %                | 12 %             | 22 %   | 12 %   |
| 800                       |        | <b>54 %</b>         | 18 %                | 20 %             | 32 %   | 30 %   |
| 1 500                     |        | <b>62 %</b>         | 54 %                | 40 %             | 48 %   | 54 %   |
| 4 000                     |        | 64 %                | <b>88 %</b>         | 64 %             | 78 %   | 74 %   |

Tabelle 19: Evaluation - Erfolgsquote auf Testset A in Abhängigkeit des Episodenbudgets, je Zeile der Höchstwert hervorgehoben, eigene Darstellung.

| Term                                                | Summe je Episode | Anteil (Betrag) |
|-----------------------------------------------------|------------------|-----------------|
| Explorations-Bonus ( $+0,02 \times 203$ neue Tiles) | +4,1             | 16 %            |
| Schritt-Malus ( $-0,01 \times 1329$ Schritte)       | -13,3            | 53 %            |
| Revisit-Penalty                                     | -6,2             | 25 %            |
| <b>PBRs gesamt</b> (gemessen)                       | <b>+1,3</b>      | <b>5 %</b>      |
| davon teleskopierter Anteil $\beta d_0/128$         | +0,8             |                 |
| davon Restterm $\beta(1 - \gamma) \sum_t d_t/128$   | +0,5             |                 |
| Summe der dichten Terme                             | -14,1            |                 |
| Terminal-Reward (sparse)                            | +100             |                 |

Tabelle 20: Evaluation - Bilanz der dichten Reward-Terme über eine mittlere erfolgreiche Episode, eigene Darstellung.

Masse; die übrigen 95 % hängen ausschließlich davon ab, *wie lange* der Agent unterwegs ist und *wie viel Fläche* er dabei abdeckt. Der Terminal-Reward bleibt zwar mit  $100 \cdot \mathbb{E}[\gamma^T] = +37,7$  (diskontiert) das dominierende Zielsignal gegenüber  $-5,3$  dichten Termen, doch über den Verlauf der Episode hinweg sieht der Agent vor allem ein Signal über Weglänge und Flächendeckung, kaum eines über die Richtung zum Ziel. Herumlaufen ist dabei nicht kostenlos, die dichte Bilanz ist mit  $-14,1$  klar negativ; die Schwäche liegt also nicht darin, dass Umherirren belohnt würde, sondern darin, dass der Agent aus dem dichten Signal kaum lernen kann, *wohin* er gehen soll.

Damit ist die zentrale Leistungsaussage dieser Arbeit einzuschränken: Belegt ist, dass der Agent die Aufgabe löst und dabei deutlich über der Zufallsuntergrenze liegt; *nicht* belegt ist, dass er eine Strategie gelernt hat, die über gerichtete Zufallssuche hinausgeht. Diese Einschränkung betrifft auch die Gegenüberstellung in Abschnitt 5.2.1: Wenn eine kompassgetriebene Zufallspolitik 92 % erreicht, ist der Vergleich „LSTM ohne BFS“ gegen „MLP mit BFS“ kein Test der Gedächtnisfähigkeit, sondern misst überwiegend, wie gut die jeweilige Politik den Kompass ausnutzt und wie stark sie in Schleifen gerät. Die notwendige Konsequenz, eine Härtung der Weltgenerierung vor jedem weiteren Training, ist in Abschnitt 6 ausgeführt.

#### 5.2.4 Der Det/Stoch-Gap: Weglängen-Analyse und POMDP-Einordnung

Der Abstand zwischen stochastischer und deterministischer SR ist kein Schwelleneffekt, sondern wächst kontinuierlich mit der Weglänge: Bei 5–15 Feldern erreicht die deterministische Betriebsart 79 %, bei 35–45 Feldern noch 31 %, während die stochastische zwischen 100 % und 82 % bleibt (Abbildung 17; 120 Welten aus dem Validierungsbereich, nach Startdistanz gruppiert). Je länger der Weg ohne direkte Zielsicht (Sichtradius 7), desto mehr Kreuzungsentscheidungen unter unsicherem Belief-State; an jeder liegen die Aktionswahrscheinlichkeiten nahe beieinander, und jede einzelne kann die Argmax-Politik in eine Oszillationsschleife

führen, aus der Sampling entkommt.

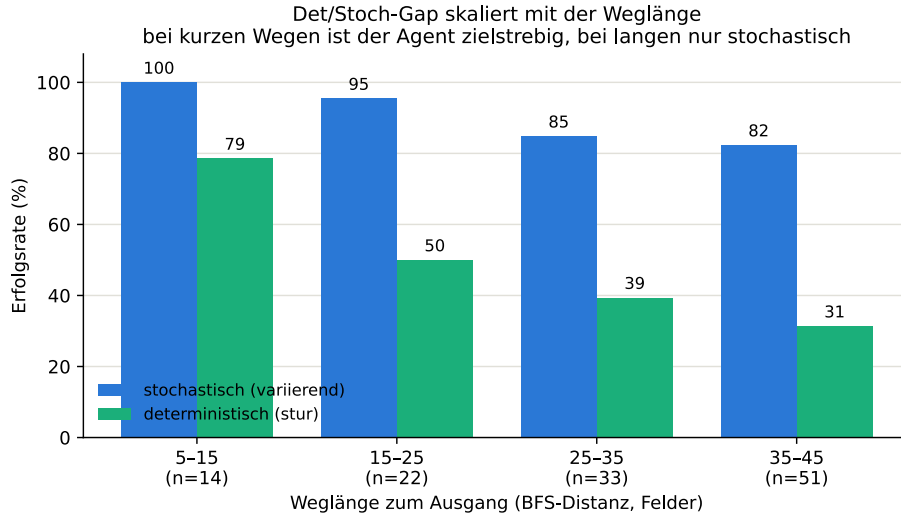


Abbildung 17: Evaluation - Erfolgsquote des finalen v12-Modells nach Startdistanz zum Exit, eigene Darstellung.

Der Gap hat zwei sich ergänzende Ursachen. Die unmittelbare, mechanistische Ursache ist die eben beschriebene Oszillation der Argmax-Politik an gleichwahrscheinlichen Kreuzungsentscheidungen. Theoretisch begrenzt wird das Erreichbare zusätzlich dadurch, dass die Generalisierungsanforderung auf unbekannte Seeds selbst eine Form partieller Beobachtbarkeit induziert (*epistemic POMDP*), deren optimale Politik stochastisch bleibt und die durch mehr Gedächtnis über die aktuelle Episode nicht auflösbar ist [30]; darauf, nicht auf die allgemeinere POMDP-Optimalität stochastischer Politiken [23], stützt sich die Wahl der stochastischen Evaluation als Primärmetrik. Der Gap ist damit teils erwartbar (epistemische Unsicherheit über Seeds), teils ein offenes Defizit des gelernten Belief-States; die Trennung beider Anteile ist mit den vorliegenden Daten nicht möglich (Diagnoseschritt im Ausblick, Abschnitt 6).

### 5.2.5 Versuche zur Schließung des Det/Stoch-Gaps

Um zu prüfen, ob der Gap durch gezielte Eingriffe verkleinerbar ist, wurden drei Varianten gerechnet, die je einen inhaltlichen Faktor gegenüber der v12-Basiskonfiguration (Tabelle 4) verändern: **E1** verstärkt den Critic (`vf_coef` 0,5  $\rightarrow$  1,0), **E2** glättet den Curriculum-Übergang (Phase 3 nur bis Distanz 35 statt 45), **E3** vergrößert das Gedächtnis (LSTM 256  $\rightarrow$  512). Diese drei Läufe sind Einzelmessungen ( $n = 1$ ) gegen eine Baseline aus drei Läufen und damit keine Wirksamkeitsnachweise, sondern eine Vorauswahl: Sie können eine Hypothese schwächen, aber nicht widerlegen; ihre Replikation mit weiteren Seeds steht im Ausblick (Abschnitt 6).

**Kein Arm zeigt einen Hinweis auf Verbesserung.** E1 hebt die deterministische SR (42 % statt 32 % auf A), liegt stochastisch aber niedriger (56 % statt 73 %); beide Abstände bleiben bei  $n = 1$  innerhalb der üblichen Lauf-zu-Lauf-Streuung und sind nicht belastbar. E2 liegt mit 26 % auf A deutlich unter der Baseline, was angesichts des reduzierten Phase-3-Trainingsbudgets dieser Variante nur teilweise ein Faktoreffekt ist. E3 liegt mit 44 % auf A ebenfalls klar unter der Baseline, ist aber ein voller Lauf mit identischem Budget und zeigt in *allen* Curriculum-Phasen Werte auf oder unter der 256er-Baseline bei rund dreifachem Rechenaufwand: Mehr Gedächtnis-*Kapazität* bedeutet in dieser Umgebung jedenfalls nicht mehr Gedächtnis-*Nutzen*, passend zu den bekannten Grenzen rekurrenter Politiken in prozeduraler

| Variante                      | Änderung ggü. Baseline                     | Start   | Test A<br>(stoch/det) | Holdout B<br>(stoch/det) |
|-------------------------------|--------------------------------------------|---------|-----------------------|--------------------------|
| <b>v12</b> (Baseline, $n=3$ ) | – (LSTM 256, <code>vf_coef</code> 0.5)     | Phase 1 | 73/32 %               | 80/42 %                  |
| <b>E1</b> critic              | <code>vf_coef</code> 0.5 $\rightarrow$ 1.0 | Phase 3 | 56/42 %               | 72/36 %                  |
| <b>E2</b> curric              | Phase 3 Exit 25–45 $\rightarrow$ 25–35     | Phase 3 | 26/16 %               | 42/24 %                  |
| <b>E3</b> lstm512             | LSTM 256 $\rightarrow$ 512                 | Phase 1 | 44/14 %               | 58/20 %                  |

Tabelle 21: Evaluation - Setup und Ergebnis der drei Gap-Experimente gegenüber der Baseline, eigene Darstellung.

Navigation, deren Gedächtnisanforderungen über reine Kapazität hinausgehen [15].

Ergänzend wurde geprüft, ob der Gap lediglich ein Kalibrierungsartefakt der Argmax-Auswertung ist. Ein Temperatur-Sweep zwischen reinem Argmax ( $T=0$ ) und vollem Sampling ( $T=1$ ) auf den drei v12-Modellen zeigt einen *monotonen* Verlauf: Mehr Stochastik ist durchweg besser, keine Zwischentemperatur übertrifft das volle Sampling.

| Temperatur                         | Testset A                         | Holdout B                        |
|------------------------------------|-----------------------------------|----------------------------------|
| Argmax ( $T=0$ )                   | $32,0 \pm 6,0$                    | $42,0 \pm 5,3$                   |
| $T = 0,25$                         | $52,0 \pm 0,0$                    | $52,7 \pm 2,3$                   |
| $T = 0,5$                          | $63,3 \pm 5,0$                    | $71,3 \pm 8,1$                   |
| $T = 0,75$                         | $63,3 \pm 9,2$                    | $72,0 \pm 8,0$                   |
| <b>Sampling (<math>T=1</math>)</b> | <b><math>70,7 \pm 10,1</math></b> | <b><math>79,3 \pm 8,1</math></b> |

Tabelle 22: Evaluation - Erfolgsquote der drei v12-Läufe in Abhängigkeit der Sampling-Temperatur, eigene Darstellung.

Der Gap ist damit *kein* Argmax-Artefakt, sondern hat eine der oben diskutierten Ursachen. Bereits  $T=0,5$  holt den Großteil des Argmax-Defizits zurück (A: +31, B: +29 Prozentpunkte) und ist ein geeigneter Betriebspunkt, falls ein weniger zufälliges Verhalten gewünscht ist. Zusammengenommen widersteht der Det/Stoch-Gap dem Tuning einzelner Stellschrauben und ist teils fundamental (POMDP); das harte, fundamentale Langhorizont-Belief-Tracking ist am ehesten über strukturiertes statt größeres Gedächtnis anzugehen (Abschnitt 6).

## 6 Fazit und Ausblick

### 6.1 Ergebnis (gemeinsam)

Die Arbeit hatte zwei Fragen gestellt. Sie fallen unterschiedlich aus, und gerade ihr Verhältnis zueinander ist das eigentliche Ergebnis.

**F1 (Weltgenerator): beantwortet.** Der entwickelte Generator erzeugt aus einem Seed reproduzierbare, chunkweise und damit unbegrenzt erweiterbare Welten. Alle fünf Anforderungen aus Z1 sind nachgewiesen erfüllt (Tabelle 14): Determinismus bis hin zu bit-identischen Trainingspfaden, 100 % Lösbarkeit über 3 150 geprüfte Seeds, eine Zielentfernung, die nach der Umstellung auf das BFS-Laufwegmaß exakt eingehalten wird, ausreichender Durchsatz für Millionen Schritte und ein spielbarer Client auf derselben Simulation. Die Kombination aus Spielbarkeit und Kontrollierbarkeit ist dabei der Punkt, an dem sich der Aufwand eines eigenen Generators gegenüber einem fertigen Benchmark auszahlt (Abschnitt 1.5).

**F2 (Navigation ohne Orakel): nicht entscheidbar.** Über sieben unabhängige Läufe erreicht der Agent  $65,7\% \pm 12,4$  (Testset A) und  $66,9\% \pm 12,8$  (Holdout B) stochastische Erfolgsquote. Das Holdout-Kriterium ( $\geq 60\%$ ) ist damit erfüllt, das Testset-Kriterium ( $\geq 70\%$ ) verfehlt. Diese Zahlen sind jedoch nur gegen die Zufallsuntergrenze von 5–10 % aussagekräftig, nicht gegen eine kompetente ungelernete Referenz: Ein Kompass-Zufallslauf erreicht 92 % und ist bei vergleichbarer Erfolgsquote zugleich dreimal wegeffizienter (Abschnitt 5.2.3). Bei einem Episodenbudget vom Hundertfachen der Optimallänge misst die gewählte Metrik nicht die Fähigkeit, um die es geht, und ein engeres Budget behebt das nicht (Tabelle 19). **F2 ist mit dem vorliegenden Aufbau daher nicht entscheidbar: Die Umgebung erfordert das Gedächtnis nicht, dessen Nutzen sie messen soll.** Dieser Befund betrifft die Interpretation sämtlicher Leistungszahlen der Arbeit, einschließlich der Gegenüberstellung in Abschnitt 5.2.1.

**Das Verhältnis beider Antworten.** Auf den ersten Blick steht ein gelungener Generator neben einem unentschiedenen Lernexperiment. Tatsächlich haben beide dieselbe Ursache. Der Generator erfüllt seine Spezifikation vollständig, und die Spezifikation war unvollständig: Z1 verlangt lösbare und in ihrer Entfernung steuerbare Welten, aber keine, in denen die untersuchte Fähigkeit auch erforderlich ist. Die konkrete Stelle, an der das entsteht, ist im Generator benennbar und inzwischen vermessen: Ohne die abgeschaltete Glättungsstufe stammt die Hindernisverteilung aus unkorreliertem Rauschen und bildet verstreute Einzelhindernisse statt zusammenhängender Barrieren. Der Umwegfaktor liegt bei 1,10, der kürzeste Weg ist also nur zehn Prozent länger als die Luftlinie (Abschnitt 4.1.3). Sackgassen, die Gedächtnis erzwingen würden, entstehen so kaum. Die naheliegende Reparatur, einfach die vorhandene Glättungsstufe einzuschalten, scheitert an derselben niedrigen Basisdichte und macht die Welt noch leerer (Tabelle 10).

Damit ist das Ergebnis dieser Arbeit weder eine Aussage über den Agenten noch eine über den Generator, sondern eine über deren Zusammenspiel: **Die Schwierigkeit einer Lernaufgabe ist eine Eigenschaft der Weltgenerierung und muss dort spezifiziert und geprüft werden, bevor ein Lernverfahren darauf angesetzt wird.** Die Referenzpolitik aus Abschnitt 5.2.3 ist genau das Instrument dafür, und sie kostet Sekunden statt Trainingsstunden. Dass dieser Zusammenhang überhaupt sichtbar wurde, setzte beide Projektteile voraus: Ohne den eigenen, in seinen Parametern zugänglichen Generator wäre die Ursache nicht lokalisierbar gewesen, ohne das RL-Experiment wäre sie nie aufgefallen.

## 6.2 Beiträge (gemeinsam)

**Technisch.** Entstanden ist ein vollständiges Spiel, das zugleich als Experimentierplattform dient: ein C++-Kern mit deterministischer prozeduraler Weltgenerierung (Biomfeld aus interpoliertem Wertrauschen, biomabhängige Basisbelegung, zelluläre Glättungsstufe, BFS-basierte Exit-Platzierung mit Lösbarkeitsgarantie), ein spielbarer raylib-Client mit KI-Zuschaumodus und eine pybind11-Kapselung als Gymnasium-Umgebung, dazu Curriculum-Trainingspipeline, standardisierte Evaluationsskripte und Live-Monitoring (Abschnitt 4.1 und 4.2). Die deterministische Reproduzierbarkeit pro Seed und die durchgängige Experiment-Protokollierung waren die Voraussetzung für die im Projektverlauf durchgeführten Root-Cause-Analysen (Tabelle 11) und sind für Folgearbeiten wiederverwendbar.

Der praktische Wert dieser Plattform liegt weniger in den einzelnen Verfahren, die sämtlich Stand der Technik sind, als in ihrer Zugänglichkeit: Weil Wandanteil, Glättung, Distanzmaß und Sicherheitsnetze einzeln schaltbar sind, ließ sich die Geometrie der Welten in Stunden vermessen statt sie zu erraten. Genau das hat die zunächst geplante Härtung über die Glättungsstufe widerlegt, bevor eine einzige Trainingsstunde darauf verfiel (Tabelle 10). Ein fremder Benchmark hätte diese Sonde nicht zugelassen.

**Inhaltlich.** Ein rekurrenter Agent ohne BFS-Gradientenfeatures erreicht mindestens dieselbe stochastische Leistung wie ein MLP *mit* diesen Features; der LSTM-Zustand ersetzt die vorberechnete lokale Pfadinformation. Diese Aussage ist doppelt begrenzt: durch die Einschränkungen der Gegenüberstellung (Tabelle 15) und dadurch, dass der globale Luftlinien-Kompass in beiden Bedingungen erhalten bleibt. Ersetzt wird also die *lokale* Wegweiserinformation, nicht jede globale Zielinformation.

**Methodisch.** Die Zwischenauswertung über drei Läufe hatte 73,3 % / 80,0 % ergeben und beide Kriterien scheinbar erfüllt. Die Aufstockung auf sieben Läufe senkte beide Werte deutlich. Ein auf drei Läufen berichtetes „beide Ziele erfüllt“ wäre nicht falsch gemessen, aber nicht haltbar gewesen. Zusammen mit dem Baseline-Befund reproduziert die Arbeit damit zweimal am eigenen Gegenstand, wovon Henderson et al. [7] warnen: zu wenige Läufe und ein fehlender Referenzpunkt führen unabhängig voneinander zu Fehlschlüssen.

## 6.3 Einschränkungen (gemeinsam)

1. **Die Erfolgsmetrik ist nicht trennscharf** und wird von einer ungelerten Heuristik übertroffen (Abschnitt 5.2.3). Dies ist die schwerwiegendste Einschränkung, weil sie alle übrigen Leistungsaussagen relativiert.
2. **Das Testset-Kriterium wird verfehlt** (65,7 % statt  $\geq 70$  %); nur das Holdout-Kriterium ist erfüllt.
3. **Der Det/Stoch-Gap bleibt bestehen** (29,1 % bzw. 32,6 % deterministisch). Er wächst mit der Weglänge und ist monoton in der Sampling-Temperatur, also kein Argmax-Artefakt. Theoretisch erwartbar ist nur der Anteil aus der epistemischen Unsicherheit über den Seed [30]; der Rest ist ein Defizit des gelernten Belief-States. Die drei Versuche zu seiner Schließung sind Einzelläufe und belegen nicht, dass er tuning-resistent *ist*.
4. **Begrenzte statistische Präzision:** Die Streuung über Läufe beträgt rund  $\pm 12$  Punkte. Das wirkt in beide Richtungen: Das 95 %-Konfidenzintervall auf Holdout B reicht bis 55,0 % hinab und damit unter die im Mittel erfüllte Schwelle; das Intervall auf Testset A reicht bis 77,2 % hinauf und schließt die 70 %-Schwelle ein. Das Testset-Kriterium ist im Punktschätzer verfehlt, nicht aber signifikant verfehlt. Für den Hauptbefund der



Arbeit ist das ohne Belang, da er nicht an dieser Schwelle hängt (Abschnitt 5.2.3). Die Aufstockung von  $n = 3$  auf  $n = 7$  war eine Ressourcenentscheidung; der naheliegende und rechnerisch billige nächste Schritt – ein voller Lauf kostet auf CPU nur wenige Stunden – ist eine Erhöhung auf  $n \geq 15$ , die das Konfidenzintervall spürbar verengen und die Schwellenfrage entscheiden würde. Sie steht aus und ist im Ausblick vermerkt (Abschnitt 6).

5. **Ein einziger Generator und keine quantifizierte Baseline nicht-rekurrenter Verfahren** (DQN, A2C) auf der finalen Umgebung: Der frühe DQN-Anlauf scheiterte vor der v11-Revision und wurde nicht wiederholt.
6. **Selektion und Berichterstattung verwenden verschiedene Betriebsarten.** Ausgewählt wird das nach *deterministischer* Validierungs-SR beste Modell (Gating-Metrik der Phasen 3 und 4), berichtet und als Primärmetrik begründet wird die *stochastische* SR. Bei einem Det/Stoch-Gap von über 30 Punkten, also dem zentralen Untersuchungsgegenstand, wird damit auf die eine Betriebsart selektiert und die andere gemessen. Das ist eine bewusste Entscheidung, weil die stochastische Auswertung theoretisch begründet ist (Abschnitt 5.2.4) und die deterministische in den frühen Phasen kein Signal trägt (Tabelle 3); der so ausgewählte Checkpoint muss aber nicht der stochastisch beste sein. Eine Selektion auf der stochastischen Validierungs-SR steht als Gegenprobe aus (Abschnitt 6).

## 6.4 Ausblick

Die Reihenfolge der folgenden Punkte ergibt sich aus dem Befund von Abschnitt 5.2.3: Solange die Aufgabe die untersuchte Fähigkeit nicht erfordert, ist jede weitere Optimierung des Agenten nicht interpretierbar. Die Reparatur des Messregimes steht deshalb vor allem anderen.

### 6.4.1 Härtung der Weltgenerierung (Laurin)

Der Wandanteil wird derzeit pro Biom fest vorgegeben (7 % im Wald bis 25 % im Bergland) und stammt aus unkorreliertem Hash-Rauschen; die Hindernisse sind dadurch überwiegend verstreute Einzeltiles statt zusammenhängender Barrieren (Abschnitt 4.1.3). Nicht der Wandanteil allein ist deshalb der wirksame Hebel, sondern die *Korreliertheit* der Verteilung. Der naheliegende Weg dorthin ist versperrt, und zwar gemessen: Die implementierte Glättungsstufe einzuschalten senkt die Waddichte auf 0,037 und den Umwegfaktor auf 1,001, und jede Regelvariante, die den Umweg spürbar hebt, zerstört die Lösbarkeit, im Extremfall auf 3 von 50 Welten (Tabelle 10). Eine Härtung verlangt deshalb drei Eingriffe im Code statt eines Schalters: eine höhere und korrelierte Basisdichte in `World::sampleBaseTile()`, eine aktive Konnektivitätsvalidierung (`enableFloodFillValidation`, `enableMacroGraphPrecheck`) und eine Neuvalidierung nach jeder Automateniteration. Zielgröße bleibt nicht ein bestimmter Wandanteil, sondern ein Betriebspunkt, an dem der Kompass-Zufallslauf einbricht, die Lösbarkeit aber bei 100 % bleibt. Das Vorgehen ist zweistufig: erst die Baseline-Politik als Kalibrierinstrument einsetzen, dann trainieren. Ein Trainingslauf ohne diese Vorabprüfung wäre erneut nicht interpretierbar.

### 6.4.2 Weiterentwicklung des RL-Trainings (Florian)

1. **Statistische Absicherung des Endergebnisses.** Bei einer Streuung von rund  $\pm 12$  Punkten über Läufe überlappt das Konfidenzintervall die Erfolgsschwelle (Abschnitt 5.2.2). Da ein voller Trainingslauf auf CPU nur wenige Stunden kostet, ist eine Erhöhung von  $n = 7$  auf  $n \geq 15$  der billigste Weg, die Schwellenfrage auf

Testset A statistisch zu entscheiden; sie ist rein rechenzeitgebunden und erfordert keine Codeänderung.

2. **Nachschärfung des dichten Reward-Signals.** Die Bilanz in Tabelle 20 zeigt, dass nur rund 5 % der dichten Reward-Masse überhaupt Richtungsinformation tragen. Naheliegend sind eine Anhebung von  $\beta$  bzw. eine kleinere Normierungskonstante, eine Deckelung des Explorations-Bonus über die Episode und die Überprüfung des Revisit-Penalty, der als letzter verbliebener Term des früher verworfenen „Straf-Stackings“ mit  $-6,2$  je Episode überraschend schwer wiegt, deutlich schwerer als der Explorations-Bonus, den er damit faktisch neutralisiert. Da PBRS policy-invariant ist, die übrigen Terme aber nicht, verschiebt die aktuelle Gewichtung das Optimum weg von der eigentlichen Aufgabe.
3. **Billige Diagnosen vor jedem Architekturwechsel.** (a) Eine lineare Probe auf dem LSTM-Hidden-State, die Exit-Richtung bzw. BFS-Distanz vorhersagt: Kollabiert sie auf langen Wegen, liegt ein Belief-Tracking-Limit vor, sonst liegt das Problem in der Politik. (b) Entropie-Annealing in Phase 4 mit Modellselektion auf der deterministischen Erfolgsquote. (c) One-Hot- oder Embedding-Kodierung des lokalen Grids statt der bisherigen ordinalen Tile-IDs; dass das Grid kaum genutzt wird, legt die Baseline-Analyse nahe. (d) Replikation von E1 und E3 mit je zwei weiteren Seeds. (e) Wiederholung des Baseline-Vergleichs mit abgeschalteter Early-Stop-Truncation, um auszuschließen, dass diese explorationsabhängige Abbruchbedingung (Abschnitt 3.2.3) den Vergleich beeinflusst; bei den trainierten Läufen greift sie in unter 3 % der Episoden, für die Referenzpolitiken ist sie bislang nicht ausgezählt.
4. **Restpotenzial in RecurrentPPO.** Ein Auxiliary-Loss-Kopf, der aus dem Hidden-State die BFS-Distanz vorhersagt, formt den Belief-State explizit und nutzt das Orakel als Trainings-, nicht als Testzeitsignal. Ergänzend Burn-in für Sequenzstarts nach R2D2 [13], temperaturkalibrierte Evaluation ( $T = 0,5$  ohne Neutraining) sowie vollwertiges Prioritized Level Replay [17] und Self-Imitation [35].
5. **Architekturvergleich als Folgearbeit.** Der Wechsel auf Transformer-basierte In-Context-Verfahren (z. B. AMAGO [36]) ist die meistgenannte Richtung, hier aber bewusst nur eine Hypothese. Denn erstens ist nicht belegt, dass der Gap ein Kapazitätsproblem ist: Eine unter hoher Entropie trainierte Politik kann ihr Restrauschen zum Verlassen von Schleifen nutzen, ohne je einen Anreiz für eine deterministisch tragfähige Aktionswahl gehabt zu haben. Zweitens bleiben rekurrente Modelle im breitesten verfügbaren POMDP-Vergleich gegenüber den dort geprüften Attention-Modellen konkurrenzfähig; zu beachten ist, dass es sich dabei um *lineare* Transformer-Varianten handelt und nicht um volle Softmax-Attention, wie AMAGO sie verwendet [14]. Die eigene E3-Ablation zeigte, dass mehr Gedächtniskapazität hier schadet. Erst wenn Schritt 2 ein Kapazitätslimit nachweist, wird der Wechsel zur begründeten nächsten Frage; dann als kontrollierter Vergleich auf demselben Generator und Protokoll.

## Literatur

- [1] Noor Shaker, Julian Togelius, and Mark J. Nelson. *Procedural Content Generation in Games*. Computational Synthesis and Creative Systems. Springer, 2016.
- [2] Julian Togelius, Georgios N. Yannakakis, Kenneth O. Stanley, and Cameron Browne. Search-based procedural content generation: A taxonomy and survey. *IEEE Transactions on Computational Intelligence and AI in Games*, 3(3):172–186, 2011. <https://ieeexplore.ieee.org/document/5963131>.
- [3] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2 edition, 2018. <http://incompleteideas.net/book/the-book-2nd.html>.
- [4] Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research (JAIR)*, 76:201–264, 2023. <https://arxiv.org/abs/2111.09794>.
- [5] Sebastian Risi and Julian Togelius. Increasing generality in machine learning through procedural content generation. *Nature Machine Intelligence*, 2(8):428–436, 2020. <https://www.nature.com/articles/s42256-020-0208-5>.
- [6] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International conference on machine learning*, pages 2048–2056. PMLR, 2020. <https://arxiv.org/abs/1912.01588>.
- [7] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018. <https://arxiv.org/abs/1709.06560>.
- [8] Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, et al. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. *Advances in Neural Information Processing Systems*, 36, 2023. <https://arxiv.org/abs/2306.13831>.
- [9] Danijar Hafner. Benchmarking the spectrum of agent capabilities. *arXiv preprint arXiv:2109.06780*, 2021. <https://arxiv.org/abs/2109.06780>.
- [10] Lawrence Johnson, Georgios N. Yannakakis, and Julian Togelius. Cellular automata for real-time generation of infinite cave levels. In *Proceedings of the 2010 Workshop on Procedural Content Generation in Games (PCGames)*, pages 1–4. ACM, 2010.
- [11] OpenAI. Procgen benchmark: Procedurally-generated game-like gym environments. Quellcode-Repository, 2020. <https://github.com/openai/procgen>.
- [12] Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable mdps. *arXiv preprint arXiv:1507.06527*, 2015. <https://arxiv.org/abs/1507.06527>.
- [13] Steven Kapturowski, Georg Ostrovski, John Quan, Rémi Munos, and Will Dabney. Recurrent experience replay in distributed reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2019.
- [14] Steven Morad, Ryan Kortvelesy, Matteo Bettini, Stephan Liwicki, and Amanda Prorok. POPGym: Benchmarking partially observable reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2023. <https://arxiv.org/abs/2303.01859>.

- [15] Marco Pleines, Matthias Pallasch, Frank Zimmer, and Mike Preuss. Generalization, mayhems and limits in recurrent proximal policy optimization. *arXiv preprint arXiv:2205.11104*, 2022. <https://arxiv.org/abs/2205.11104>.
- [16] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th Annual International Conference on Machine Learning (ICML)*, pages 41–48, 2009. <https://dl.acm.org/doi/10.1145/1553374.1553380>.
- [17] Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In *International Conference on Machine Learning (ICML)*, pages 4940–4950. PMLR, 2021. <https://arxiv.org/abs/2010.03934>.
- [18] Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*, pages 2778–2787. PMLR, 2017. <https://arxiv.org/abs/1705.05363>.
- [19] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations (ICLR)*, 2019. Preprint 2018, <https://arxiv.org/abs/1810.12894>.
- [20] Ken Perlin. An image synthesizer. In *Proceedings of the 12th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 287–296. ACM, 1985.
- [21] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.
- [22] Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.
- [23] Satinder P Singh, Tommi Jaakkola, and Michael I Jordan. Learning without state-estimation in partially observable markovian decision processes. In *Proceedings of the Eleventh International Conference on Machine Learning (ICML)*, pages 284–292, 1994. <https://www.cs.utexas.edu/~shivaram/readings/b2hd-SinghJJ1994.html>.
- [24] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015. <https://www.nature.com/articles/nature14236>.
- [25] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016. <https://arxiv.org/abs/1506.02438>.
- [26] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, pages 1928–1937. PMLR, 2016. <https://arxiv.org/abs/1602.01783>.
- [27] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017. <https://arxiv.org/abs/1707.06347>.

- [28] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. <http://jmlr.org/papers/v22/20-1364.html>.
- [29] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [30] Dibya Ghosh, Jad Rahme, Aviral Kumar, Amy Zhang, Ryan P Adams, and Sergey Levine. Why generalization in RL is difficult: Epistemic POMDPs and implicit partial observability. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021. <https://arxiv.org/abs/2107.06277>.
- [31] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the Sixteenth International Conference on Machine Learning (ICML)*, pages 278–287, 1999.
- [32] Norbert L. Kerr. HARKing: Hypothesizing after the results are known. *Personality and Social Psychology Review*, 2(3):196–217, 1998. [https://journals.sagepub.com/doi/10.1207/s15327957pspr0203\\_4](https://journals.sagepub.com/doi/10.1207/s15327957pspr0203_4).
- [33] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d’Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (a report from the neurips 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021. <https://arxiv.org/abs/2003.12206>.
- [34] Guy L. Steele, Doug Lea, and Christine H. Flood. Fast splittable pseudorandom number generators. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA)*, pages 453–472. ACM, 2014.
- [35] Junhyuk Oh, Yijie Guo, Satinder Singh, and Honglak Lee. Self-imitation learning. In *International Conference on Machine Learning (ICML)*, pages 3878–3887. PMLR, 2018. <https://arxiv.org/abs/1806.05635>.
- [36] Jake Grigsby, Linxi Fan, and Yuke Zhu. AMAGO: Scalable in-context reinforcement learning for adaptive agents. In *International Conference on Learning Representations (ICLR)*, 2024. <https://arxiv.org/abs/2310.09971>.